Network Analysis

Why networks are so important?

- They are everywhere and constantly growing
- Their topology is complex, with structural properties that can be studied to understand their evolution
- Nowadays we have the computational power to study them, but we need proper tools

Example: WWW

Direct graph: nodes represent Web pages, arcs represent hyperlinks Considering the Web as a graph is fundamental in the field of Information Retrieval Search engines use the Web graph to rank answers to queries.

Example: Internet

Built by human beings, collection of routers connected by physical links. Without a central authority. Whenever a new router is connected, distance and bandwidth are considered for paths computing The study of the Internet topology is crucial to understand its robustness in presence of failures.

Example: power grid, transportation and road networks

Built by human beings, they can be involved into random failures or targeted attacks Faults can generate cascading failures: a single breakdown can induce failures on the connected nodes with catastrophic consequences (es. electrical blackout or air traffic jam) The topology of the network can influence the size and the severity of the fault.

Example: biological networks

Built by nature, proteins can be seen as nodes of a complex network, connected by links if they mutually interact Robustness of these “life maps” in presence of failures determines our survival ability in presence of diseases The presence of strongly connected proteins (hub) allows researchers to study effective treatments against diseases.

Example: disease gene network

Nodes are diseases connected if they have a common genetic origin, the map was created to illustrate the genetic interconnectedness of apparently distinct diseases.

Example: food web

Built by nature, species are connected in a prey-predator relationship Usually represented as directed graphs The study of these networks allows one to understand ecosystems dynamics An ecosystem can survive if species are cut out at random, in case of removal of highly connected species (hub), the ecosystem can collapse.

Example: social networks

Connecting people (relatives, friends, colleagues, …) Fundamental to understand and anticipate the diffusion of ideas, innovations, trends, biological and computer viruses.

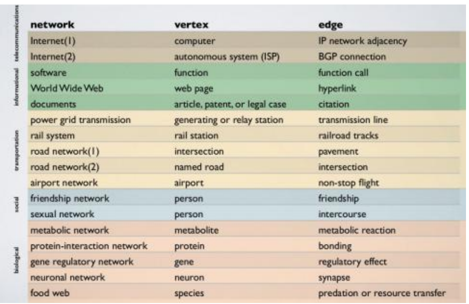
Different networks ...

Some have been autonomously created by nature, are decentralized, and exist by million of years Others have been built by humans In the last two decades, researchers started studying networks, proving that they share similar structures, independently from their age, function, purpose This structure is not random: the nature does not play dices, neither human beings who build networks!

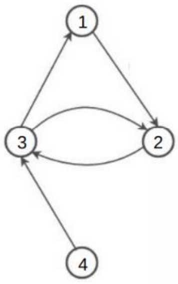
Basics on graphs

Graphs allow us to model networks of different types, providing an abstract view of the domain under analysis In this way it is possible to see the “overall picture” without considering details which are irrelevant for the analysis.

A network – also called a graph in the mathematical literature – is a collection of vertices joined by edges – nodes, links in computer science – sites, bonds in physics – actors, ties in sociology.



	network	vertex	edge
telecommunications	Internet(1)	computer	IP network adjacency
	Internet(2)	autonomous system (ISP)	BGP connection
informational	software	function	function call
	World Wide Web	web page	hyperlink
	documents	article, patent, or legal case	citation
transportation	power grid transmission	generating or relay station	transmission line
	rail system	rail station	railroad tracks
	road network(1)	intersection	pavement
	road network(2)	named road	intersection
	airport network	airport	non-stop flight
social	friendship network	person	friendship
	sexual network	person	intercourse
biological	metabolic network	metabolite	metabolic reaction
	protein-interaction network	protein	bonding
	gene regulatory network	gene	regulatory effect
	neuronal network	neuron	synapse
	food web	species	predation or resource transfer



Graphs can be

- directed or
- undirected weighted

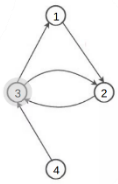
Many of the graphs we will study have edges that form simple on/off connections between vertices of the same type (one-mode) In some situations, it is also useful to represent edges as having a strength or weight, usually a real number.

From pictures to numbers: the first impression of a graph is often visual since network diagrams can provide insights on the underlying network data. But it is not possible to formally compare networks using their pictures, specially in the case of large networks.

From pictures to numbers: Adj matrix Adjacency matrix A (v1)

In one-mode networks rows and columns represent the same set of nodes In the of case of weighted graphs, the elements of the matrix are the weights.

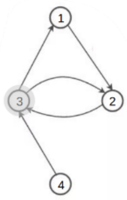
$$\begin{vmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{vmatrix}$$



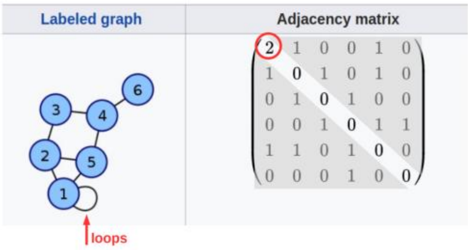
From pictures to numbers: Adj matrix Adjacency matrix A (v2)

Note that in many books A_{ij} = 1 if there is an edge from j to i

$$\begin{vmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{vmatrix}$$



From pictures to numbers: Adj matrix Undirected graph with self-edge



From pictures to numbers: Adj matrix

Drawbacks:

- A matrix is less intuitive than a picture
- It is difficult to see emergent social structure or the overall topology, but it can be used to measure them
- Expensive way to store data (N^2 elements for N nodes, since also the lack of a link is represented with a 0 value)

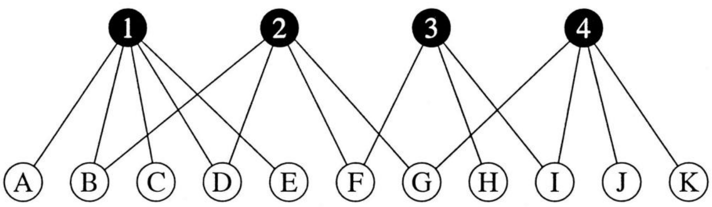
From pictures to numbers: Edge list

The graph has n=4 vertices labeled 1,2,3,4
Edges (1,2), (2,3), (3,1), (3,2), (4,3)
Edge lists store the edges: for each value > 0 in the adj matrix there is a row in the edge list
1 2
2 3
3 1
3 2
4 3
They can be cumbersome for some mathematical reasoning.

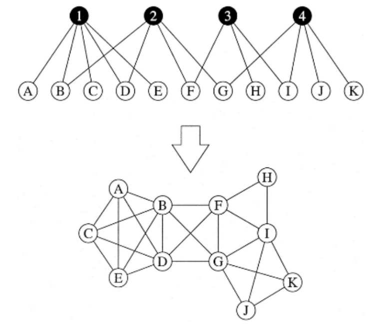
From pictures to numbers: Adjacency list

Similar to the edge list but each node is listed as a row
1 2
2 3
3 1 2
4 3
Also XML files can be adopted to represent networks.

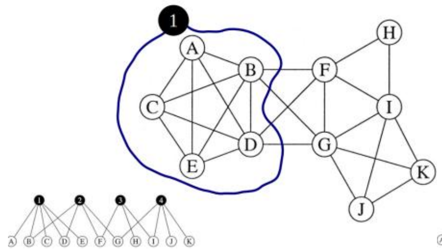
Bipartite graph, also called two-mode graph

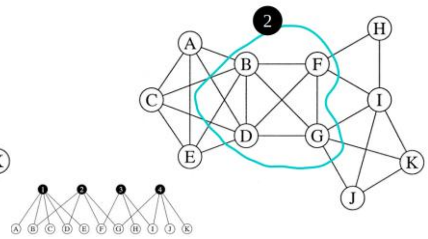


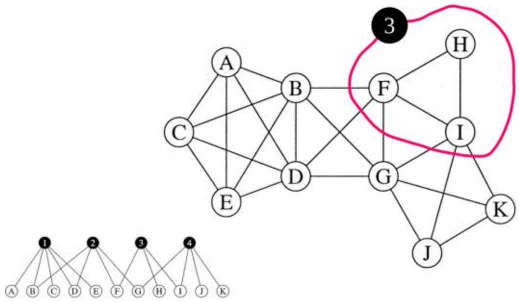
Two kinds of vertices, no within-in type edges Incidence (or affiliation) matrix (rectangular) Often the smaller set of nodes (black) is arranged as columns, the larger set (white) as rows (11 X 4).

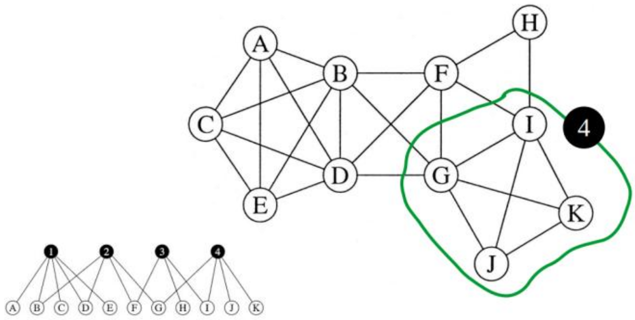


Each group in the two-mode graph results in a cluster of vertices in the one-mode projection that are all connected to each other (a "clique" in network jargon)

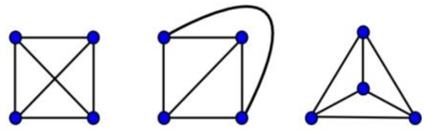


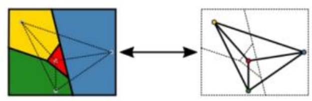






A **planar graph** can be drawn in such a way that no edges cross each other





Most of the networks we will study are not planar but there are some some important examples

- All trees are planar
- The river network is planar (rivers never cross one another; they only flow together)
- The road network is a (good) approximation of a planar network due to the presence of bridges in which roads meet without intersecting

A **path** in a graph is any sequence of vertices connected by edges.
The **length** of a path is the number of edges traversed along the path (number of "hops").
Paths lengths in a graph grow "slowly" with respect to the size N of the network, in many examples we will see logarithmic growth (log N).

A **geodesic path** (shortest path) is a path between two vertices such that no shorter paths exist.
Shortest paths are not necessarily unique since it is possible to have two or more paths of equal length between a given pair of vertices.
If two vertices are not connected, the shortest path can be considered infinite or omitted from the computation.

Graph eccentricity: vertex-centric measure, it computes the maximum distance of a specific vertex with respect to the other vertices within the graph.
Graph diameter: graph-centric measure, it focuses on the entire graph and considers the farthest possible distance between any two vertices. It represents the maximum eccentricity among all vertices in the graph.

The number of paths of a given length n can be computed using the adjacency matrix
An entry equal to 1 in A_{ij} represents a path of length 1 between i and j

To compute the number of paths of length 2, the matrix of length 1 must be multiplied by itself, and the product matrix is the matrix representation of paths of length 2.
In general, to generate the matrix of paths of length n, take the matrix of paths of length n-1, and multiply it with the matrix of paths of length 1 (adjacency matrix).

Eulerian path is a path that traverses each edge in a graph only once.
Hamiltonian path is a path that traverses each node in a graph exactly once.

Graph measures

Networks can be "measured".

Some measures provide indicators to know:

- The importance of a node or an area in the network
- The distance among nodes or areas in the network
- Cohesion degree of an area in the network

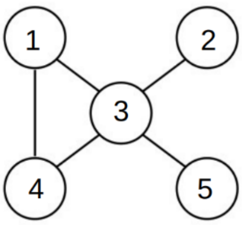
For example, central individuals in a social network generally correspond to the influentials who provide value to the entire network

Degree centrality
The simplest centrality measure in a graph is node degree and it can be applied to directed and undirected graphs:
- indegree: number of links entering node i
- outdegree: number of links leaving node i
- degree: number of links of node i

Provides an indication of the ability of a node engaging in a direct relationship with the other nodes
Local measure computed from immediate connections.

Degree centrality: undirected graph

Given the adjacency matrix of an undirected graph, the node degree is a sum over either the rows or the columns of the matrix.



	1	2	3	4	5
1			1	1	
2			1		
3	1	1		1	1
4	1		1		
5			1		

k_i is the degree of node i

The total number of links can be expressed as

$$L = \frac{1}{2} \sum_{i=1}^{N} k_i$$

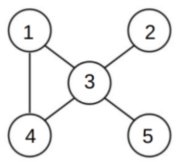
Average degree

$$\langle k \rangle = \frac{1}{N} \sum_{i=1}^{N} k_i = \frac{2L}{N}$$

$$k_i = k_i^{in} + k_i^{out}$$

L = ?

<k> = ?



	1	2	3	4	5
1			1	1	
2			1		
3	1	1		1	1
4	1		1		
5			1		

Total number of links

$$L = \sum_{i=1}^{N} k_i^{in} = \sum_{i=1}^{N} k_i^{out}$$

Average degree

$$\langle k^{in} \rangle = \frac{1}{N} \sum_{i=1}^{N} k_i^{in} = \langle k^{out} \rangle = \frac{1}{N} \sum_{i=1}^{N} k_i^{out} = \frac{L}{N}$$

The maximun number of possible edges in a simple undirected graph is ½ (N)(N-1) (clique).
The density ρ of a graph is the fraction of these edges really present in the network.

ρ = L / (½ (N)(N-1)) = 2L / N (N – 1)

ρ = <k> / (N-1)

0 <= ρ <= 1

If ρ → costant when N → ∞ **dense graph**

If ρ → 0 when N → ∞ **sparse graph**

The maximun number of possible edges in a simple undirected graph is ½ (N)(N-1) (clique).
The density ρ of a graph is the fraction of these edges really present in the network.

0 <= ρ <= 1

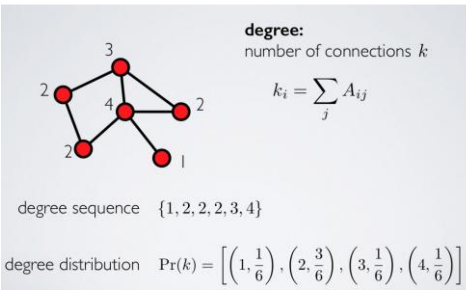
If ρ → costant when N → ∞ **dense graph**

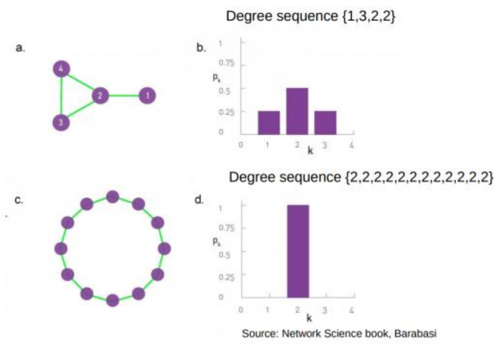
If ρ → 0 when N → ∞ **sparse graph**

Degree distribution pk

Provides the probability that a randomly selected node in the graph has degree k.
For a graph with N nodes the degree distribution is the normalized histogram, where Nk is the number of degreek nodes.

$$p_k = \frac{N_k}{N}$$

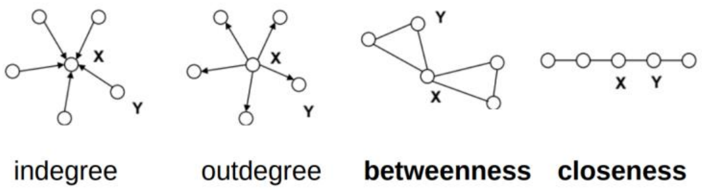




* *Trivial here since we can see the graph*

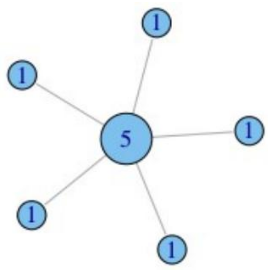
Different notions of centrality

In each of the following networks, X has higher centrality than Y according to a particular measure



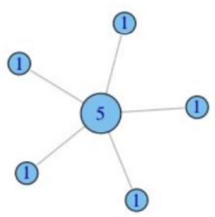
Putting values of degrees...

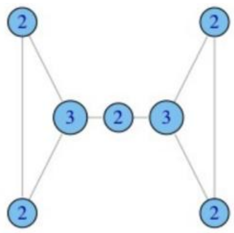
Undirected degree, e.g., nodes with more friends are more central



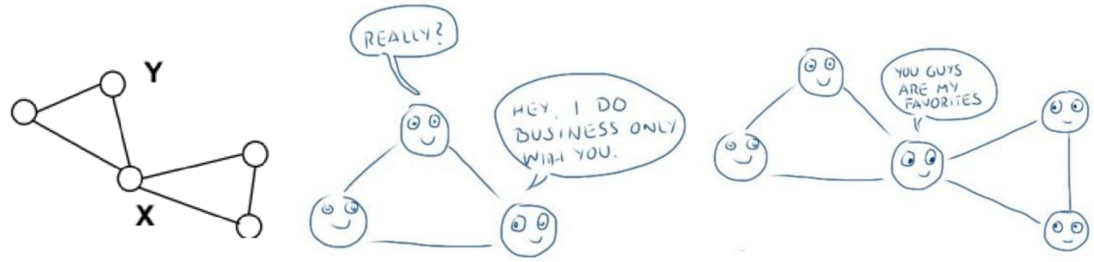
What does degree not capture?

In what ways does degree fail to capture centrality in the following graphs?





Brokerage not captured by degree



Betweenness: capturing brokerage

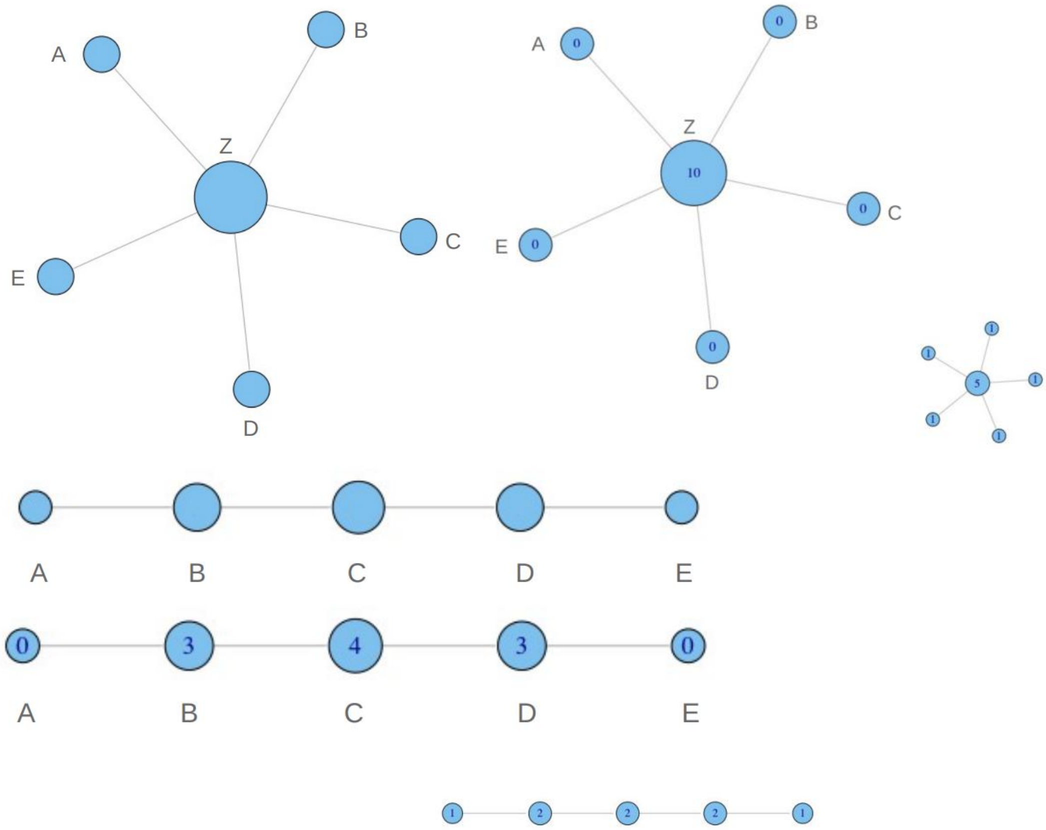
Intuition: how many pairs of individuals would have to go through you in order to reach one another in the minimum number of hops? Measure from the entire graph.

Betweenness: definition

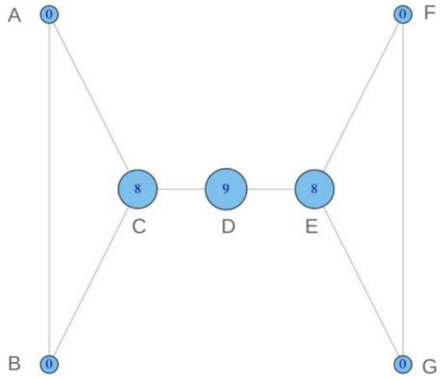
$$B(i) = \sum_{j<k} g_{jk}(i)$$

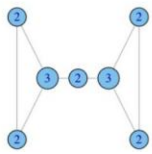
Number of geodesic (shortest) paths from j to k passing through vertex i.

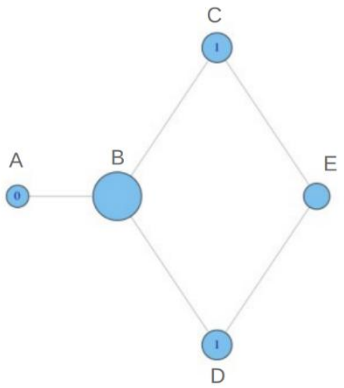
Betweenness: toy example



- A lies between no two other vertices
- B lies between A and 3 other vertices: C, D, and E
- C lies between 4 pairs of vertices (A,D),(A,E),(B,D),(B,E)
- note that there are no alternate paths for these pairs to take, so C gets full credit







Why do C and D each have betweenness 1?
They are both on shortest paths for pairs (A,E), and (B,E), and so must share credit: ½+½ = 1

Betweenness: normalized definition

$$B(i) = \sum_{j<k} g_{jk}(i)/g_{jk}$$

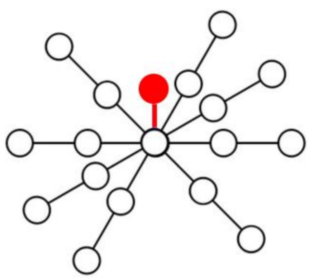
Where g_{jk} = the number of shortest paths connecting jk
$g_{jk}(i)$ = the number that actor *i* is on.

Usually normalized by:

$$\frac{B(i)}{[(N-1)(N-2)/2]}$$ *number of pairs of vertices excluding the vertex i*

Closeness
What if it is not so important to have many direct friends?
Or be "between" others
...but one still wants to be in the "middle" of things, not too far from the center

Need not to be in a brokerage position



Closeness: definition

Closeness measures the mean distance from a vertex to the other vertices.
Given d(i,j), length of a shortest path from i to j.

$$\langle L_i \rangle = 1/N * \sum_{j=1}^{N} d(i,j)$$

$\langle L_i \rangle$ is low for vertices which are close (small d(i,j) values) These vertices can have a better access to information or more direct influence on other vertices Usually, for closeness the inverse of $\langle L_i \rangle$ is considered.

$$C(i) = 1 / \langle L_i \rangle = N / \sum_{j=1}^{N} d(i,j)$$

C(i)is the closeness centrality, often used in social and other network studies but has some problems. One issue is that the difference between the values of vertices closeness in a network is small (due to the logarithmic growth of shortest paths).
Moreover, even small fluctuations in the structure of the network can change the order of the values substantially.
* Some authors use N-1 instead of N

A better solution is to redefine closeness in terms of the harmonic mean distance between vertices, i.e., the inverse of the average of the inverse distances

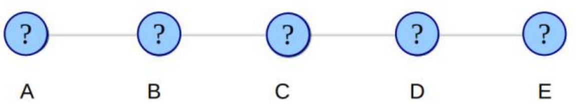
$$H = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \cdots + \frac{1}{x_n}} = \frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}} = \left(\frac{\sum_{i=1}^{n} x_i^{-1}}{n} \right)^{-1}$$

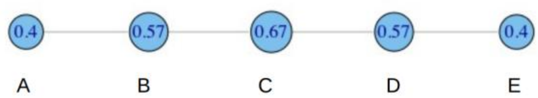
Armonic mean definition from Wikipedia

$$C(i) = (N-1) * \sum_{\substack{j=1 \\ j \neq i}}^{N} \frac{1}{d(i,j)}$$

We must omit d(i,i) = 0 and therefore we have only N-1 terms in the sum
“Closer” nodes are more important in this formula

Closeness: toy example

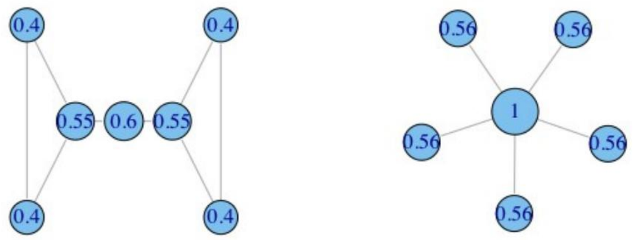




$$C(A) = \left[\frac{\sum_{j=1}^{N} d(A,j)}{N-1} \right]^{-1} = \quad ???????$$

$$C(A) = \left[\frac{\sum_{j=1}^{N} d(A,j)}{N-1} \right]^{-1} = \left[\frac{1+2+3+4}{4} \right]^{-1} = \left[\frac{10}{4} \right]^{-1} = 0.4$$

Nodes with closeness close to 1 have short paths towards all the other nodes.

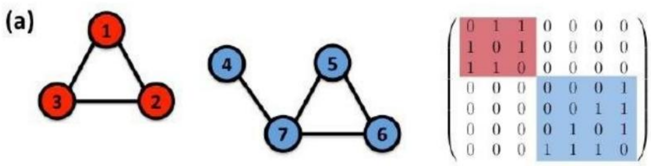


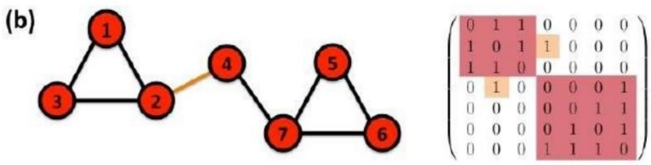
If we consider links in the network like channels in which information flows, nodes with high closeness can access to information as well as spread information very fast.

Take away points

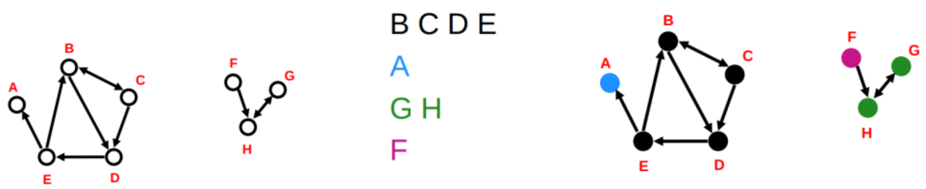
Closeness measures the mean distance of a vertex to other vertices: nodes with high closeness have better access to information or more direct influence on other vertices.
Betweenness measures the extent to which a vertex lies on paths between other vertices: nodes with high betweenness may have considerable influence in a network by virtue of the information they control. Their removal may disrupt communication.
Moreover, different definitions of these (and other) metrics have been proposed… we are not interested in absolute values but in the relative ranking among nodes in a network.

Is everything connected?
If there is a path for each pair of nodes, the network is connected, but this is not always the case …



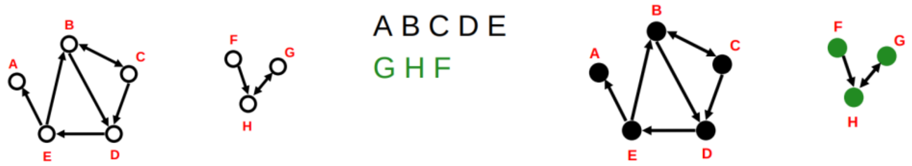


Connected components
SCC Strongly Connected Components
Each node within the component can be reached from every other node in the component by following directed links.



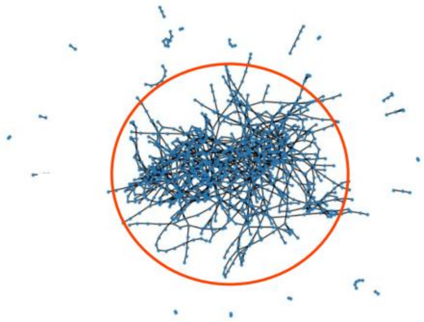
WCC Weakly Connected Components

Weakly connected components: every node can be reached from every other node by following links in either direction



The giant component

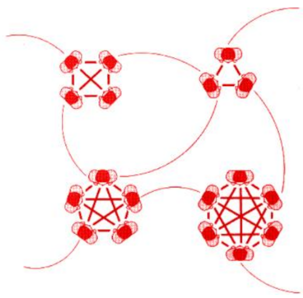
If the largest component encompasses a significant fraction of the graph, it is called the giant component.



In real-world undirected networks typically there is a large component that fills most of the network - usually more than half and not infrequently over 90% - while the rest of the network is divided into a large number of small components disconnected from the rest.
The giant component is not only the largest component in a network, but its size grows in proportion to the number of nodes of the network.

Strong and weak ties

Evidence suggests that in most real-world networks, and in particular in social networks, nodes tend to create tightly knit groups characterised by a relatively high density of ties.



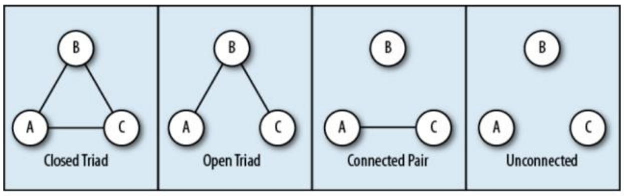
Strong ties connecting nodes “close” in the network.
Weak ties connecting nodes “far” in the network (that guarantee the global network structure).

Transitivity

An important property in social networks, and useful in other networks as well, is transitivity.
A relation R is said to be transitive if a **R** b and b **R** c implies a **R** c
An example of transitive relation is equality =

If a **=** b and b **=** c, then a **=** c

Transitivity in networks



If A "knows" B and B "knows" C we have a connected triple, or **open triad**.
If A "knows" also C we have a loop of length three, or triangle, or **closed triad**.
"The friend of my friend is also my friend"

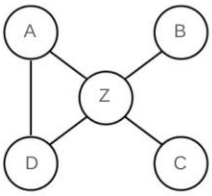
Global clustering

The global clustering coefficient quantifies the fraction of paths of length 2 in a network which are closed.

$$\mathbf{C} = \frac{\text{(number of triangles) x 3}}{\text{number of connected triples}}$$

- C = 1 means that all components in the network are cliques (complete connectivity).
- C = 0 implies that there are no closed triads, which happens for various network topologies, for example trees and stars

Example: compute global clustering



In the example we have **1 triangle** and **8 connected triples**
(<AZD>, <DAZ>, <ZDA>,<AZB>, <AZC>, <BZC>, <BZD>, <CZD>)

C = (1 x 3) / 8 = 0.375

Local clustering

It is possible to define a clustering coefficient C_i for any single vertex i.
C_i captures the density of links in the immediate neighborhood of vertex i, and it represents the average probability that a pair of i's friends are friends as well.

$$\mathbf{C_i} = \frac{\text{number of pairs of neighbors of i that are connected}}{\text{total number of pairs of neighbors of i}}$$

Given a node i of degree K_i , the total number of neighbors pairs is ½ K_i (K_i -1)
If L_i links exist among these pairs, the local clustering is

$$C_i = \frac{2L_i}{k_i(k_i - 1)}$$

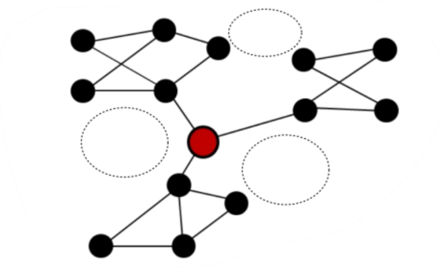
Example: compute local clustering

C_i = 0 means that there are no links between i's neighbors (e.g., star topology)

C_i = 1 implies that each of the i's neighbors link to each other (e.g., clique)

In many networks it is found empirically that nodes with high degree have a lower local clustering coefficient, on average.

Local clustering is an indicator of "structural holes".



Local clustering and structural holes

If we are interested in efficient spread of information or other traffic on a network, then structural holes are a bad things since they reduce the number of alternative routes.

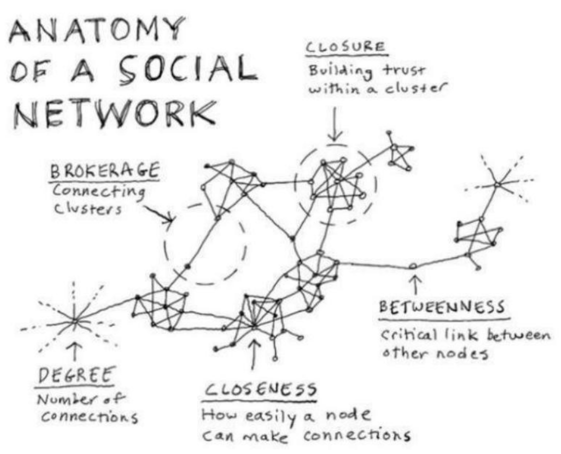
On the other hand, they are a good thing for a vertex whose friends lack connections, somehow measuring its influence.

Lower values of the local clustering coefficient mean more structural holes in the neighborhood of a given node.

Local clustering coefficient can be considered as a type of centrality measure, that takes small values for powerful individuals.

Somehow close to betweenness centrality but "local", it measures the control over flows between just the immediate neighbors of a vertex.

The choice of the measure to use depends on the questions on the network. Computation of betweenness is more expensive (O(N^3))



Erdős–Rényi random graphs

Network models

We can measure the structure of networks diameter, density, degree, betweenness, clustering...
Question: "If I know a network has some particular property, such as a particular degree distribution, what effect will that have on the wider behavior of the system?"

One of the best ways to understand and get a feel for these effects is to build mathematical models:

- Simple representation of complex networks
- Derive properties mathematically
- Predict future behavior of the network

In what ways is your real-world network different from the hypothesized (synthetic) model?

Random graphs model networks in which some specific set of parameters take fixed values, but the network is random in other aspects.
We will see

1) Erdős–Rényi random graph model
2) Watts-Strogatz small-world model
3) Barabási-Albert scale-free model

Erdős–Rényi random graphs
Mimic the patterns of connections in real networks trying to understand their implications.
Erdős–Rényi model assumptions:

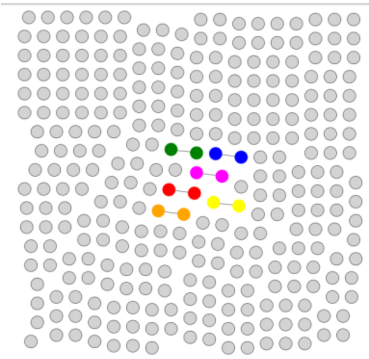
- Start with N isolated nodes and connect them
- Resulting network is undirected
- Key parameter: p or M
- p = probability that any two nodes share an edge G(N,p)
- M = total number of edges in the graph G(N,M)

The G(N,p) model

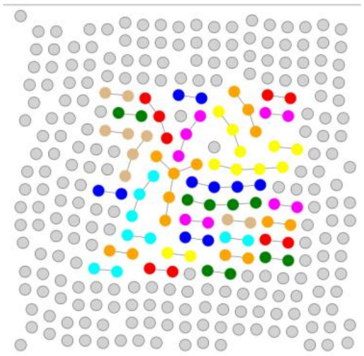
- given N isolated nodes in the graph
- for each distinct pair of nodes, select a random number between 0 and 1
    - if the selected number is <= p add the edge
    - otherwise do nothing

The G(N,p) model is not defined in terms of a single randomly generated network, but as an ensemble of networks, e.g., all networks which can be obtained for the given set of parameters.

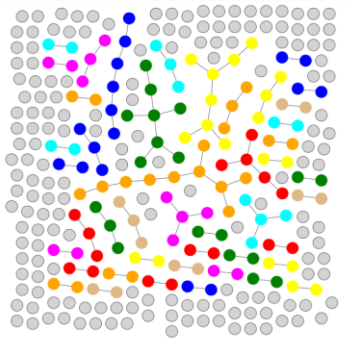
G(N,p) N = 300, p = 0.0001



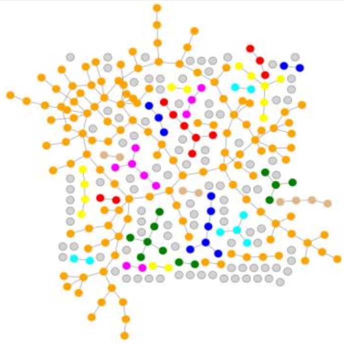
G(N,p) N = 300, p = 0.001



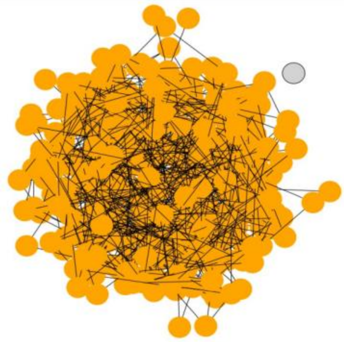
G(N,p) N = 300, p = 0.002



G(N,p) N = 300, p = 0.0035



G(N,p) N = 300, p = 0.02



G(N,p) properties

When we think about the properties of random graphs we typically mean the **average properties** of the ensemble:

- Degree distribution
- Distances
- Clustering

Degree distribution in G(N,p)

In G(N,p):

- For p close to 0, nodes are mainly isolated
- For p close to 1, nodes are mainly connected with all the other nodes
- The probability that a node i has exactly 1 link is modeled as a Bernoulli process

$$p_i = \begin{cases} q = 1 - p & \text{if } k = 0 \\ p & \text{if } k = 1 \end{cases}$$

We are interested in more trials... and therefore we obtain a Binomial distribution.
The probability that a node i has exactly k links is the product of three terms.

$$p_k = \binom{N-1}{k} p^k (1-p)^{N-1-k}$$

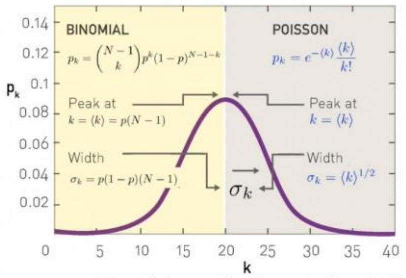
Number of ways to choose the k vertices

Probability of being connected to k vertices and not to any of the others

Under certain conditions, e.g.

- the number of trials N is large (ideally N tends to infinity)
- the probability of success p is small (ideally p tends to zero) but the product Np remains moderate

The Binomial distribution can be approximated by the Poisson distribution with with parameter λ=Np.



In many cases we are interested in the **properties of large networks**, so that **N** can be assumed to be **large**. In the limit of large N, G(N, p) has a Poisson degree distribution

G(N,p) properties

Graph generation is a democratic process: all nodes are equally treated.
In the context of social networks, for instance, the probability of connecting two school friends is equal to the probability of connecting one inhabitant of Ronco Scrivia with Bill Gates.
The evolution of a random graph starts from isolated nodes that are selected and connected in pairs, when possible.

The random universe of Erdős–Rényi - for sufficiently large graphs - is ruled by mean values.
All nodes get nearly the same number of arcs:

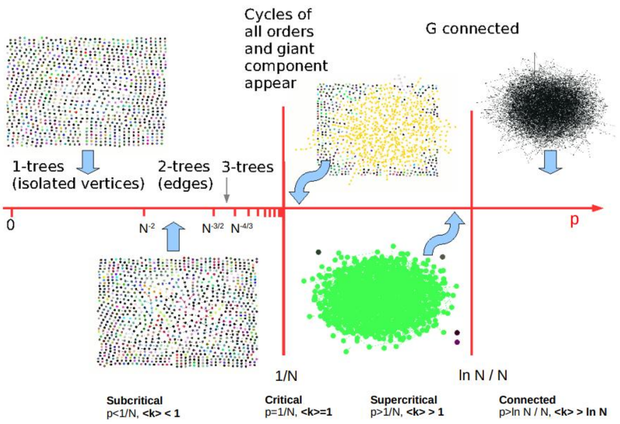
- Individuals have more or less the same number of friends
- Neurons connect more or less to the same number of other neurons
- Companies have more or less the same number of business relations with other companies
- Web sites receive more or less the same number of visits

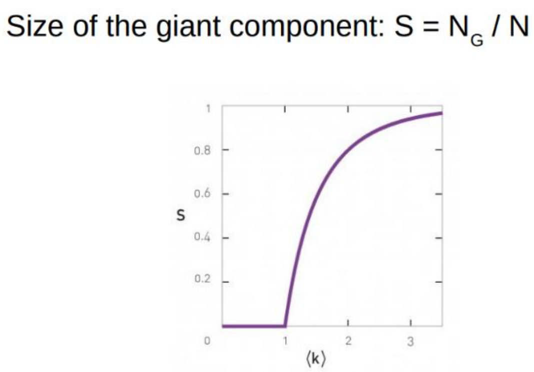
Evolution of G(N,p)

What happens for increasing values of p?
How varies the size N_G of the giant component?

- For p = 0 we have ‹k› = 0, hence we observe only isolated nodes. Therefore N_G = 1 and it does not depend from N
- For p = 1 we have ‹k› = N-1, hence the network is a complete graph and all nodes belong to a single cluster. Therefore N_G = N and it grows with N
- For 0 < p < 1? Erdős and Rényi predicted (1959) that the condition for the emergence of the giant component is ‹k› = 1

The emergence of a connected network within the G(N,p) model is not a smooth process: the isolated nodes and tiny components observed for small ‹k› organize themselves into a giant component rather suddenly, through a process called phase transition.





Network models

From previous lecture:

- One way to study the origin of network characteristics is to formulate a model, i.e., a set of instructions used to assemble a network
- Given the model we can compare it with real networks to see how they are similar or different

Measures in random graphs

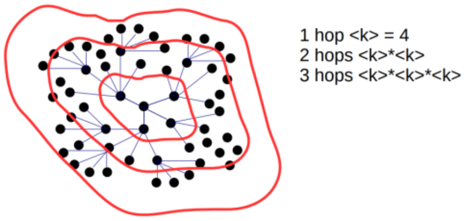
G(N,p)

- Mean degree = p * (N–1) or p * N
- Giant component ‹k› > 1, p > 1 / N
- Expected number of links ‹L› = $\binom{N}{2} * p = \frac{p*N*(N-1)}{2}$
- Density ρ = ‹k› / (N-1) = p * (N-1) / (N-1) = p
- Diameter?
- Clustering?

Distances in random graphs

The basic idea behind the estimation of the diameter of a random graph is simple.

The average number of vertices s steps away from a randomly chosen vertex can be approximated by $< K >^s$.



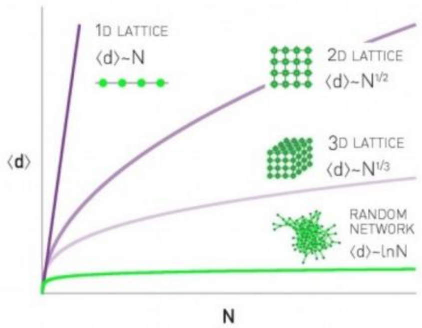
Since this number grows exponentially with s it does not take many steps such that ‹k› ^{s} ~ N

By applying logarithm we have

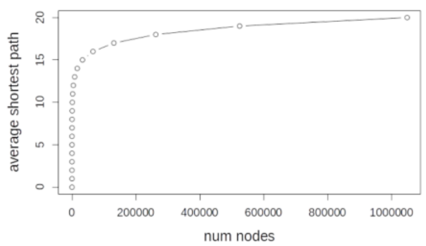
- ln (‹k› ^{s}) ~ ln (N)
- s * ln (‹k›) ~ ln (N)
- s = ‹d› ~ ln (N) / ln (‹k›)

Note: some authors use log() instead of ln() but with the constant value at the denominator, the result is the same.

Although the random graph is not an accurate model of most real networks, it shows the small-world effect empirically observed in many real networks!



Erdős-Rényi networks can grow to be very large but nodes will be just a few hops apart



In the language of network science the smallworld phenomena implies that the distance between two randomly chosen nodes in a network is surprisingly short
... in random networks the average path length or the diameter depends logarithmically on the system size. Hence, “small” means that ‹d› is proportional to ln N, rather than N or some power of N.

Real networks are supercritical, disconnected, with short paths

Network	N	L	$<k>$	ln N
Internet	192,244	609,066	6.34	12.17
Power Grid	4,941	6,594	2.67	8.51
Science Collaboration	23,133	186,936	8.08	10.04
Actor Network	212,250	3,054,278	28.78	12.27
Yeast Protein Interactions	2,018	2,930	2.90	7.61

The average degree of real networks is larger than ‹k› = 1, implying that they all have a giant component.
But most real networks do not satisfy ‹k› > ln N, indicating that they should consist of several disconnected components.

Measures in random graphs
G(N,p)
- Mean degree ‹k› = p * (N–1) or p * N
- Giant component ‹k› > 1, p > 1 / N
- Expected number of links ‹L› = $\binom{N}{2}$ * p = p * N * (N-1) / 2
- Density ρ = ‹k› / (N-1) = p * (N-1) / (N-1) = p
- Diameter ‹d› ~ ln N
- Clustering?

Problems with random graphs
G(n,p) is one of the most studied network model but one clear problem is that it shows no transitivity or clustering.
In a random graph the probability that any two vertices are neighbors is exactly the same,
p = ‹k› / (N – 1), which is also the clustering coefficient.
C = / (N – 1) which becomes C = ‹k› / N for large N.

Clustering in random graphs
C = ‹k› / N
C depends on N and tends to zero for large values of N (if ‹k› stay fixed); it is generally smaller than the values empirically observed in many real-world networks.
For the human population we have N ~ 8 billion people, suppose each having about 150 stable social relationships
C ~ 150 /(8*10^9) ~ 15*10/(8*10^9) ~ 2/10^8

150 is the Dunbar number

Moreover, C depends from the average degree ‹k› and it is independent from the actual node's degree. Again, this is not true in many real networks where generally nodes with higher degree show a lower clustering.

Problems with random graphs
Another difference between random graphs and real networks is the shape of their degree distribution. Poisson degree distribution (bell-shape) is not so common in real networks...
This makes the Poisson random graphs inadequate to explain many of the interesting phenomena we see in networks today, including resilience phenomena, epidemic spreading processes, and many others.

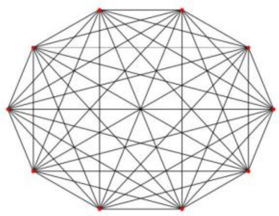
Take away points
G(N,p) is a simple model to build complex networks in a probabilistic way.
It shows "nice" properties:
- Logarithmic diameter
- Phase transition among different network configurations
- For p values large enough, G(N,p) graphs have a giant component whose size grows with N

But something is missing:
- No transitivity (either too dense for p close to 1, or too few triangles for small values of p)
- The shape of the degree distribution in many real networks is different

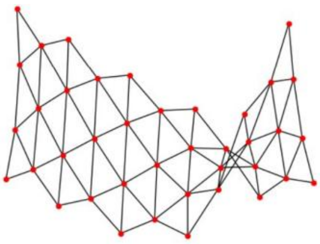
Watts-Strogatz model

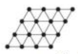
Short distances and Clustering
We need a new model with some specific rules to create short distances and "enough" triangles as empirically observed in real networks.

Distances in regular networks



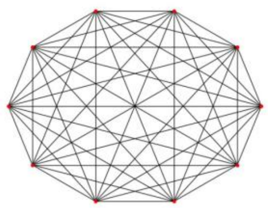
Clique
`complete_graph()`
Only **one hop** between all pairs



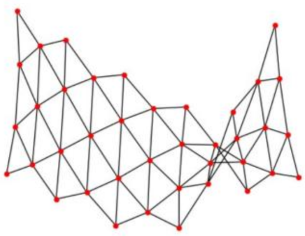


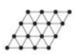
Triangular lattice
`triangular_lattice_graph()`
Long paths

Clustering in regular networks



Clique
`complete_graph()`
C = 1



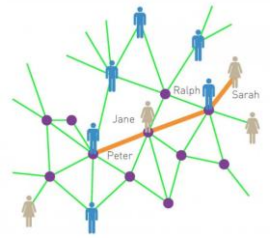


Triangular lattice
`triangular_lattice_graph()`
Internal nodes have degree k=6 and clustering **C converges to 2/5** in the limit of infinite lattice

Unfortunately these graphs do not mimic real networks

Small-world phenomena

In the language of network science the small-world phenomena implies that the distance between two randomly chosen nodes in a network is surprisingly short.



Milgram's experiment

The first* experimental study of small world phenomena was performed by Stanley Milgram and his colleagues in 1967.
He asked to a collection of 296 randomly chosen “starters” to try forwarding a letter from Nebraska and Kansas to target persons in Massachusetts and Pennsylvania.

The starters were given some personal information of the targets (including address and occupation). They were asked to forward the letter to someone they knew on a first-name basis, with the same instruction, in order to eventually reach the target.
Each letter passed through the hands of a sequence of relatives, acquaintances, and friends until it reached the target or got lost.

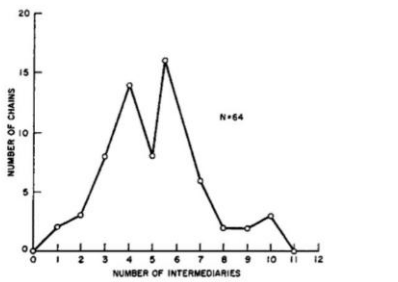
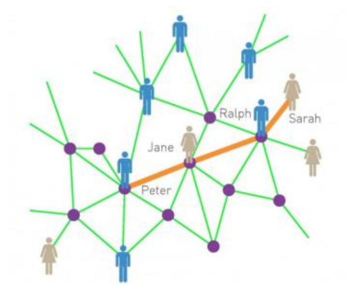
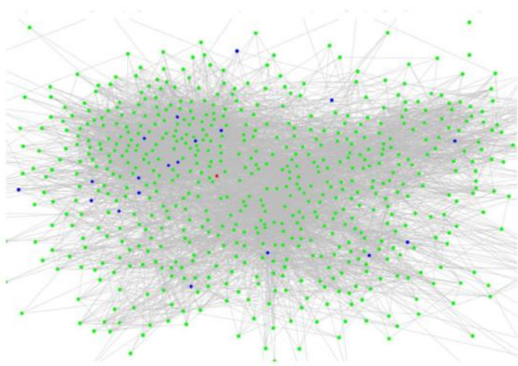


Figure 2.10: A histogram from Travers and Milgram's paper on their small-world experiment [391]. For each possible length (labeled “number of intermediaries” on the x-axis), the plot shows the number of successfully completed chains of that length. In total, 64 chains reached the target person, with a median length of six.

Small-world phenomena



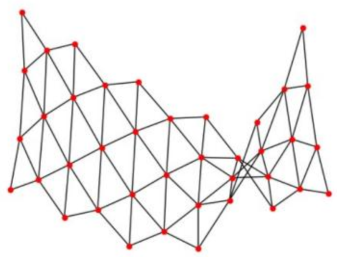
My friends know each others



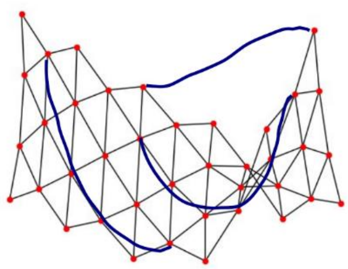
In a few steps I can reach other worlds (**six-degree of separation**). Short paths exist and individuals can discover them on the basis of local knowledge

Small-world generative model
Proposed by Duncan Watts and Steven Strogatz, 1998.
Two observation of real networks:
- The average distance between two nodes depends logarithmically on N
- The average clustering coefficient of real networks is much higher that expected for a random network of similar N and L

Distances in regular networks



Triangular lattice
Long paths



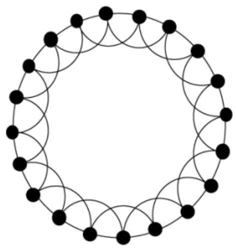
Triangular lattice
~~Long paths~~
Shorter paths

We can create new links to connect randomly selected nodes and create shortcuts.

Since only a fraction of the triangles are disrupted by this procedure, the clustering coefficient remains high.

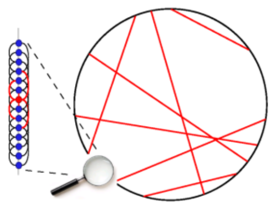
This is the starting point of Watts and Strogatz, who imagined a ring in which each node is connected to its k nearest neighbors.

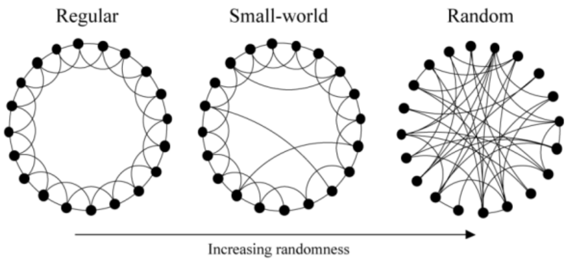
Small-world generative model
Take a ring with N nodes, each node has k neighbors (representing other nodes in the same community).



Such a network has high clustering (0 when k = 2, ½ for k=4, ¾ for k → infinite)
But has also constant degree distribution (k for each vertex) and long distances (avg shortest path N/2k).
In the case of the population $8*10^9$ / 2*150 (millions of steps).

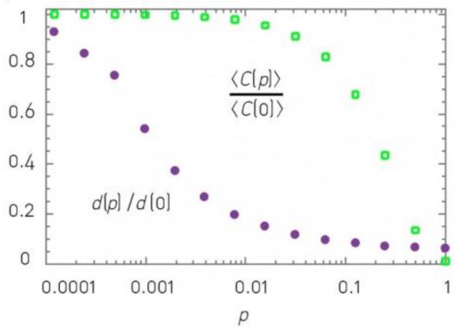
Take a ring with N nodes, each node has k neighbors (representing other nodes in the same community).
Replace some links with shortcuts (weak ties) selected uniformly at random with rewiring probability p.





The rewiring probability p controls the evolution from regular to random graphs.
When p=0 high clustering without small-world, when p=1 we have exactly the opposite.
For increasing values of p the model keeps high clustering but also exhibits the small world effect, showing that there is a range of values of p in which the two characteristics co-exist.

Clustering <C[p]> vs Diameter d[p]



For values of the rewiring probability p in (0.01, 0.1) we have short distances and high clustering.

Regular vs random graphs

Real networks are not random

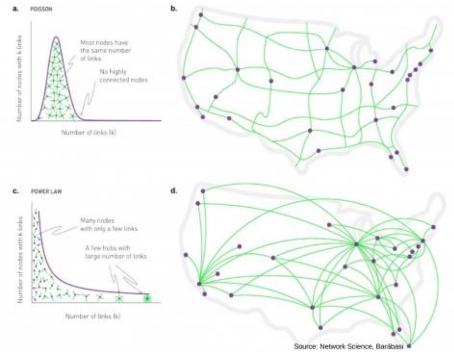
For many years complexity = randomness, but…

- The degree distribution of a random network follows a Binomial distribution, well approximated by a Poisson distribution for large N
- Yet, the Binomial/Poisson distribution fails to capture the degree distribution of real networks…
- Random network theory predicts that for ‹k› > 1 we should observe a giant component, a condition satisfied by many networks
- But most networks do not satisfy the ‹k› > ln N condition, which implies that these networks should be broken into smaller isolated clusters
- In a random network the local clustering coefficient is independent of the node's degree and depends on the system size as 1 / N
- In contrast, measurements indicate that for real networks the clustering coefficient decreases with the node degrees and is largely independent of the system size

Barabási-Albert Model

The Web

Regardless of the right number of pages which are indexed by Google or other search engines, the Web is the largest network humanity has ever built. Some claims that it exceeds in size even the human brain (N ≃ 1011 neurons).
In 1998 there were reasons to believe that the Web could be well approximated by a random network.
But…



Power law distribution
We are used to bell shaped distributions.
Power law is completely different:

- a fraction of nodes exists with very high degree (heavy or long tail distribution)
- the mean value does not always make sense

Networks whose degree distribution follows a powerlaw are also called scale-free networks (under certain conditions for the values of the parameter).
The probability of observing a high degree node, or hub, is several orders of magnitudes higher in a power law network than in a random network.

Describes phenomena where large events are rare but exist, and small ones are quite common:

- There are few large earthquakes, but many small ones
- There are a few mega-cities, but many small towns
- There are few words such as “and” and “the” that occur very frequently, but many which occur rarely

Consider a world where the heights of Americans were distributed as a power law, with approximately the same average as the true distribution (which is Normal).

In this case, we would expect:

- nearly 60,000 individuals to be as tall as the tallest adult male on record, at 2.72 meters
- further, we would expect ridiculous facts such as 10,000 individuals being as tall as an adult male giraffe
- one individual as tall as the Empire State Building (381 meters)
- and 180 million tiny individuals standing a mere 17 cm tall, like squirrels

Distributions in which the frequency of an event varies as a power of k

$$p_k = C \cdot k^{-\gamma}$$

γ is the exponent characterizing the power law, the degree exponent
Const C is determined by the normalization condition

$$\sum_{k=1}^{\infty} p_k = 1$$

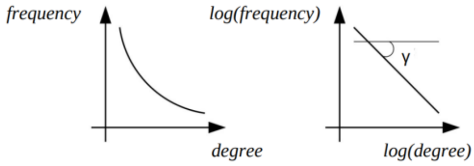
$$\sum_{k=1}^{\infty} C \cdot k^{-\gamma} = 1 \qquad C = \frac{1}{\sum_{k=1}^{\infty} k^{-\gamma}}$$

Rieman zeta function
https://en.wikipedia.org/wiki/Riemann_zeta_function

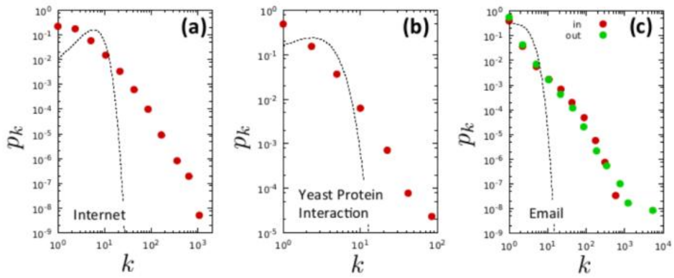
$$p_k = \frac{k^{-\gamma}}{\sum_{k=1}^{\infty} k^{-\gamma}}$$

p_k diverges at k = 0, we therefore need to separately specify p_0, representing the fraction of isolated nodes.

Signature of the power law

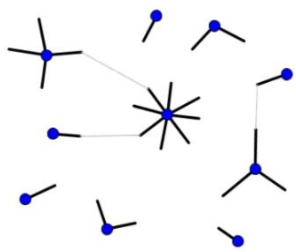


$$\log p_k = \log(C \cdot k^{-\gamma}) = -\gamma \cdot \log k + \log C$$



The diversity of the systems that share the scale-free property is remarkable. In some networks the nodes are molecules, in others they are computers. It is this diversity that prompts us to call the scale-free property a universal network characteristics. Dashed black line shows the Poisson distribution with the same ‹k› as the real network.

Configuration model



Goal: generate networks whose nodes have an arbitrary degree sequence
Example:
[4,1,2,8,1,3,1,2,1,1]
(the sum of the degrees must be even)
Assign to each node a number of stubs corresponding to the degree of the node.

Repeat until all stubs are joined:
- Select at random a pair of stubs
- Join to each other and form a link

This model generates:
- all possible networks with the given degree distribution
- networks can have multiple edges between nodes and self-loops

Degree preserving randomization

Other constraints can be imposed, for example we can be interested in networks with a given number of triangles.
Broad class of network models known ad **Exponential Random Graphs.**
Limitations: this model (and all models seen so far):
- is **static**, all the nodes are there from the beginning (in real networks nodes and links appear and disappear)
- **cannot explain the existence of hubs** if this is not specified in the initial degree sequence (but this is not helpful in explaining how hubs emerge in real networks)

Barabási-Albert model
Random graphs and small worlds describe an egalitarian society built on random relationships.
Many real networks are different due to the presence of special nodes, the hubs.

In late 1990s Barabási-Albert, while studying the Web, tried to answer the following questions:
"How hubs manifest?"
"How many hubs can appear in a network?"
"Why hubs are not present in known models?"

Answer num. 1: growth.
Complex networks grow in time while previous models consider a fixed number of nodes, and suggest different connection methods.
The dynamic evolution of a model radically changes its properties but, on its own, does not justify the appearance of hubs.

Answer num. 2: preferential attachment.
Whenever a new node joins the network, it does not connect randomly but it is driven by node popularity:
- A new Web page connects to already densely connected pages
- In the Hollywood universe, stars guarantees advertisements and earnings to movies

The phenomenon follows the "Rich get richer" metaphor and real networks are "aristocratic".

In general, each node adds exactly m links and the probability $\Pi(K_i)$ that a link of the new node connects to node i depends on the degree K_i of node i as

$$\boxed{\Pi(k_i) = \frac{k_i}{\sum_j k_j}}$$

Preferential attachment is a probabilistic rule: a new node is free to connect to any node in the network, whether it is a hub or has a single link. The equation of $\Pi(K_i)$ implies, however, that if a new node has a choice between a degree-two and a degree-four node, it is twice as likely that it connects to the degree-four node.

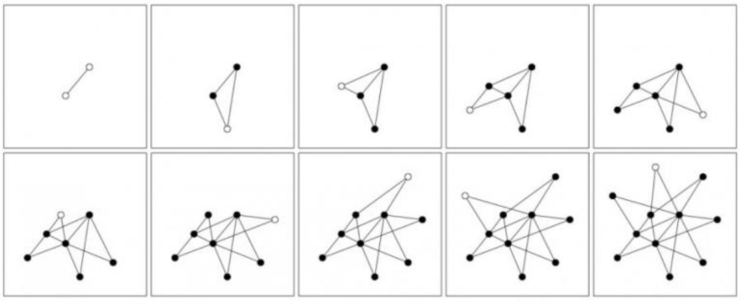


Figure 5.2: Time evolution of the Barabási-Albert model.

The sequence of images shows the gradual emergence of a few highly connected nodes, or hubs, through growth and preferential attachment. White circles mark the newly added nodes to the network, which decide where to connect their two links (m=2) through preferential attachment (1). After [9].

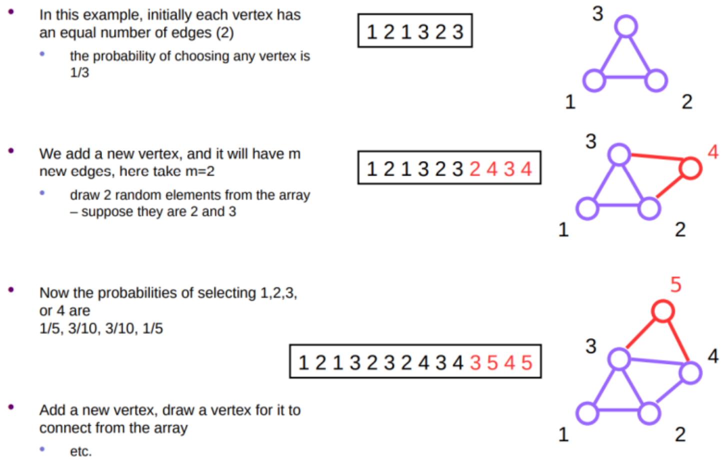
Computer simulation of preferential attachment

Choosing a vertex uniformly is easy.

Choosing a vertex in proportion to its degree is only slightly harder and can be computed keeping track of edge endpoints in one large array and selecting an element from this array at random:

- the probability of selecting any one vertex will be proportional to the number of times it appears in the array, which corresponds to its degree

Selection of a vertex considering its degree



After t time steps the BA model generates a network with N = n_0 + t nodes and L = m_0 + m_t links

Barabási-Albert have shown that the degree distribution of networks built with this process, for large values of k, is represented by a power law

$$\boxed{p_k \sim k^{-3}}$$

Barabási-Albert model: limitations

1. The degree distribution is fixed (the exponent is always 3, for any choice of the parameters)
2. The hubs are always the oldest nodes
3. It does not create many triangles (the clustering coefficient is lower than in many real networks)
4. Nodes and links are only added, never deleted
5. The network consists of a single connected component (many real networks have multiple components)

Similar (previous) models

The Barabási-Albert model is similar to other models previously proposed by Herbert Simon (economic data) and Derek de Solla Price (citation networks).

The economist Herbert Simon (1955) noted the occurrence of power laws in a variety of (non-network) economic data, such as the distribution of people's personal wealth (see also Pareto).
He proposed an explanation for the wealth distribution based on the idea that people who have money already, gain more at a rate that is proportional to how much they already have.
"Rich get richer".

Price (in the 1970s) adapted Simon model to the citation network introducing the idea of attractiveness.

Papers are continually published:

- Each paper brings on average c citation
- New nodes are created but never destroyed
- The resulting network is growing, directed (from present to past) and acyclic
- The link probability is proportional to the sum of the degree plus a constant attractiveness (which is necessary so that also new papers can get edges)

Price computed the degree distribution of his model as

$$p_k \sim k^{-(2+\frac{a}{c})}$$

The distribution depends on the **constant a**
If a = c we have the Barabási-Albert model

Barabási-Albert model: extensions

Extensions of preferential attachment models have been suggested, trying to make the model more faithful to the way real networks grow

- By contrast with citations, links in other networks are not permanent. They can disappear as well as they can be added at any time after their entrance in the network
- Not all vertices are created equal. Some papers or websites might be intrinsically more interesting or important by virtue of their content and hence attract more links

Can this process be incorporated into the model?

Barabási-Bianconi Model

The case of Google

The Barabási-Albert model suggests that the dynamics of a complex network is ruled by the "First mover advantage" metaphor.
The older nodes in the system have more chances to get a higher number of links.
In 1997 Google was the "late Web arrival", but in the period 1997-2000, it became the most significant node in the network.
A new arrival is frequent in many networks but its success cannot be explained by the models seen so far, where all nodes of a given degree are equally likely to gain a new edge.
In real networks, indeed, nodes have certain "intrinsic quality" far beyond their connectivity.

Competition
Competition for links is a common feature of complex systems: web sites compete for URLs to enhance their visibility, in the business world companies compete for links to consumers, ...
When considering competition, each node has a certain fitness and the Barabási-Albert model has been extended by Barabási-Bianconi to take into account also this parameter.

Adding fitness
The growth is still ruled by preferential attachment but it does not consider nodes connectivity only.
To incorporate the different ability of the nodes to compete for links a fitness value η_i is initially assigned to each node i, chosen from a distribution $\rho(\eta)$.
The probability that a link of a new node connects to a preexisting node i is proportional to the product of node i's degree and its fitness.

$$\Pi(k_i) = \frac{k_i \cdot \eta_i}{\sum_j k_j \cdot \eta_j}$$

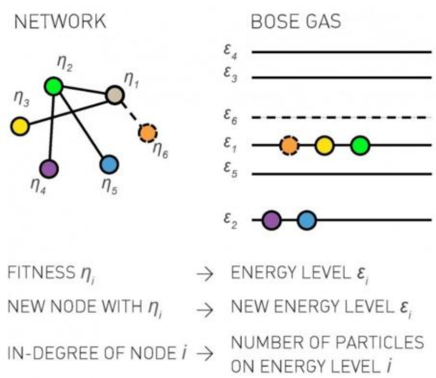
Bianconi-Barabási model

The dependence of Π_i on k_i captures the fact that higher degree nodes are easier to encounter, hence we are more likely to link to them.
The dependence of Π_i on η_i implies that between two nodes with the same degree, the one with higher fitness is selected with a higher probability.
Hence, previous equation ensures that even a relatively young node with initially only a few links can acquire new links rapidly if it has high fitness, larger than the rest of the nodes.

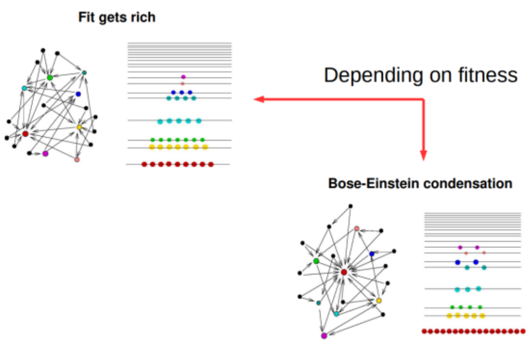
The overall degree distribution for the entire network may or may not have a power law tail, depending of the distribution $\rho(\eta)$.
In the trivial case, in which all vertices have the same η, the model reduces to the original Barabási-Albert. If η is broadly distributed, however, in general the degree distribution will not yield another power law.

Bianconi-Barabási result
How does the network topology depend on the fitness function?
This question was clear after the discovery that "some networks can undergo Bose-Einstein condensation"

Fitness → Energy
Each node with fitness η_i has an energy level ε_i
Links → Particles
For each link from node i to node j we add a particle to level ε_j
Nodes → Energy
level The arrival of a new node with m links corresponds to adding a new energy level (corresponding to the new node) and m new particles to the Bose gas; the particles are placed at the energy levels to which the new nodes link.



Source: Network Science, Barabási



Winner takes all
Complex networks with preferential attachment thus show three different growth models

"first mover advantage" (scale-free)
"fit get rich" (scale-free)
"winner takes all" (star-like hub-and-spoke)

Scale-free properties

Which is the role of the degree exponent γ in scale-free networks?

Probability distribution n^{th} moment

In statistics, moments are used to describe the characteristic of a distribution, for example its shape and properties.

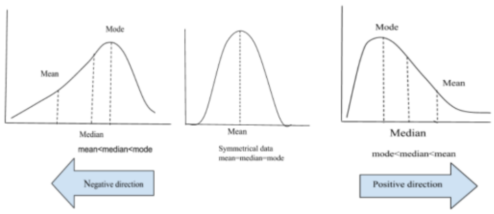
Given a random variable X, the moments are defined as the X's expected values of E[X], E[X^2], E[X^3], E[X^4],..., etc.

E[X], the first moment, is the expected value that measures the location of the central point (e.g., its mean or average)

E[X^2], the second moment, measures the spread of the distribution around the mean. Used to compute the variance Var(X) = E[X^2] - $E[X]^2$

E[X^3], the third moment, is the skewness, and it measures how asymmetric the distribution is about its mean

- **Symmetrical distribution** (E[X^3]=0)
- **Positively skewed**, where the right tail is longer. Tells us about "outliers" that have values higher than the mean.
- **Negatively skewed**, where the left tail is longer. Tells us about "outliers" that have values lower than the mean.



If we consider a degree distribution we have

$$\langle k^n \rangle = \sum_{k_{min}}^{\infty} k^n p_k \simeq \int_{k_{min}}^{\infty} k^n p(k) dk$$

The first moment (n=1) is the average degree, ‹k›.

The second moment (n=2) allows to compute the variance and the standard deviation, measuring the spread in the degrees.

The third moment (n=3) tells us how much p_k is symmetric around the average degree ‹k›.

For scale-free networks

$$\begin{aligned} \langle k^n \rangle &= \int_{k_{min}}^{k_{max}} k^n p(k) dk = \int_{k_{min}}^{k_{max}} k^n C k^{-\gamma} dk \\ &= C \int_{k_{min}}^{k_{max}} k^{n-\gamma} dk = \frac{C}{n-\gamma+1} \left[k^{n-\gamma+1} \right]_{k_{min}}^{k_{max}} \\ &= \frac{C}{n-\gamma+1} \left[k_{max}^{n-\gamma+1} - k_{min}^{n-\gamma+1} \right] \end{aligned}$$

k_{min} is fixed
k_{max}, e.g., the size of the largest hub grows with the size of the network: for N→∞, also k_{max}→∞

$$\frac{C}{n-\gamma+1}\left[k_{max}^{n-\gamma+1} - k_{min}^{n-\gamma+1}\right]$$

For large networks $\langle k^n \rangle$ is finite if n-γ+1 <= 0, e.g., all moments that satisfy n <= γ-1 are finite
All moments larger than γ-1 diverge

In many scale-free networks the degree exponent is 2 < γ < 3, and in the limit of N→∞, n <= 2-1 <= 1
Only the first moment k is finite ⟨k⟩ , the other moments, $\langle k^2 \rangle$, $\langle k^3 \rangle$, ... go to infinity
Scale-free term: "When we randomly choose a node, we do not know what to expect, its degree could be tiny or arbitrarily large. Hence networks with γ ‹ 3 do not have a meaningful internal scale, but are scale-free".

For one Web sample:

- ⟨k⟩ = 4.60
- γ ≈ 2.1
- For N→∞ the in-degree of a randomly chosen web page is k=4.60 ± ∞
- A randomly chosen web page could easily yeld a degree 1 or 2, as in this sample 74.02% of nodes have in-degree less than k, but also ⟨k⟩ millions of links, like google.com or facebook.com

Lack of an internal scale

In summary, the scale-free name captures the lack of an internal scale, a consequence of the fact that nodes with widely different degrees coexist in the same network

This feature distinguishes scale-free networks from:
- regular networks, where all nodes have exactly the same degree (σ = 0)
- random networks, whose degrees vary in a narrow range ($\sigma = \langle k \rangle^{1/2}$)

This divergence is the origin of some properties of scale-free networks, from their robustness to random failures to the anomalous spread of viruses…

Small-world property
Do hubs affect the small-world property?

Ultra-small property
The dependence of the average distance d on the system ⟨k⟩ size N and the degree exponent γ is captured by

$$\langle d \rangle \sim \begin{cases} const. & \gamma = 2 \\ \ln \ln N & 2 < \gamma < 3 \\ \frac{\ln N}{\ln \ln N} & \gamma = 3 \\ \ln N & \gamma > 3 \end{cases}$$

Anomalous regime, γ = 2, = const
For γ = 2, the degree of the largest hub grows linearly with the system size: $k_{max} \sim N$
This forces the network into a hub-and-spoke configuration (everyone is close) and in this regime the average path length does not depend on N.

Ultra-small-world, 2 < γ < 3, = ln ln N
As several real networks have degree exponent between 2 and 3, this regime is of particular practical interest.
The average distance ‹d› increases as ln ln N, a significantly slower dependence than the value ln N found for random networks.
Networks in this regime are ultra-small, as the hubs radically reduce path lengths. They do so by linking to a large number of small degree nodes, creating short distances between them.

To see the implication of the ultra-small world property consider again the world's social network with N ≈ $8x10^9$
If the society is described by a random network, the Ndependent term is ln N = 22.80
In contrast for a scale-free network the N-dependent term is ln ln N = 3.13, indicating that the hubs radically shrink the distance between the nodes.

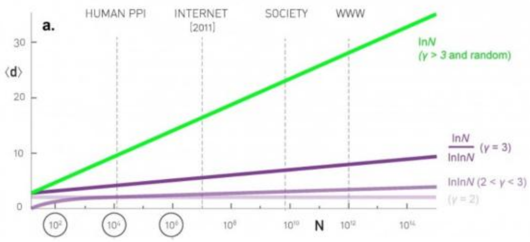
Critical point, γ = 3, = ln N / ln ln N
This value is of particular theoretical interest, as the 2nd moment $E[X^2]$ of the degree distribution does not diverge any longer.

At this critical point the ln N dependence encountered for random networks returns. However, the calculations indicate the presence of a double logarithmic correction ln ln N, which shrinks slightly the distances compared to a random network of similar size.

Note: this is the case of the Barabási-Albert model

Small-world, γ > 3, = ln N
In this regime the 2nd moment $E[X^2]$ is finite and the average distance follows the small-world result derived for random networks.
While hubs continue to be present, for γ > 3 they are not sufficiently large and numerous to have a significant impact on the distances between the nodes (shorter tail in the distribution).

Distances in scale-free networks



The dotted lines mark the approximate size of several real networks of practical interest
For large networks (like the WWW) the small-world assumption does not correctly estimate the real <d>

Summary
The scale-free name captures the lack of an internal scale, a consequence of the fact that nodes with widely different degrees coexist in the same network.
To document the presence of a power-law degree distribution, k_{max} should be orders of mag\nitude larger that k_{min} at least 10^2 - 10^3 times.
The larger are the hubs, the smaller is the network, e.g, the distances between nodes decrease.

Random walk model

Networks built with Barabási-Albert or BarabásiBianconi models have low clustering coefficient but do not favor triadic closure.

The random walk model can start from any small network and, at each iteration
- A new node i is added with m > 1 new links
- The first link is attached to a random node j (chosen with uniform probability)
- Each other link is attached to:
  - a randomly selected neighbor of j with probability p or
  - another random node with probability 1-p

The number of triangles depends on the parameter p
- If p=1, we have the largest density of triangles
- If p is large enough, thanks to the Friendship Paradox, we have heavy-tailed degree distributions, since high degree nodes have more chances to acquire new links
- If p=0, we have random graphs

This model does not assume that new nodes have knowledge of the degrees of the old nodes:
- New nodes “explore” the network in a random fashion and find other nodes with a frequency proportional to their degrees
- The process of triadic closure induces preferential attachment
- For p large enough, there is also a high density of triangles and the model creates networks with community structures

Scale-free networks with tunable clustering
Similar to the random walk model but it uses the preferential attachment in the first step.
The sociological interpretation is that actor i (e.g., node i), after being connected with actor j in the network, which is chosen for its popularity, is likely to know some friends of j as well.

Friendship paradox
Suppose you want to find a node with high degree in a network of size N
Random choice: probability 1/N

What if you ask to a random node about their friends?
Reaching out to someone’s friends actually means choosing links instead of nodes.

When we look for nodes, each of them has the same probability of being selected, when we look for links, the higher is the degree of a node, the higher is the probability it will be selected.

Take away points
We can categorize models in three main classes:

1. **Static** models that start from a fixed number of nodes (random and small-world networks).
2. **Generative** models that generate networks with a predefined degree distribution (configuration model).
3. **Evolving** models that grow in time and capture the dynamic evolution of a network. The most studied example is the Barabási-Albert model but many extensions exist to take into account aging, link deletion, and other details.

If we want to understand the origin of a network property, we must use evolving network models that capture the processes that build the network in the first place.

The Web Graph

World Wide Web
1980: TBL writes enquire to connect related documents
1989: writes the paper "Information Management: A Proposal"
1990: first interaction between a web server and a client
1994: W3C

Past: Static Web
Static pages formed the first navigational backbone

- Not resulting from server-side programming
- No "?" in URLs
- Did not change often

Public pages

- No passwords
- No file robots.txt
- No meta tag noindex

Today: Dynamic Web
Today the majority of web pages are not static

- Built on the fly (headers, navigation bars,...)
- Dynamic pages that look static (catalogues, news,...)
- Sites like blogs or wikis
- REST API, microservices
- Spider traps, honeypots, spam pages

Why study it?
Largest network ever built.
We can use its structure to study:

- Crawling strategies
- Searching strategies
- Spam detection
- Communities detection

Which properties?

- Global structure
- Degree distribution
- Reachability
- Connected components

Web graph

Broder et. Al, 2000, starting from two datasets provided by Altavista (with 200 millions pages and 1.5 billions links each) computed:

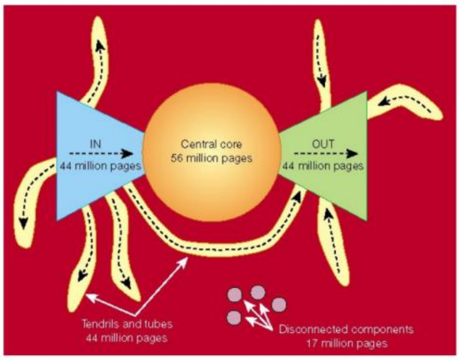
- Macroscopic structure of the Web
- Diameter and paths
- Degree distribution
- Connected components

Components in directed networks

The component structure in directed networks is more complicated that for undirected ones.
Directed graphs have weakly and strongly connected components.
There is typically one large weakly / strongly connected component in a graph, and a selection of smaller ones.
Associated with each strongly connected component are:

- an OUT-component
- an IN-component

Macroscopic structure



Several “continents”

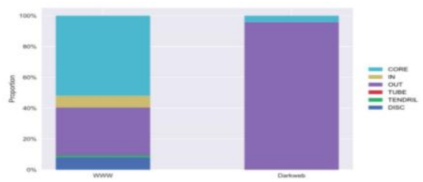
The analysis shown that “... the web is not the ball of highlyconnected spaghetti we believed it to be; rather, the connectivity is strongly limited by a high-level global structure.”
90% of the pages form a weakly connected component (with 186 millions pages).
The connected web breaks into 4 pieces, roughly of the same size.
The largest strongly connected component, the CORE, has 56 millions pages.
IN and OUT have millions of contacts with the CORE and thousands of weakly connected components.

Power-law is everywhere.
The CORE diameter is at least 28.
The entire graph diameter is at least 500.
For each pair of nodes, they are connected through a path with probability 0.24.
If there is a path between two nodes, its average length is 16 (directed), 6 (undirected).

The Dark Web

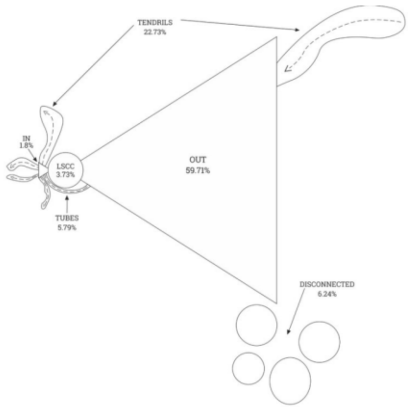
Definition: "…subset of the web, where websites are identified by randomly generated 16 characters addresses instead of readable hostnames".
Unexplored dataset.
Usually accessed using sophisticated routing techniques.

The dark web is being used for both good and bad activities.
Users with good intentions can leverage this anonymous platform to express their thoughts without any censorship.
The downside of the dark web is that it is being misused by criminals and fraudsters. Many illicit trades carried out on the physical world have now been shifted to the dark web.
There is a need to study the structure of the dark web to get insight into the criminal activity on it.



	World-Wide-Web		Darkweb	
CORE	22.3M	51.94%	297	4.14%
IN	3.3M	7.65%	0	0.0%
OUT	13.3M	30.98%	6881	95.86%
TUBES	17k	.04%	0	0.0%
TENDRILS	514k	1.2%	0	0.0%
DISCONNECTED	3.5M	8.2%	0	0.0%

Meusel R., Vigna S., Lehmberg O., Bizer C., "The structure in the Web", 2015
Griffith V., Xu Y., Ratti C., "Graph of the Darkweb", 2017



Abdullah Alharbi et al. "Exploring the Topological Properties of the Tor Dark Web", 2021, IEEE Access

Take away points

The web graph has been deeply studied and its structure is now well known. Different studies have been performed and the "bow tie" always shows up (possibly with different sizes in the components). Can you imagine a different structure for a directed network?
The dark web is scale-free, with few high degree hubs, and it can be considered a sparse hub-and-spoke place.

Search Engines

A search engine:

- Explores the public web (crawling).
- Stores web pages and creates data structures (index) to map content to the pages that contain it.
- Ranks and returns answers to users' queries.

Explore: Web crawlers

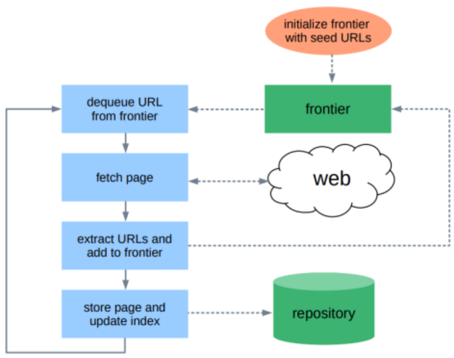
Any program that uses the HTTP protocol to request content from web servers is a web client:

- Browsers
- Web crawlers: programs that automatically download web pages

BFS algorithm running on the web link graph:

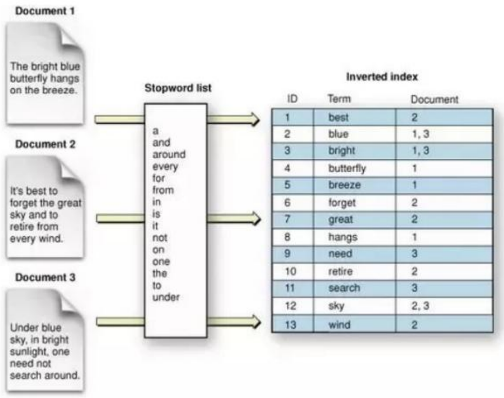
- start from a set of seed (good) pages
- collect new pages
- find links and explore them until there are unvisited links

New links are visited according to a first-in-first-out queue, before we visit any page at distance n from a seed page we visit all pages at distance n-1 or less.



Store: document inverted index

Type of document index used in search engines and information retrieval systems.
Maps terms (words or phrases) to the documents (web pages) that contain them.
For each term in the stored documents, the inverted index maintains a list of document identifiers (or pointers) where the term appears.



* More complex data structures exist for indexes

Rank: first generation search engines

First generation search engines are probably the most popular application in the field of Information Retrieval (IR)

- Return resources (pages) which are relevant to an information need (expressed through a search query)
- A query does not identify a single object but multiple objects may match the query, with different degree of relevance (ranking)

Text based ranking systems:

- Indexing of plain text pages
- Return a list of pages that satisfy the query, ordered according to their numeric scores
- Top ranking pages are shown to users

TF-IDF (IR topic)

Statistical measure that evaluates how relevant a word is to a document in a collection of documents. Represents each document as a vector where each dimension corresponds to a unique term in the vocabulary, and the value of each dimension is the TF-IDF score of that term in the document. Product of two metrics: how many times a word appears in a document (TF), and the inverse document frequency of the word across a set of documents (IDF).

TF (Term Frequency): frequency of a term tin a given document d.

Different ways of computing:

- count the number of occurrences of t in d
- divide by length of a document, or by the raw frequency of the most frequent word in a document

Problems with documents of different sizes…

IDF (Inverse Document Frequency), is a measure of how much information the word provides, that is, whether the term is common or rare across all documents.

This metric can be calculated by taking the total number of documents, dividing it by the number of documents that contain a word, and calculating the logarithm.

$$\mathrm{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

N: total number of documents in the corpus $N = |D|$

$|\{d \in D : t \in d\}|$: number of documents where the term t appears

If the word is very common and appears in many documents, this number will approach 0.

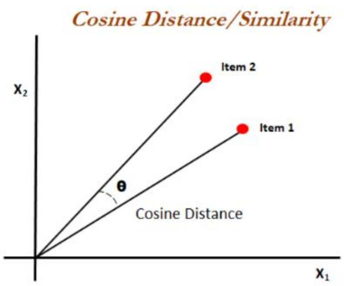
The higher the TF-IDF score, the more relevant that word is in that particular document.

In case of queries with multiple keywords, individual results are added together.

More sophisticated ranking functions are variants of this simple model.

Cosine similarity (IR topic and others)

Cosine similarity measures the similarity between two vectors. It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in roughly the same direction. It is often used to measure document similarity in text analysis.



Can be employed to rank documents based on their relevance to a user query. Documents with vectors closest to the query vector in terms of cosine similarity are considered more relevant and are presented higher in search results.

TF-IDF vectors can be used for the computation.

Web pages ranking

Each document satisfying the query is ranked considering also the context of the terms expressed in the query.

Terms can have different "scores" in the document:

- for example, the presence of a term in the <title> or in <h1> would made the page more relevant with respect to the same term within the page
- **stopwords**, that appear frequently in text but usually bring little information, should not influence the rank and these are ignored when indexing and when retrieving

Problem with text based ranking systems

Suppose a popular movie is just out, and therefore users are querying for the main actress, for example Emma Stone.

By considering words count only, it is possible to obtain high rankings for web pages not related to that specific movie.

Web spammers, in fact, knowing that "Emma Stone" is a popular query, have filled their pages with copies and copies and copies of the name of the actress.... Possibly with the same color of the background...

Rank: second generation search engines

1996: Google exploited the fact that the Web is not only plain text to find authoritative pages:

- link analysis
- anchor text analysis
- click analysis on search results

The importance of pages can be computed by considering links.

A page is more important if it has:

a) a large number of outgoing links?

b) a large number of incoming links?

Link analysis

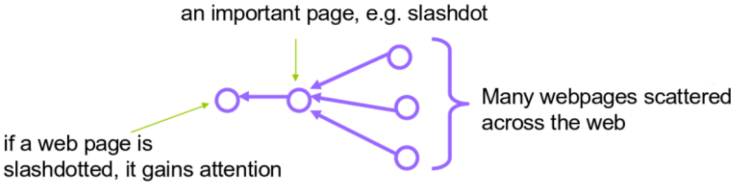
Links can be considered as recommendations:

- web pages "vote" using links
- pages getting more links should be more important

But links from different pages can have:

- high value (e.g., from www.slashdot.org)
- little value (e.g., from www.mysite.it/anypage)
- null value (e.g., from a link spam site)

It is not only the number of pages that point to you, but how many pages point to those pages, etc.



Moreover, the content of a page was judged not only by the terms in the page, but also by the terms used in or near the links pointing to that page.

Spammers can control their pages, but cannot easily control the pages that link to their own pages. The two techinques together made hard for spammers to fool Google...

Math background for PageRank

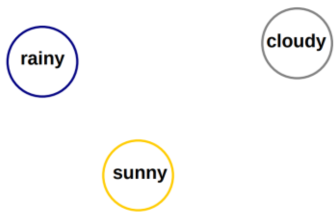
Markov process

The dynamic behavior of a system is described by enumerating all its possible states and defining how the system can move from the current state to the next one.
The change of state of an automaton may be described by a discrete random variable. If the random variable satisfies the memoryless property, then it is called a (discrete space) Markov process.
Memoryless example: National Lottery.

Markov process: three main ingredients

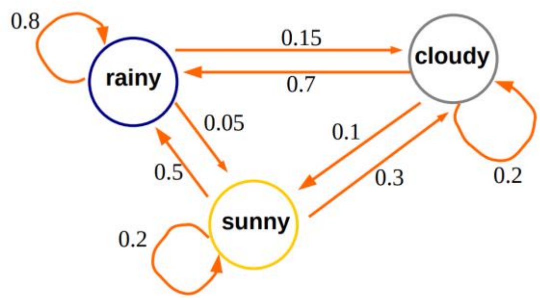
1. State space
2. Markov assumption for future behavior
3. Transition matrix

1) State space: Consider the daily weather change in Dublin, Ireland. Assume the state space is formed by only three weather conditions: rainy, cloudy, and sunny.



$$\Pr(X_{n+1} = x \mid X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) = \Pr(X_{n+1} = x \mid X_n = x_n)$$

2) Markov assumption: Depending only on today current weather (rainy, cloudy, and sunny) the system can change into a new weather tomorrow.



3) Transition matrix: Square matrix with transition probabilities.

$$\mathbf{P} = \begin{pmatrix} 0.8 & 0.15 & 0.05 \\ 0.7 & 0.2 & 0.1 \\ 0.5 & 0.3 & 0.2 \end{pmatrix}$$

Legend

$$\begin{array}{lll} p_{11} = p_{rainy,rainy} & p_{12} = p_{rainy,cloudy} & p_{13} = p_{rainy,sunny} \\ p_{21} = p_{cloudy,rainy} & p_{22} = p_{cloudy,cloudy} & p_{23} = p_{cloudy,sunny} \\ p_{31} = p_{sunny,rainy} & p_{32} = p_{sunny,cloudy} & p_{33} = p_{sunny,sunny} \end{array}$$

In order to forecast the weather in Dublin the day after tomorrow we can compute the matrix P^2.

$$\mathbf{P^2} = \begin{pmatrix} 0.770 & 0.165 & 0.065 \\ 0.750 & 0.175 & 0.075 \\ 0.710 & 0.195 & 0.095 \end{pmatrix}$$

The single-step transition probability matrix can be generalized to the multi-step case (n-step) by computing.

$$p_{ij}^{(n)} = Prob\{X_{m+n} = j \mid X_m = i\}$$

These values are computed by the Chapman-Kolmogorov equation.

$$p_{ij}^{(n)} = \sum_{\text{all } k} p_{ik}^{(l)} p_{kj}^{(n-l)}, \quad 0 < l < n$$

Informally, this says that the probability of going from state i to state j in n steps can be found from the probabilities of going from i to an intermediate state k and then from k to j, by adding up over all the possible intermediate states k.

Successive powers of the matrix P converge* to the limiting matrix

$$\mathbf{P}^{\infty} = \begin{pmatrix} 0.76250 & 0.16875 & 0.06875 \\ 0.76250 & 0.16875 & 0.06875 \\ 0.76250 & 0.16875 & 0.06875 \end{pmatrix}$$

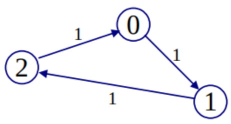
Stationary distribution

Classification of states

If it is possible to go from state x_i to state x_j, then state x_j is accessible from state x_i written as $x_i \to x_j$, if $p_{x_i x_j}^{(n)} > 0$ for some n Two states x_i and x_j communicate, written as $x_i \leftrightarrow x_j$, if they are accessible from each other: $x_i \leftrightarrow x_j$ means $x_i \to x_j$ and $x_j \to x_i$ The states of a Markov chain can be partitioned into communicating classes such that only members of the same class communicate with each other. That is, two states xi and x_j belong to the same class if and only if xi$\leftrightarrow x_j$

A Markov chain is said to be irreducible if it has only one communicating class, e.g., if all states communicate with each other.
A state is said to be recurrent if, any time that we leave that state, we will return to that state in the future with probability 1. If the probability of returning is less than 1, the state is called transient.
If two states are in the same class, either both of them are recurrent, or both of them are transient.

Some Markov chains exhibit a periodic pattern when, starting from a state, the same state is reached again after a number of steps multiple of a given number d, called the period (here we have d=3).

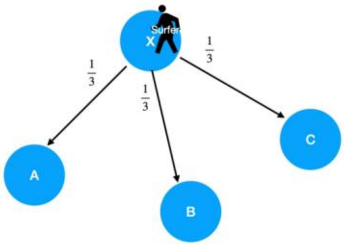


If every state has period 1 then the Markov chain is called aperiodic.
If a Markov chain is both irreducible and aperiodic, it is called ergodic, and it possesses some nice properties such as a unique stationary distribution and convergence of the state probabilities to that distribution over time.

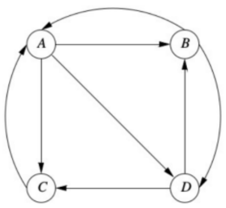
Google PageRank

Google PageRank, developed by Larry Page and Sergey Brin, the founders of Google, revolutionized web search by providing more accurate and relevant search results.
PageRank views a link from one web page to another as a vote or endorsement.
If a page is linked to by many other pages, it is considered more important or valuable.
Of course, only if the linking pages are important as well (recursive definition).

Ranking pages by following a random surfer
The structure of the web graph was used, introducing the idea of a web surfer who follows links at random.
This is a simple model of user browsing: each link in a page has an equal chance to be clicked.



The random surfer following links for a very long time will spend a proportion of time at each node which can be used as a measure of importance of the node.
We can measure how often a page would be visited in the long run and more important pages will be visited most often.



$$P = \begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{bmatrix} 0 & 1/3 & 1/3 & 1/3 \\ 1/2 & 0 & 0 & 1/2 \\ 1 & 0 & 0 & 0 \\ 0 & 1/2 & 1/2 & 0 \end{bmatrix} \end{array}$$

The probability distribution for the position of a random surfer can be described by a vector whose j-th component is the probability that the surfer is at page j.

$v_0 = [¼, ¼, ¼, ¼]$
$v_1 = v_0 * P$, $v_2 = v_1 * P$, $v_3 = v_2 * P$, ...

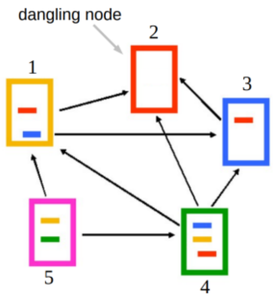
It is known* that this probability distribution approaches a stationary distribution $v = v * P$ if the graph is strongly connected and there are no dangling nodes (e.g., nodes without outgoing links).
**the math here is quite complex*

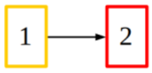
Q1) Is it possible to find a stationary distribution for the web graph?
Q2) The result allows important pages to emerge?

Yes, but only after appropriate changes

Transition matrix

$$P = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/2 & 0 & 0 & 1/2 & 0 \end{bmatrix}$$





$$P = \begin{vmatrix} 0 & 1 \\ 0 & 0 \end{vmatrix}$$

$v = v * P \qquad v_0 = [1, 0]$

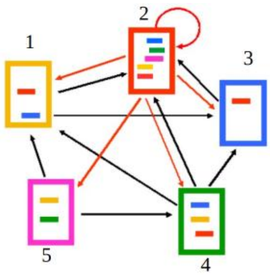
$$v_1 = [1, 0] * \begin{vmatrix} 0 & 1 \\ 0 & 0 \end{vmatrix} = [0, 1]$$

$$v_2 = [0, 1] * \begin{vmatrix} 0 & 1 \\ 0 & 0 \end{vmatrix} = [0, 0]$$

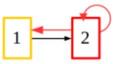
At each step node 2 gets some importance from node 1 but it does not pass it to any other node and therefore it "drains out" this value of importance from the graph.

New transition matrix (no dangling node)

$$P' = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 0 & 1 & 0 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/2 & 0 & 0 & 1/2 & 0 \end{bmatrix}$$



The matrix become stochastic



$$P = \begin{vmatrix} 0 & 1 \\ ½ & ½ \end{vmatrix}$$

$v = v * P \qquad v_0 = [1, 0]$

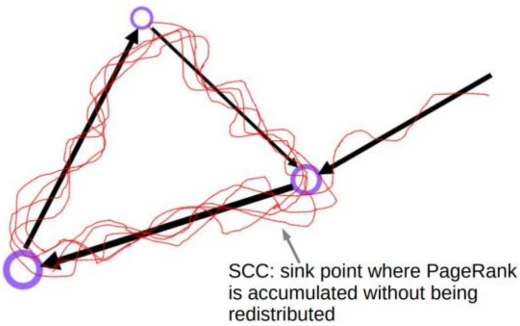
$$v_1 = [1, 0] * \begin{vmatrix} 0 & 1 \\ ½ & ½ \end{vmatrix} = [0, 1]$$

$v_2 = [1/2, 1/2]$
$v_3 = [1/4, 3/4]$
$v_4 = [3/8, 5/8]$
$v_5 = [5/16, 11/16]$
...
$v_{10} = [155/448, 293/448]$
...

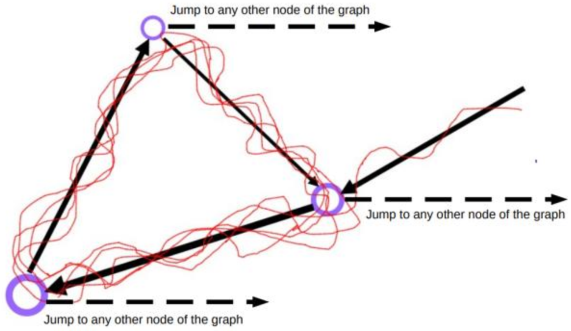
v converges to [1/3, 2/3], the second node has twice the importance of the first

Cycles in the graph

Problem: with pure random walk metric, the random surfer can be "trapped" and end up going in circle.



Solution: the random surfer can jump from the current page to any other randomly selected page.



process called **teleportation.**

Transition matrix

$$P'' = \alpha \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 0 & 1 & 0 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/2 & 0 & 0 & 0 & 1/2 \end{bmatrix} + (1-\alpha) \begin{bmatrix} 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \end{bmatrix}$$

“teleportation”

P'' is stochastic primitive (all values > 0) and is ergodic.

... its structure ensures the convergence to the stationary distribution and the existence of a solution for λ_1v = vP'' with eigenvalue λ_1 = 1

Solution of the Google matrix
α is called the dumping factor and plays a crucial role:
- if α = 1 the matrix is that of the original Web graph
- if α = 0 the Web graph is completely lost

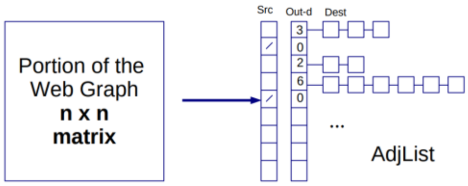
For the Google matrix α is close to 0.85
The stationary v distribution can be computed using the power method

v^0 initial probability vector
repeat until convergence
1. $v^{i+1} = v^i \times P''$
2. $i = i + 1$

After 50-100 iterations the computation converges but the Google Matrix P'' is huge
Moreover...

1) The Web graph is sparse since, on average, a web page has 10 out-links and therefore the number of 0 in the matrix is huge
2) The Web graph is more efficiently stored using an adjacency list

Solution of the Google matrix with AdjList



V₀, initial probability vector

dumping factor **α**

v^0 initial probability vector

Iterative step $i+1$

1. Initialize the new vector $v^{i+1} = \vec{0}$
2. Initialize teleporting probability $p_t = 1 - \alpha$
3. For each node j:
   - If j is a dangling node, then $p_t = p_t + \alpha v^i[j]$

$$P'' = \alpha \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 0 & 1 & 0 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/2 & 0 & 0 & 0 & 1/2 \end{bmatrix}$$

Adjust teleporting probability p_t to take into account the contribution of dangling nodes

v^0 initial probability vector

Iterative step $i+1$

1. Initialize the new vector $v^{i+1} = \vec{0}$
2. Initialize teleporting probability $p_t = 1 - \alpha$
3. For each node j:
   - If j is a dangling node, then $p_t = p_t + \alpha v^i[j]$
   - Else, for each of the m outgoing links from j, let k be the node receiving the link:

$$v^{i+1}[k] = v^{i+1}[k] + \alpha \frac{v^i[j]}{m}$$

4. For each node j, adjust random jumps:

$$v^{i+1}[j] = v^{i+1}[j] + \frac{p_t}{n}$$

Using PageRank in Search Engine

PageRank is query-independent, the computed probability vector is used together with many other factors to decide the order of pages.
The search terms in the query must appear in the pages, and they are more valuable depending on their position in the page (but values have changed to fight spammers).
Many are the signals Google uses to return its results...

"[...] Links do remain a part of the algorithm, but they are now just one of many contributing factors and no longer the driving force pushing webpages to the top of Google's rankings [...]"

Take away points
PageRank is another centrality measure, it is a variation of the eingenvector centrality.
Bring and Page understood that the web is indeed a (large) graph and applied well known math for ranking pages in their search engine, providing a new score that does not take into account the content of the pages, but only the structure of the web graph.

Topic-Sensitive PageRank

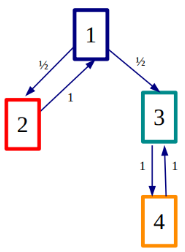
PageRank measures the popularity of web pages without considering their content.
With Topic-Sensitive PageRank the popularity of web pages is computed with respect to specific topics.
This allows to take into account the interests of the users when answering to their search queries, e.g., it provides an early approach to personalization.

In PageRank the random surfer can teleport with probability (1-α)*1/N to any page in the graph to avoid the problem of dangling nodes and cycles (see the Google matrix P'' of the lesson on PageRank).
In Topic-Sensitive PageRank, set of pages on different topics are identified and the random surfer can teleport to those pages relevant for one (or more) topic (teleport set).
The topics of the paper were the 16 top-level categories of DMOZ directory (now closed).

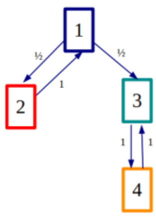
For each topic (category) i, a teleport set T_i of relevant pages is built, and T is the union of all T_i.
Idea: Bias the random walk and change the PageRank matrix considering a new teleport part

$$M_{i,j} = \begin{cases} \beta A_{i,j} + \dfrac{(1-\beta)}{|T|} & \texttt{if } i \in T \\ \beta A_{i,j} & \texttt{otherwise} \end{cases}$$

For each T a different rank vector is computed.
Particularly interesting for ambiguous queries.
"[...] Consider the query "blues" taken from our test set. This term has several different senses; for instance, it could refer to a musical genre or to a form of depression."

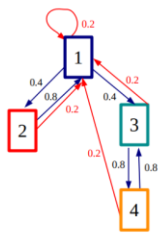


$$A = \begin{vmatrix} 0 & ½ & ½ & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{vmatrix}$$



T = {1}, β= 0.8

$$M = 0.8 \begin{vmatrix} 0 & ½ & ½ & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{vmatrix} + 0.2 \begin{vmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{vmatrix}$$



T = {1}, β= 0.8

$$M = \begin{vmatrix} 0.2 & 0.4 & 0.4 & 0 \\ 1 & 0 & 0 & 0 \\ 0.2 & 0 & 0 & 0.8 \\ 0.2 & 0 & 0.8 & 0 \end{vmatrix}$$

Compute the stationary distribution as seen in PageRank Of course, the results will change in accordance with the value of the parameter β and the selected teleport set T.

Which topic used for the queries?
- Select from a menu
- Classify the query into topic
- Use the context of the query
    - The query is issued from a web page talking about a known topic
    - History of the queries, for example “basketball” followed by “Jordan”
- Use the context of the user
    - Bookmarks

Web Spam

From a web designer's perspective, it is important not only to create a “good quality” website, but also that other pages (possibly “good” ones) link to it so that the website can be easily found.
A large quantity of HTTP traffic passes through search engines and companies get many advantages if their websites result in the first 10/20 results.

What is web spam?
Spam (e-mail): Spam is unsolicited bulk e-mail, it is bad because it shifts the cost of advertising to the recipients.
Web spam:
- Intentional attempts to manipulate search engine rankings for specific keywords or keyword phrase queries.
- According to various studies the amount of web spam varies from 6 to 22 percent.

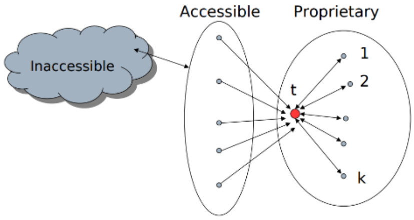
Search engines fight web spam because:
- confuses users with irrelevant results
- causes a considerable waste of resources since spam pages are collected by crawlers (waste of bandwidth), analyzed (waste of CPU), indexed (waste of memory), returned in query results
- spam websites serve as means of malware and adult content dissemination, and phishing attacks

Three categories of spam techniques have been defined
1. Content-based methods
2. Link-based methods
3. Methods based on non-traditional data such as user behaviors, clicks, HTTP sessions

First web spam attack
The first attack to the TF-IDF method used by first generation search engines was in 1995 when search engines themselves started selling search keywords to advertisers.
Queries with these keywords returned also banners and the sponsored sites were improving their positioning.

Link spamming
Spammers have learned to build link structures to increase the importance of web pages.
Pages can be classified as:

- **Inaccessible**, spammers cannot influence outgoing links
- **Accessible**, spammers can modify them in a limited way, for example sending posts on a blog
- **Proprietary**, administered by the spammer (e.g., spam farm)



It might seem surprising that one can affect a page without owning it.
However, sites such as blogs or newspapers invite others to post their comments on the site.

Honey pot, e.g., pages "useful", for example Microsoft documentation, hoping to attract incoming links. In the honey pot there are also hidden links towards those pages willing to increase their ranking.
Link exchange between spammers who build websites that connect each others
Buying expired domains whose pages, with spam content, acquire the importance of the old links.
Note: now Google resets the reputation of expired domains.
Spam farm, arbitrary link structures owned by spammers.

Miscellaneous: Click spam
Search engines use click stream data as an implicit feedback and therefore spammers generate fraudolent clicks on web pages.
Click spam occurs also on online advertising when spammers click on ads of competitors in order to decrease their budgets, make them zero, and place their ads on the same spot.

Many algorithms have been proposed after PageRank: TrustRank, Anti-TrustRank, BadRank, SpamRank.
Also Social Networks suffer from different forms of spam; topics of the moment are fake news, fake images, deepfake.

Google has access to a gigantic quantity of data from:

- Search results
- Chrome browser activities
- Google analytics
- Android

It also uses a new technique to understand users' queries called BERT (Bidirectional Encoder Representations, introduced in 2018).
BERT helps computers understanding the nuances of human language, and it is based on transformers, a type of artificial intelligence model that can analyze all the words in a sentence at once, rather than one by one.

Take away points

Search engines are complex: they collect web pages and return results in response to users' queries.
It is necessary to rank these results considering the texts contained into pages and also their importance in the web graph.
The in-degree distribution is not enough, also the quality of the incoming links needs to be considered.
Many businesses depend on their positions in the results of a search engine and there is an entire search engine optimization industry to help websites improving their search rankings.
Some methods are not approved by search engines since they try to change the returned results (for example "spamdexing").

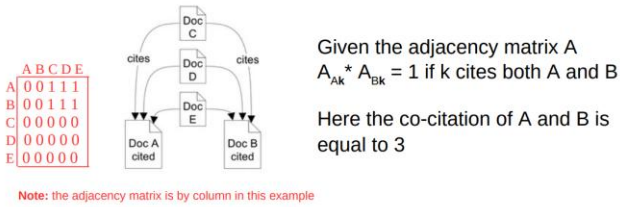
Co-citation & Bibliographic coupling

Sometime directed networks are turned into undirected networks to use analysis techniques that do not exist for the direct case.
Ignoring the direction of the edges is straightforward, but many useful information on the network are lost.
More sophisticated approaches to build undirected graphs are:

- Co-citation
- Bibliographic coupling (co-reference)

Co-citation

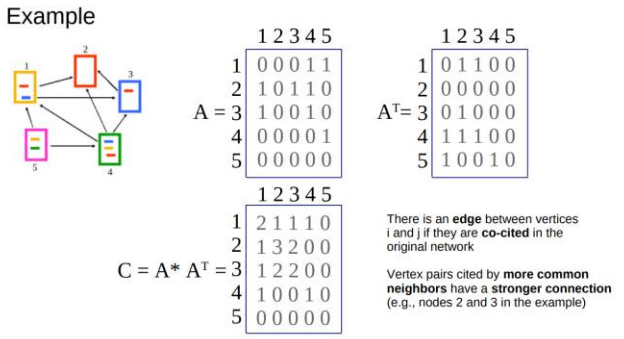
The co-citation of two vertices i and j in a directed network is the number of vertices that have outgoing edges that point to both i and j.
In the language of citation networks, the co-citation of two papers is the number of other papers that cite both.

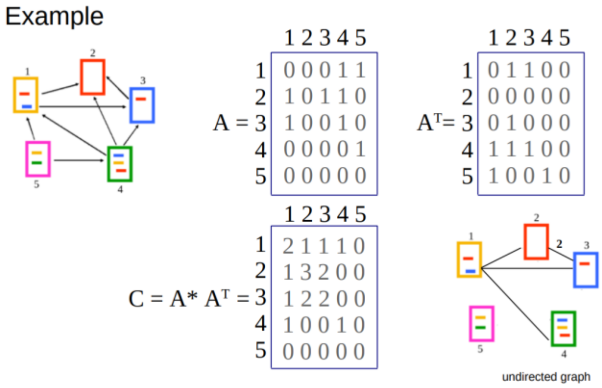


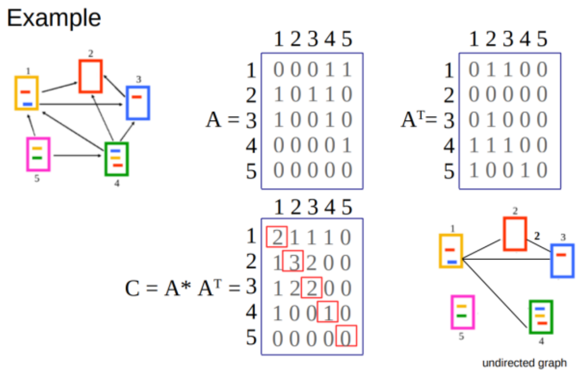
The co-citation of nodes i and j is

$$C_{ij} = \sum_{k=1}^{n} A_{ik} A_{jk} = \sum_{k=1}^{n} A_{ik} A_{kj}^{T}$$

The n x n adjacency matrix C of the corresponding cocitation network is $C = AA^T$





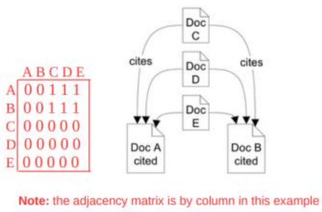


Diagonal elements are the in-degrees of the original network but are generally set to zero in the matrix C.

Co-reference

The co-reference of two vertices i and j in a directed network is the number of other vertices they both point to.

In the language of citation networks, the co-reference is the number of other papers that both i and j cite.



Given the adjacency matrix A

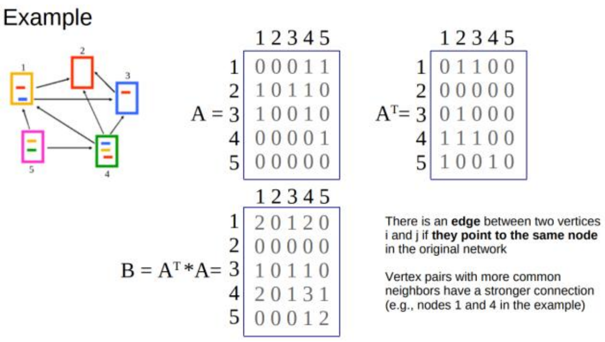
$A_{kC} * A_{kD} = 1$ if C and D both cite k

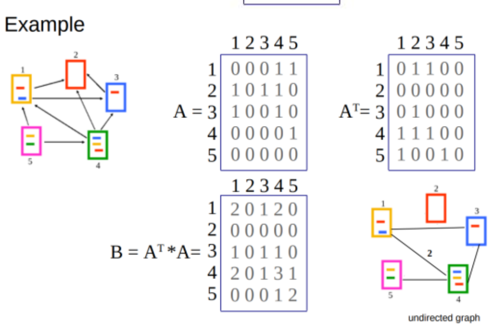
Here the co-reference of C, D, E is equal to 2

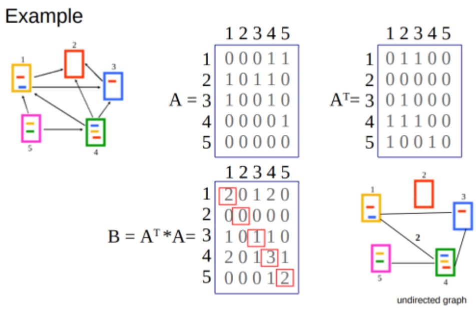
The co-reference of nodes i and j is

$$B_{ij} = \sum_{k=1}^{n} A_{ki} A_{kj} = \sum_{k=1}^{n} A_{ik}^{T} A_{kj}$$

The n x n adjacency matrix B of the corresponding coreference network is $B = A^T A$

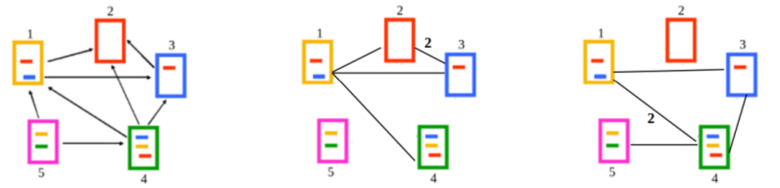






Diagonal elements are the out-degrees of the original network but are generally set to zero in the matrix B.

Co-citation vs Co-reference



a) Directed network (b) Co-citation network (c) Co-reference network

Mathematically similar measures, but give different results.
In the citation network:

1. Strong co-citation:
   a. both papers are pointed to by many same papers
   b. both papers have a lot of incoming links in the first place
   c. if they are frequently cited together by others, there is some commonality or association between them
2. Strong co-reference:
   a. both papers cite many other papers
   b. they have large bibliographies
   c. surveys, review articles, and so on

The sizes of bibliographies vary less than the number of citations, hence co-reference is a more uniform indicator of paper similarity.
Co-reference can be computed as soon as the paper is published.
Co-citation can be computed only after the paper has been cited and therefore changes over time.
Differences between incoming and outgoing links in a directed network (cf. PageRank vs HITS).

Hubs and Authorities

The Problem of Ranking
In PageRank get a high centrality (i.e., a high rank) those pages pointed by other important pages.
However, in other cases it is appropriate to accord a vertex high centrality if it points to other vertices with high centrality:

- For example a review paper in the citation network
- Web directories like dmoz or Yahoo! directory (now closed)

In a directed network there are two types of important nodes:

- Authorities are nodes that contain useful information on a topic of interest
- Hubs are nodes that tell us where the best authorities can be found

Two different centrality measures can be computed for directed networks, the authority centrality and the hub centrality, and this idea was proposed in 1998 by Jon Kleinberg in the link analysis algorithm called Hyperlink-Induced Topic Search or HITS.

HITS: idea behind
Kleinberg distinguished between two types of Web pages which pertain to a certain topic.
The first are authoritative pages and they are valuable because they provide information about a topic.
The second are hub pages and they are valuable because tell you where to go to find out about a given topic (resource lists).

Just as PageRank uses the recursive definition of importance that
“a page is important if important pages link to it”

HITS uses a mutually reinforcing relationship
“a good hub will point to many authorities, and a good authority will be pointed at by many hubs”

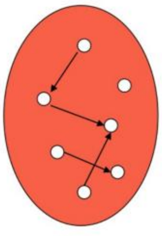
HITS: one-word query “newspapers”
There is not necessarily a single “best” answer but a mix of prominent newspapers (i.e., the results you want) along with pages that are going to receive a lot of in-links no matter what the query is — pages like Wikipedia, Facebook, Amazon, and others.
A generic query like “newspaper”, returns on-line journals but also pages that get a high score, independently from the query itself.
Some pages have been able to recognize “good” pages for the query It is reasonable to think that these pages can find good answers These pages are the hubs.
Propagating score values backwards we obtain.
Pages that recognize good answers should have a higher impact. Hence, we propagate votes forward...

Normalize auth with the sum of auth of all nodes.
Normalize hub with the sum of hub of all nodes.

HITS
This is HITS, and deals with the computation of a fix point involving repeated matrix multiplication.
But, unlike PageRank, HITS is query dependent, e.g., first finds those pages that satisfy the query and then computes the ranking at query time only on this subset of relevant pages.
HITS works on a reduced set of pages Sσ which is query dependent.

The set S_σ should:
- be relatively small
- be rich of relevant pages
- contain the main authorities on the subject

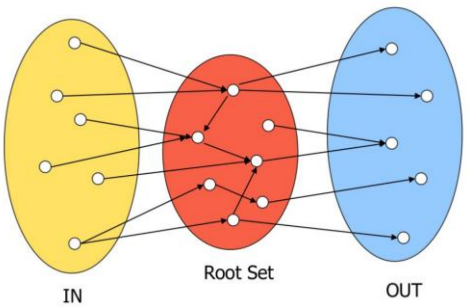
HITS: page set

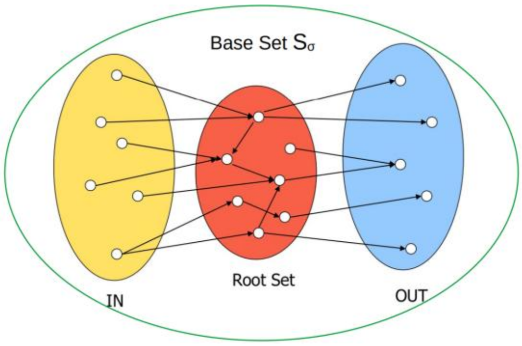


Root Set

The Root Set contains the first top t pages satisfying the query (term-based search):

- This set is generally small and rich of relevant pages
- Authorities not necessarily contain text matching the query
- ... but they link to relevant pages

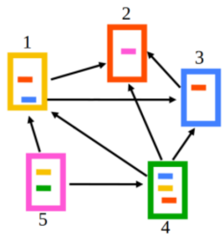


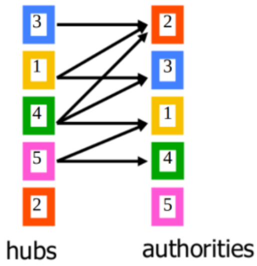


HITS: link filtering

S_σ is then reduced by deleting navigational links connecting pages in the same domain.
S_σ has limited size, it is focused on the query and contains relevant and authoritative pages.
Authoritative pages should "emerge"...

HITS: hubs and authorities



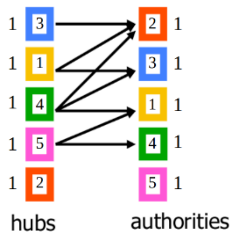


Each page i has a double role: hub and authority and therefore acquires a pair of weights:

- a_i as authority
- h_i as hub

Initially all weights are equal to 1, for each page i:

- a_i = 1
- h_i = 1



Until convergence, for each page i repeat

- Authority Update Rule authorities collect the weights of the hubs

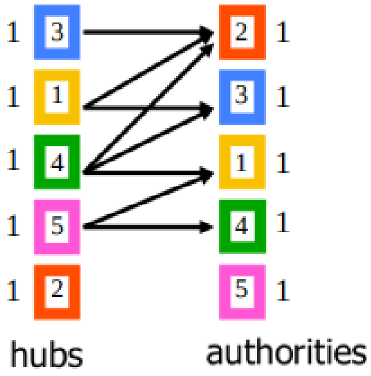
$$\boxed{a_i = \sum_{j:j \to i} h_j}$$

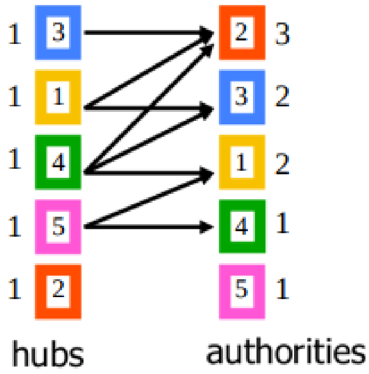
- Hub Update Rule hubs collect the weights of the authorities

$$\boxed{h_i = \sum_{j:i \to j} a_j}$$

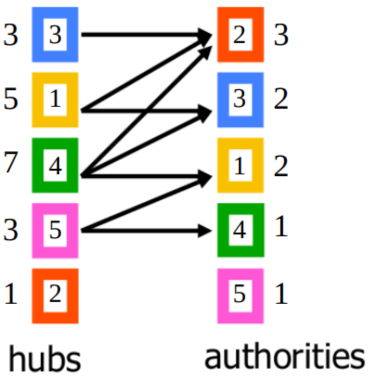
- Normalize weights under some norm

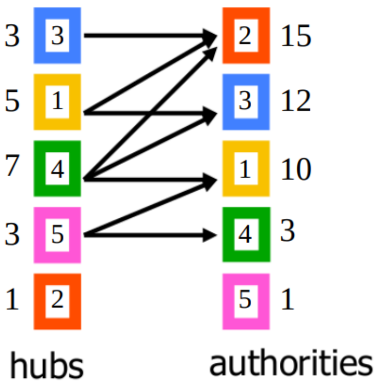
Kleinberg suggests Euclidean norm $\|\boldsymbol{X}\|_2 := \sqrt{x_1^2 + \cdots + x_n^2}$

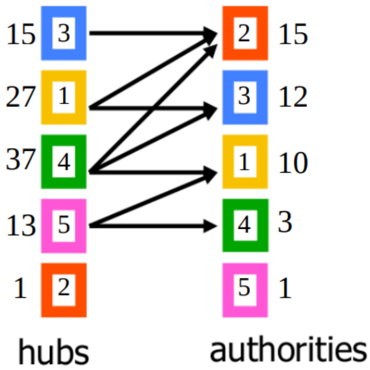




Normalization should be computed at each step!







HITS: in matrix term…

Given the adjacency matrix A of the graph.

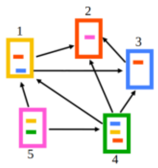
The scores for authorities/hubs can be computed as:

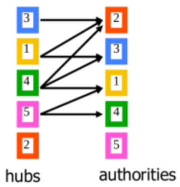
- a = $A^T h$
- h = Aa

Hence we have:

- $a = A^T A a$
- $h = AA^T h$

- **a =** $\boxed{\mathbf{A^T A}}$**a** → Co-reference matrix
- **h =** $\boxed{\mathbf{AA^T}}$**h** → Co-citation matrix



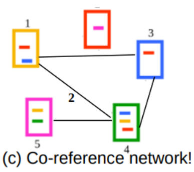


********* HUB MATRIX ***************

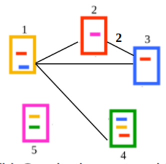
```
[[2. 0. 1. 2. 0.]
 [0. 0. 0. 0. 0.]
 [1. 0. 1. 1. 0.]
 [2. 0. 1. 3. 1.]
 [0. 0. 0. 1. 2.]]
```

********* AUTHORITY MATRIX *********

```
[[2. 1. 1. 1. 0.]
 [1. 3. 2. 0. 0.]
 [1. 2. 2. 0. 0.]
 [1. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0.]]
```



(c) Co-reference network!

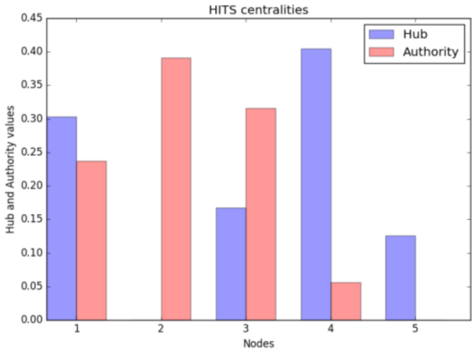


(b) Co-citation network!

*********** HITS CENTRALITIES *************

```
(
 {1: 0.3028419086392422, 2: 0.0, 3: 0.16745199220945273, 4: 0.4042648717068945, 5: 0.12544122744441058},
 {1: 0.23681288036482998, 2: 0.390984323456401, 3: 0.31612245503718583, 4: 0.05608034114158311, 5: 0.0}
)
```

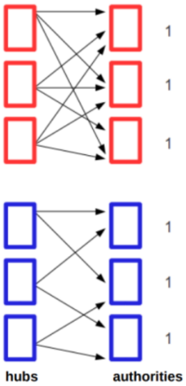
HITS: solution with NetworkX



HITS: TKC (Tightly Knit Community)

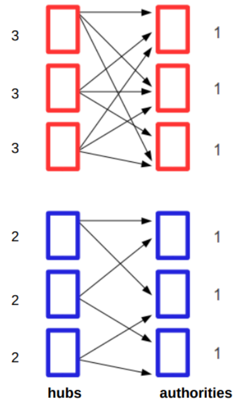
Tightly knit community: a number of web pages all link to each other and thus all receive high authority values, even though their true value may be much smaller

The HITS algorithm favors these **most dense community** of hubs and authorities

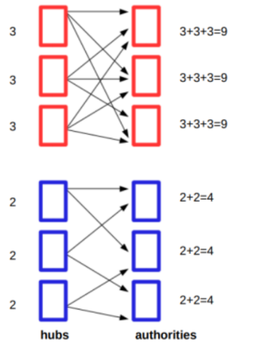


Note: this is a fictitious example to understand the problem

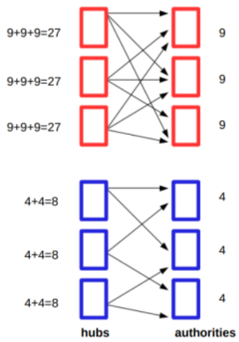
Update
hub
score



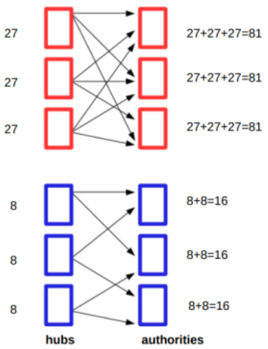
Update
auth
score



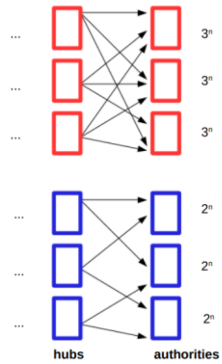
Update
hub
score

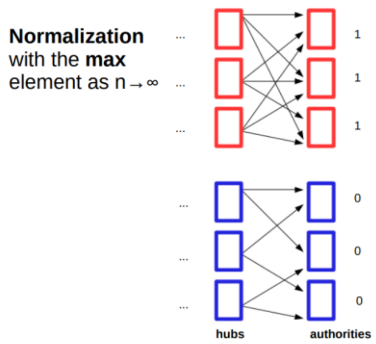


Update
auth
score



After **n**
iterations





Normalization should be computed at each step!

HITS: advantages/disadvantages

Pros: Works on graphs already containing interesting pages.
Cons: Suffers from the so called TKC effect.

Moreover, no applications in which HITS excels have been found yet (it is used in the search engine Teoma, today Ask.com).

Take away points

The Web has shifted much of the information retrieval question from a problem of scarcity to a problem of abundance.
A search engine has no problem finding and indexing millions of millions of documents.
The problem is that the human being performing a search wants to look at only a few of these.
Understanding the network structure of Web pages has been crucial for addressing this issue.

PageRank **precomputes a rank vector** that provides **a-priori "importance" estimates** for **all of the pages on the Web**	HITS relies on **query-time processing** to deduce the **hubs** and **authorities** that exist in a **subgraph of the Web** consisting of both the **results to a query and the local neighborhood** of these results
This vector is computed once, **offline**, and is **independent of the search query**	Suffers of the **TCK effect**
At **query time**, these importance scores are used **in conjunction with query-specific IR scores** to rank the query results	
PageRank can be considered as **a kind of "fluid"** that circulates through the network, passing from nodes across the edges	HITS rates web pages, and it is also known as **voting by in-links:** scores start from 1 and then are updated at the sum of the incoming links

Patterns of relations

Global, statistical properties of networks:

- Average degree
- Average clustering
- Average path length

Local, per vertex properties:

- Centrality metrics (e.g., pagerank)

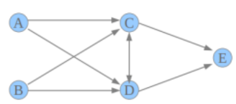
Pairwise properties:

- Node equivalence
- Node similarity
- Mixing patterns

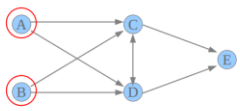
Networks describe connections between entities and pairwise properties describe the relationship between two individual nodes.
Analyzing pairwise properties across the network can expose larger patterns that might not be evident when looking at individual nodes or the overall network structure.
For instance, we can discover communities formed by nodes with dense pairwise connections.

Structural equivalence

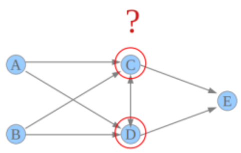
(In social network analysis), two nodes are considered structurally equivalent if they have the same neighborhoods – that is, they are connected to the same others.



	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	1	0
C	0	0	0	1	1
D	0	0	1	0	1
E	0	0	0	0	0

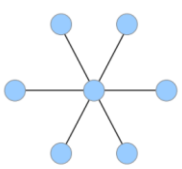


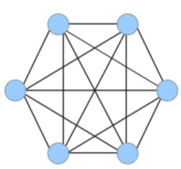
	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	1	0
C	0	0	0	1	1
D	0	0	1	0	1
E	0	0	0	0	0

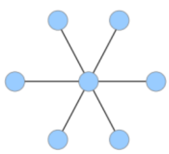


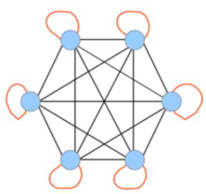
	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	1	0
C	0	0	0	1	1
D	0	0	1	0	1
E	0	0	0	0	0

Structurally equivalent nodes are totally interchangeable, but this is a very strict definition.

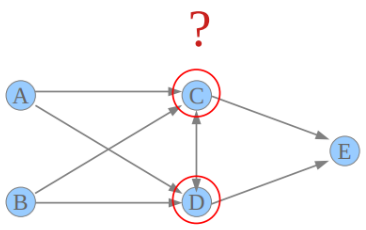








Automorphic equivalence
Less demanding than structural equivalence, automorphic equivalence allows to identify symmetries in graphs.
Two nodes (or sets of nodes) are automorphically equivalent if there exists a relabeling of the nodes that maps one node to the other while preserving all connections.



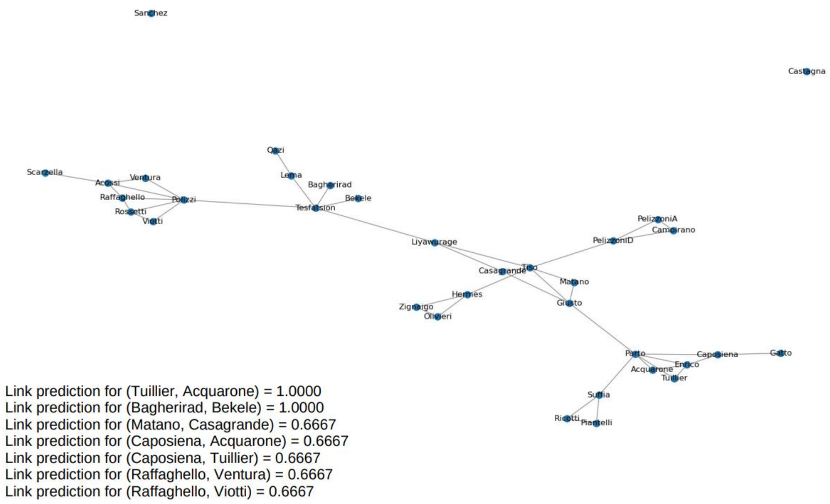
Similarity measures
In real life it is difficult to find networks whose nodes are structurally or automorphically equivalent.
When the neighborhood of the nodes overlap partially, we can use relaxed metrics, e.g., similarities.
For example:
- Jaccard similarity
- Cosine similarity
- Pearson correlation coefficient

Jaccard similarity
Jaccard similarity, also called the Jaccard Index or Jaccard Coefficient, is a classic measure of similarity between two sets introduced by Paul Jaccard in 1901.
Given two sets, A and B, the Jaccard Similarity is defined as the size of the intersection of set A and set B (i.e., the number of common elements) over the size of the union of set A and set B (i.e., the number of unique elements).

Jaccard similarity in networks
Represents the similarity of the neighborhoods N( ) of nodes: two nodes are considered similar if they share many of the same neighbors.

$$J_{AB} = \frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|}$$

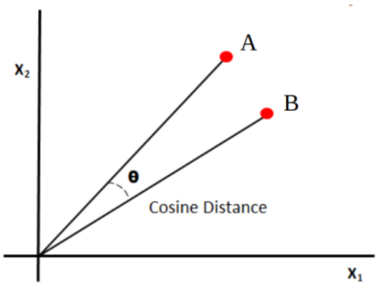
Jaccard coefficient in NetworkX
NetworkX provides the function nx.jaccard_coefficient(), which computes the Jaccard coefficients (by default only for non-adjacent node pairs).
It can be used for link prediction, assuming that:
- nodes with many common neighbors are more likely to form a link in the future
- the Jaccard coefficient measures this likelihood by comparing the overlap of neighbors between two nodes



(Tuillier, Acquarone) and (Bagherirad, Bekele) are the most likely missing links, but also other links will be possible in the future (see Exercise 10)

Cosine similarity

Cosine similarity measures the similarity between two vectors. It is measured by the cosine of the angle between two vectors and determines whether they are pointing in roughly the same direction. Often used to measure document similarity in text analysis.



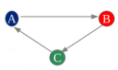
$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

Cosine similarity values range from -1 to 1, depending on the angle between the two vectors:

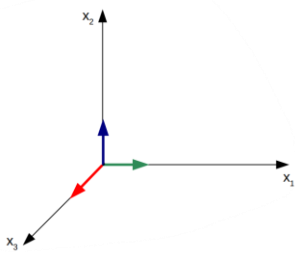
- 1, if the vectors point in the same direction
- 0, if the vectors are orthogonal (no similarity)
- -1, if the vectors are opposite (point in opposite directions)

Cosine similarity in networks

Nodes in the adjacency matrix can be seen as ndimensional vectors.
Cosine similarity calculates the cosine of the angle between pairs of node vectors and tells how "aligned" they are.



	A	B	C
A	0	1	0
B	0	0	1
C	1	0	0



In the case of networks, cosine similarity values generally range from 0 to 1:
- 0, if there are no common connections between the two nodes
- 1, if the nodes connect to exactly the same other nodes
- values between 0 and 1 mean some common connections, with higher values indicating a stronger degree of overlap

Negative values are possible only in the case of negative edge weights.
Higher cosine similarity implies a higher chance of a future link.
Cosine similarity is not implemented with a built-in function in NetworkX.

Pearson correlation coefficient

In statistics, the Pearson correlation coefficient is a measure of linear correlation between two sets of data.
It is given by the covariance of two random variables X and Y, divided by the product of their standard deviations.
The covariance tells how two random variables X and Y move together, for example if larger values in X tend to coincide with larger values in Y (and vice versa for smaller values), the covariance is positive.
Covariance does not have a fixed range of values and it can theoretically take on any number, from positive infinity (+∞) to negative infinity (-∞).

$$cov(x, y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

The Pearson correlation is a normalized version of the covariance and ranges in [-1,1]

$$\rho_{xy} = Correlation\ (x, y) = \frac{cov(x, y)}{\sqrt{var(x)}\sqrt{var(y)}.}$$

Examples of **positive** correlation:
- The age and height of a sample of teenagers from a high school should have a Pearson correlation coefficient significantly greater than 0, but less than 1, which corresponds to perfect correlation, e.g., all students of the same age have the same height
- The less time you spend marketing your business, the fewer new customers you will have

Examples of **negative** correlation:
- If a train increases speed, the time to get to the destination decreases
- The more time you prepare for a test, the fewer mistakes you will make
- When the supply of a particular product decreases, the demand for it increases

The correlation coefficient ranges from −1 to 1:
- 1 implies that a linear equation describes the relationship between X and Y perfectly, with all data points lying on a line for which Y increases as X increases
- -1 implies that all data points lie on a line for which Y decreases as X increases
- 0 implies that there is no linear correlation between the variables

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

Pearson correlation coefficient in networks
For two nodes it measures how similar are their neighbor structures.
Compares the number of common neighbors with the expected value in a network where the same nodes are connected randomly.
This quantity lies strictly in the range [-1, 1] and is the base for measuring assortativity by degree in networks.

In Erdős-Rényi (ER) random graphs, usually the Pearson correlation coefficient is close to 0:

- By design, ER graphs exhibit no significant correlation in node connectivity, and they are often used as a baseline model for comparison

In the Barabási-Albert (BA) model, the Pearson correlation coefficient is typically negative (around -0.1 to -0.2):

- By design, BA networks show a negative Pearson correlation coefficient, with a strong presence of hubs that link to low-degree nodes

Assortativity

Homophily
One of the most basic notions governing the structure of social networks is homophily - the principle that we tend to be similar to our friends.
Viewed collectively, friends are generally similar along:

- racial and ethnic dimensions
- age
- places they live
- occupations, interests, beliefs, opinions…

Homophily provides us with a first, fundamental illustration of how a network's surrounding contexts can drive the formation of its links.

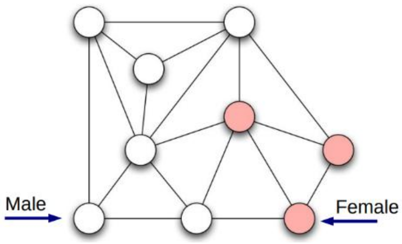
Assortative and Disassortative mixing
Homophily, i.e., the tendency to associate with others we perceive as similar is also called assortative mixing.
More rarely, we encounter disassortative mixing, i.e., the tendency for people to associate with others who are unlike them (opposite attracts).

Measuring assortative mixing
Given a particular characteristic of interest (a category or attribute), is there a simple test that can be applied to estimate whether a given network exhibits homophily?
If the values of the category are enumerative, e.g.,

- nationality, race, gender in the case of people
- language in the case of web pages
- habitat in the case of species

the network is assortative if a significant fraction of the edges in the network run between nodes of the same type.

Example: What would it mean for a network not to exhibit homophily by gender?



It would mean that the proportion of male and female friends of a person looks like the male/female distribution in the full population. Friendships are formed as though they were created randomly across the given characteristic.

p fraction of individuals are male
q fraction of individuals are female

- End-to-end male: p*p
- End-to-end female: q*q
- Cross-gender edge: p*q + q*p = 2p*q

Homophile test: If the fraction of cross-gender edges is significantly less that 2p*q, then there is evidence of homophile.

9 nodes, 18 edges
6 males (p = 6/9 = 2/3)
3 females (q = 3/9 = 1/3)
of the 18 edges, 5 are cross-gender
fraction of links: 5/18 = 0.277
2p*q = 2*2/3*1/3 = 4/9 = 0.444
0.277 cross-gender edges << 0.444
Hence the network shows some evidence of homophily.

What if we do not have any category but we want to compute assortativity in a network?
We can use the degrees of the nodes!

Mark Newman proposed to compute assortativity as the the number of edges that run between vertices of the same type minus the number of such edges we would expect to find if the configuration model is assumed, that is if edges were positioned at random while preserving the node degrees.
This quantity highlights a non-random pattern in network connections.
Lies in the range [-1,1]

It is positive if the number of edges within groups exceeds the number expected on the basis of chance.

By **categorical attributes**

- Given a network with nodes labeled with categories (gender, race, nationality), how much more often do categories match across edges than expected at random?

By **scalar values**

- Given a network with nodes labeled with scalar values (age, salary, weight), how much more often do values match across edges than expected at random?

By **degree**

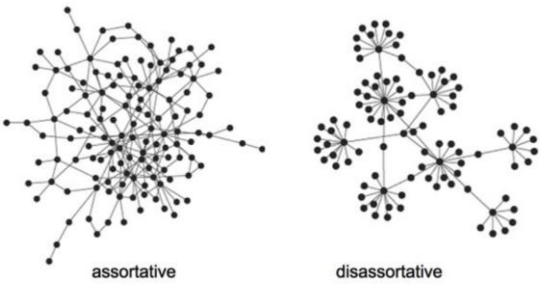
- special case of assortative mixing where the scalar value is node degree
- particularly interesting because, unlike age or income, the degree is itself a property of the network structure

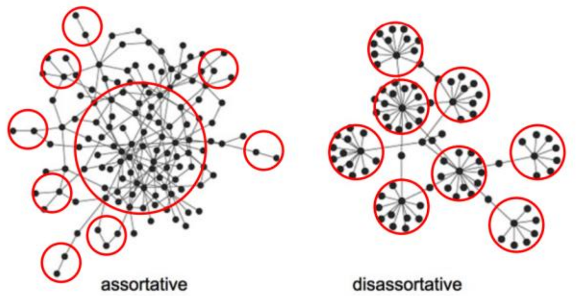
Degree assortativity is Pearson correlation (!) applied to node degrees in a network

$$\boxed{r = \frac{\sum_{ij}\left(A_{ij} - \frac{k_i k_j}{2m}\right) k_i k_j}{\sum_{ij}\left(k_i \delta_{ij} - \frac{k_i k_j}{2m}\right) k_i k_j}}$$

If **r > 0**, high-degree nodes prefer high-degree nodes (assortative mixing).
If **r < 0**, high-degree nodes prefer low-degree nodes (disassortative mixing).

Degree assortativity





Core of high-degree nodes surrounded by a less dense **periphery**

High-degree nodes tend to connect to low-degree ones, creating star-like features in the network

Assortativity in real networks

	network	n	r
real-world networks	physics coauthorship[a]	52 909	0.363
	biology coauthorship[a]	1 520 251	0.127
	mathematics coauthorship[b]	253 339	0.120
	film actor collaborations[c]	449 913	0.208
	company directors[d]	7 673	0.276
	Internet[e]	10 697	−0.189
	World-Wide Web[f]	269 504	−0.065
	protein interactions[g]	2 115	−0.156
	neural network[h]	307	−0.163
	food web[i]	92	−0.276
models	random graph[u]		0
	Callaway *et al.*[v]		$\delta/(1+2\delta)$
	Barabási and Albert[w]		0

None of the values of r in the previous table are of large magnitude, but there is a clear tendency:

- The social networks have positive values of r, hence assortative mixing by degree
- The other networks (technological, information, biological) have negative r, denoting disassortative mixing

Newman has shown that assortativity has a direct relation with the emergence of the giant component.

Assortative networks

- The phase transition point moves to a lower , since the giant component emerges for < 1 (high degree nodes connect to each other in the core)

Disassortative networks

- The phase transition is delayed and these networks have difficulty forming the giant component

Assortativity has also a direct relation with the robustness of the network, in terms of connectivity of the network.

Assortative networks

- A failure of a high degree node would leave other high degree nodes connected to one another, minimizing the chance of the network to become disconnected

Disassortative networks

- High degree nodes are less connected to one another and the failure of a high degree node would have more impact on the connectedness of the network

Degree correlation

A network displays degree correlation if the number of links between the high- and low-degree nodes is different from what expected by chance.

The information about potential degree correlation can be captured by the degree correlation matrix M whose:

- rows and columns represent degree values
- eij are the probabilities of finding nodes with degrees i and j at the two ends of a randomly selected link

The degree correlation matrix M can be defined:

- counting the number of edges that connect nodes of degree i and j in the network
- normalizing by the total number of edges

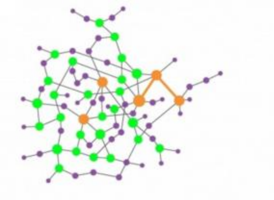
For undirected networks the matrix is symmetric

	1	2	3	4	5
1					
2					
3				0.003	
4			0.003		
5					

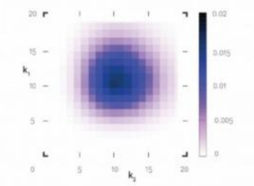
$$\sum_{i,j} e_{ij} = 1$$

Probability of an edge between nodes of degrees 3 and 4

Random mixing



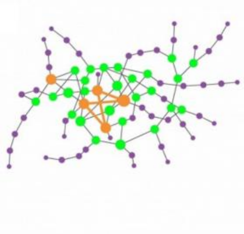
Nodes link to each other randomly



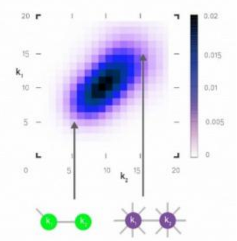
The density of links is symmetric, around the average degree

N=1000, <k>=10

Assortative network

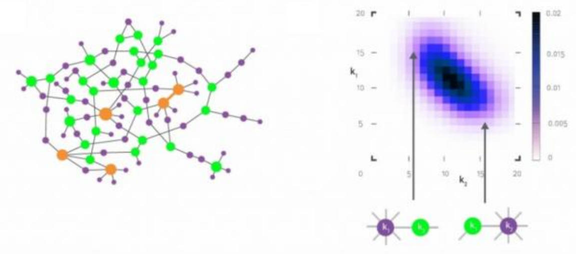


Nodes with comparable degree tend to link to each other: small-degree with small-degree, hubs with hubs



The density of links is high along the main diagonal

Disassortative network



Small-degree nodes tend to connect with hubs and viceversa

The density of links is high along the secondary diagonal

Take away points

Assortativity measures the tendency of nodes with similar attributes to connect with each other. The study of degree correlation through the inspection of the matrix M is difficult and therefore the coefficient r proposed by Newman is most often used.

Possible extensions include:

- considering multiple characteristics and explore how nodes with similar combinations of multiple attributes tend to connect
- considering how nodes with similar attributes are connected not just directly, but also indirectly through paths or walks in the network

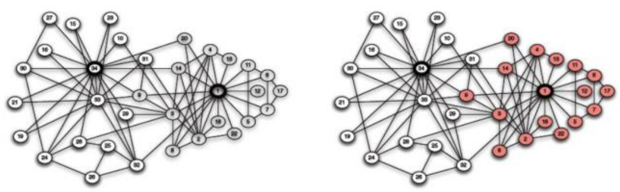
Communities in networks

Community structure

Why do we look for community structure?

- Find cohesive subgroups
    - e.g. family, friends, terrorists…
- Measure isolation of subgroups
- Understand opinion dynamics / adoption
- Understand human disease

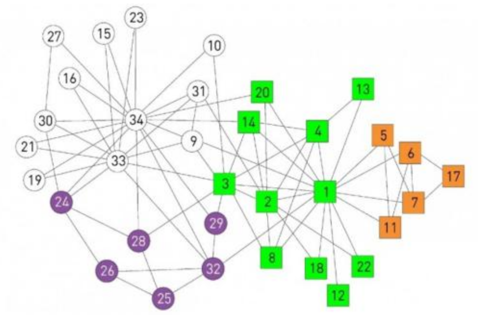
Zachary Karate Club



(a) *Karate club network*

(b) *After a split into two clubs*

A conflict between the club's president and the instructor split the club into two. About half of the members followed the instructor and the other half the president, a breakup that revealed the ground truth, representing club's underlying community structure.



Today community finding algorithms are often tested based on their ability to infer these communities from the Zachary Karate Club network before the split.
Note: the function in NetworkX labels the nodes differently

Community structure
Communities are groups that are densely connected among their members, and sparsely connected with the rest of the network: high cohesion and high separation.
Can reveal hidden information about complex networks not easy to detect by simple observation.
Many algorithms exist, some work on nodes, others on links, some assign each node to a single community (partition), others can find overlapping communities (cover).

Many algorithms
Many algorithms, but some with limitations.
For example, some proposals:

- do not perform well on large networks
- need to fix in advance the number of communities
- cannot recognize overlapping communities
- depend on multiple parameters
- are unable to discover small communities
- are domain-specific, e.g., work with specific structures
- do not generate stable partitions

Find communities in networks
Two general classes of algorithms:

1. **Graph partitioning**
2. **Community detection**

They are distinguished from one another by whether the number and size of the groups are fixed by the experimenter or whether they are left unspecified.

1. **Graph partitioning**

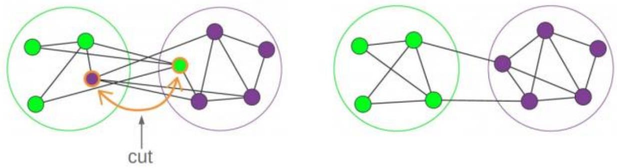
Classic problem in computer science, studied since the 1960s.
A partition is a division, or grouping of a network into communities, such that each node belongs to only one community.
The number of all possible partitions is the Bell number and increases very fast with the number of nodes. Finding the best partition of a network by enumerating all possible partitions is hopeless.

Classical algorithms divide the nodes of a graph into a given number (fixed a priori) of non-overlapping groups such that the number of links (or weights) between the groups is minimized.

Used for example in:

- parallel computing, to distribute tasks to processors, trying to minimize the number of communication links between the processors, to speed up the computation
- VLSI design, to optimize chip layout minimizing the connections between its elements, leading to a more efficient design

<u>Example: Graph bisection</u>



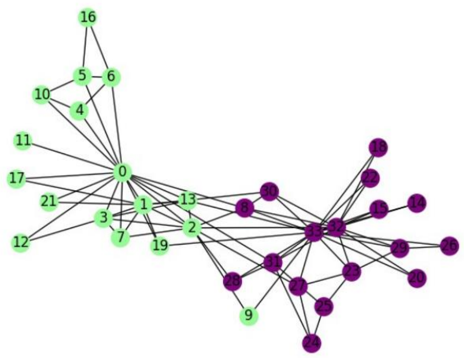
Goal: split the network in two groups, typically of equal size:

- Start from an arbitrary partition in two groups
- Among all pairs (i, j) find and swap the pair that reduces the cut size by the largest amount; each pair can be moved only once

Problems:

- The solution depends on the choice of the initial partition
- Finding the perfect split that minimizes connections across halves is known to be an NP-hard problem, meaning there is no known efficient solution for all cases

<u>Example: Graph bisection</u>



Kernighan-Lin algorithm for bisection.
Greedy algorithm: tries to minimize the cut size at each step, can get stuck in a local optima.

What if we want to achieve graph partitioning into more than two partitions?
Recursive bisection:

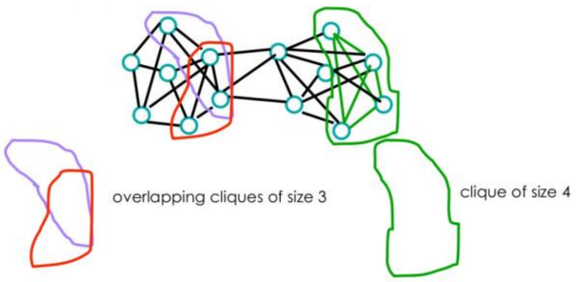
- Apply the Kernighan-Lin algorithm or another bisection method to initially split the graph into two sets
- Recursively apply the bisection method on each of the newly formed sets
- Until the desired number of partitions is computed

Multi-level graph partitioning (sophisticated):
- Create a smaller graph that captures the overall structure by iteratively merging nodes
- Perform bisection on the smaller graph
- The partition information is then projected back to the original graph and refined to improve the quality of the partition
- Repeat the process of coarsening, bisection, and refinement of the partitions, until the desired number of partitions is reached

Are graph partitioning algorithms interesting for finding communities in networks?
In community detection we want to separate the network into groups of nodes that have few connections between them BUT the missing point here is that the number or size of the groups should not be fixed in advance.

What makes a community?
1. Mutuality of ties:
   a. "everybody" in the group knows everybody else
2. Frequency of ties among members
   a. "everybody" in the group has links to at least k others in the group
3. Closeness or reachability of subgroup members
   a. "individuals" are separated by at most k hops

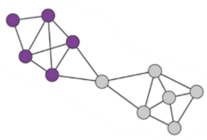
1. Mutuality of ties: Maximum cliques
Cliques are communities: each member of the group has links to every other member (density1).
Cliques can overlap.



Problems with cliques
Not robust: one missing link can disqualify a clique.
Not interesting:
- everybody is connected to everybody else
- no core-periphery structure
- no centrality measures apply

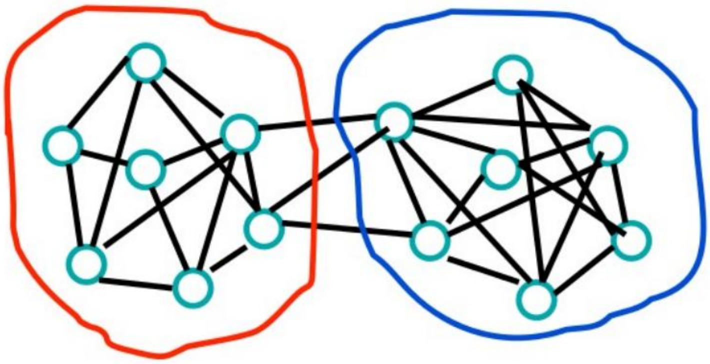
How cliques overlap (e.g., nodes are members of different communities) can be more interesting than the fact that cliques exist in a network.

2. Frequency of ties: Strong and Weak communities
Less rigid than cliques:
- Strong community C: each node in C has more links within the community than with the rest of the graph
- The internal degree of each node exceeds its external degree

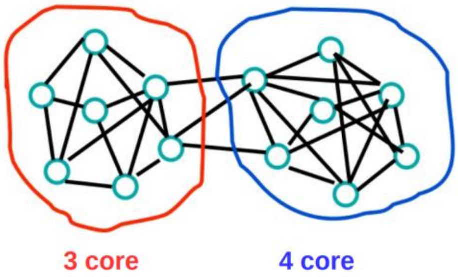


- Weak community C: the sum of the internal degrees of all nodes exceeds the sum of their external degrees
- There may be individuals who are part of the community but not as strongly integrated
- These communities often have members who act as connectors to other communities

2. Frequency of ties: k-cores
Each node within a group is connected to at least k other nodes in the group.



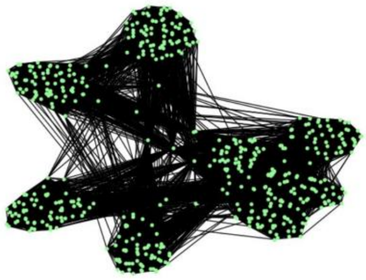
Questions:
- What is the k for the core circled in red?
- What is the k for the core circled in blue?



"everybody in the group has links to at least k others in the group"

Core decomposition
The **0-core** of a graph is simply the entire graph since every vertex has at least zero edges.
The **1-core** of a graph contains the vertices that are connected to other vertices. To form the 1-core simply throw away the set of isolated vertices.
From the 1-core it is possible to form the **2-core** by removing nodes of degree 1 until everything that is left has degree at least 2.

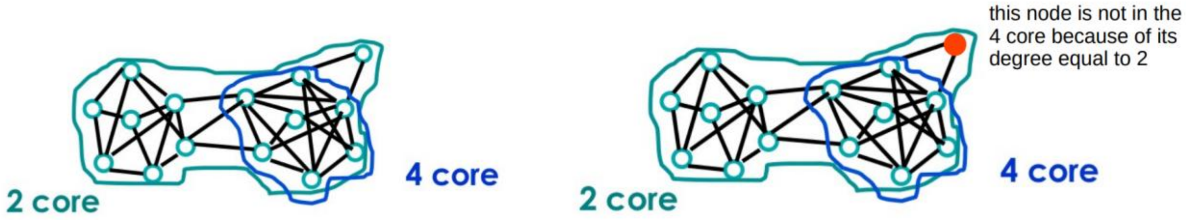
Core decomposition is helpful for filtering out peripheral nodes when visualizing large networks.



Original dataset
Number of nodes: 15220
Number of links: 194103

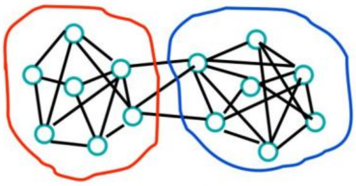
k_core(G, k=60)
Number of nodes: 526
Number of links: 21001

k-cores: problem
Also this definition can be too stringent for identifying “natural” communities.



3. Closeness or reachability: k-distance cliques
k-distance-cliques:
- the maximal distance between any two nodes in a subgroup is equal to k

Theoretical justification:
- information flow through intermediaries (individuals are separated by at most k hops)



Questions
- What is the value of k in this network?
- What is the value of k in a clique?

Communities in networks (cnt)

Which definition of community?
Different methods to find “communities” exist:
- Which is the “right” one?
- How many ways can we group the nodes of a network into communities?

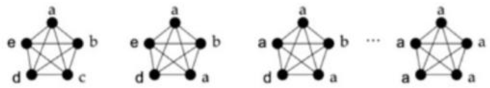
Classical graph partitioning does not detect the right communities but also the previous structures do not seem a “natural” way to find communities.

2. Community detection
Most recent approaches do not use structural formal definitions but extract communities by maximizing internal (intra-group) cohesion and minimizing external (inter-group) connections.
Different approaches exist:
- Divisive methods detect inter-group links and remove them from the network
- Agglomerative methods merge similar nodes/communities recursively

Heuristic algorithms that produce an optimal (not necessarily the best) solution in a reasonable time.

<u>Example 1. Label propagation</u>
Semi-supervised machine learning algorithm that assigns labels to previously unlabeled data points. At the start of the algorithm, a (generally small) subset of the data points have labels (or classifications). These labels are propagated to the unlabeled points throughout the course of the algorithm.
Within complex networks, this algorithm is based on the idea that neighbors usually belong to the same community.

Algorithm:
- Initialize labels on all nodes
- Randomize node order
- For each node replace its label with the one occurring with the highest frequency among neighbors (if there is no unique majority, select randomly among majority labels)
- If every node has the majority label of its neighbors, stop. Else repeat previous step



Advantages:
- no parameters, number or size of the communities are required
- simple and fast

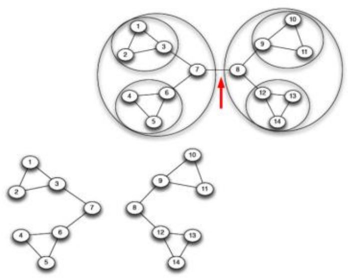
Disadvantages:
- it produces no unique solution, but an aggregate of many solutions; the outcome depends on the order in which nodes are visited, which is set to be random
- detects very large communities, for example in random graphs it will detect the giant component

<u>Example 2. The Girvan-Newman Algorithm</u>
Divisive algorithm: removes the links (bridges) connecting nodes that belong to different communities, eventually breaking a network into isolated communities.
Requires a notion of centrality to identify edges.
Uses edge betweenness, a centrality measure which counts the number of shortest paths between pairs of nodes that pass through an edge.
Removes edges with highest edge betweenness.

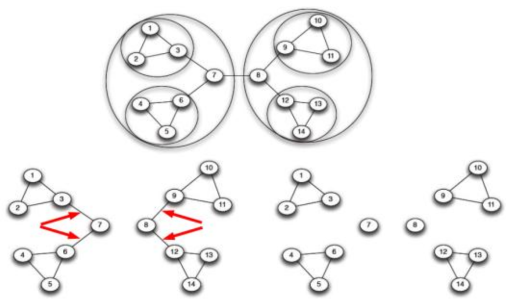
Algorithm
Until all links are removed:
1. Compute the edge betweenness for all links in the network connecting all pairs of nodes i and j
2. Remove the link with the largest centrality. In case of ties, choose one link randomly
3. Recalculate the betweenness of the remaining links
4. Repeat steps 2 and 3 until all links are removed

The algorithm recovers communities by removing edges and returns a dendogram

The Zachary Karate Club



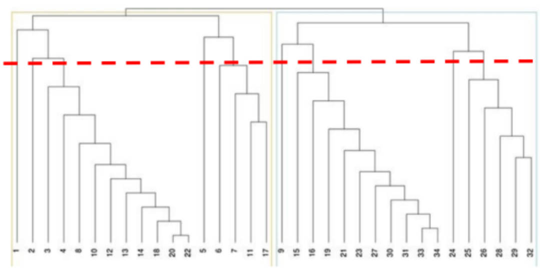
(a) Step 1



(a) Step 1 (b) Step 2

Example 2. The Girvan-Newman Algorithm

The algorithm is divisive, at each step the number of communities (clusters) increases.
The progress of the algorithm is represented using a dendrogram (tree).
The leaves of the dendogram represent the nodes of the network, in isolation.
As we move up the tree, the leaves join together first in pairs and then in larger groups; at the top of the tree, nodes are joined together to form the original network.
Nested components, where one can take a cut at any level of the tree.



The problem with this algorithm is that there is no real "stopping" criteria: which is the real structure (best cut) of the communities?

Moreover, in this algorithm, edge betweenness needs to be recalculated at each step since the removal of a link can impact the betweenness of another link:

- very expensive: all pairs shortest path is O(N3)
- may need to repeat up to N times
- does not scale to more than a few hundred nodes, even with the fastest algorithms
- real networks have millions when not billions of nodes!

How can we tell how "good" a partition is?

Modularity is a measure used to assess the quality of a community structure in a network.
It is defined as the fraction of edges within communities minus the expected fraction of such edges in a random network built assuming the configuration model (e.g., same degree distribution).

$$Q = \sum_{\forall C_x} \left(\frac{1}{2m} \sum_{(i,j) \in C_x} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \right)$$

Q > 0: there are more connections within communities than expected by chance in a random network and this suggests a good community structure.
Q = 0: the network structure is no different from a random network in terms of edge distribution within communities.
Q < 0: worse community structure than expected in a random network.

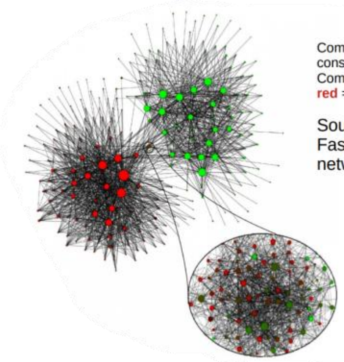
Modularity was introduced to rank the partitions computed with the Girvan-Newman algorithm.
It suggests where to cut the dendogram to find the best partition, e.g., we look for the cut corresponding to the highest value of Q.

Modularity vs Assortativity

	Modularity	Assortativity
Definition	Measures the strength of community structure	Measures the correlation between node degrees
Focus	Community structure or clustering in networks	Connectivity patterns based on node degrees
Interpretation	High modularity = strong community structure	Positive assortativity = high-degree nodes connect to high-degree nodes
Scale	Global network property (over all nodes)	Pairwise property (between pairs of connected nodes)

<u>Example 3. Louvain method</u>

The algorithm allows for a fast detection of communities in large networks.



Communities extracted from the call pattern of the consumers of the largest Belgian mobile phone Company. Main language spoken in the community red = French, green = Dutch

Source: Blondel et al.
Fast unfolding of communities in large networks, 2008 (Louvaine method)

Agglomerative algorithm: starts from a partition into singletons and aggregates nodes to optimize (locally) modularity.
Works in two phases, first computes communities and then aggregates them into super-nodes and restarts to find other communities.

Algorithm

0. Assign every node to its own community
1. Phase 1
    a. For every node, evaluate the modularity gain from removing the node from its community and placing it in the community of one of its neighbor
    b. Place each node in the community which maximizes the modularity gain
    c. Repeat until no more improvement is possible (local max of modularity)
2. Phase 2
    a. Create "super nodes" from the communities
    b. Put weights on the links between super nodes and within super nodes (self loops)
3. Repeat Phase 1 and Phase 2 until no more changes are possible (max modularity)

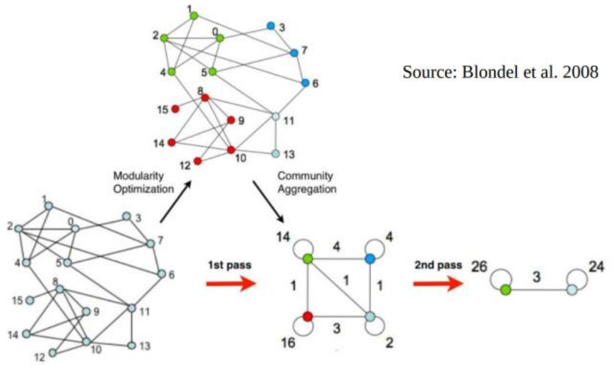


Figure 1. Visualization of the steps of our algorithm. Each pass is made of two phases: one where modularity is optimized by allowing only local changes of communities; one where the found communities are aggregated in order to build a new network of communities. The passes are repeated iteratively until no increase of modularity is possible.

Take away points

Communities play a key role in the structure and function of networks but they are not a well defined object.
It is possible to look for specific structures (k-cliques, k-cores, etc.) but most interesting is to discover the "natural" community boundaries.
The number of possible partitions of a network into communities is huge, even for a small graph, and it is impossible to search them all.

Different algorithms have been proposed (divisive vs agglomerative) and all have pros and cons.
The Louvain algorithm is agglomerative and efficient; it optimizes network modularity but the result depends on the starting node; a hierarchy of communities can be computed.
The Louvain algorithm is popular but has some problems discussed in the paper "From Louvain to Leiden: guaranteeing well-connected communities", which is the topic of the next lesson.

Louvain resolution parameter

The Louvain algorithm uses modularity to evaluate the quality or the partitions.

$$Q = \sum_{\forall C_x} \left(\frac{1}{2m} \sum_{(i,j)\in C_x} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \right)$$

Note: the formula for the modularity varies depending on the authors but the idea is always the same.
Modularity has also a resolution parameter γ:

- **γ < 1** favors larger communities
- **γ = 1** is the default behavior
- **γ > 1** favors smaller, more fragmented communities

Finding the right value for γ is not easy at all!

$$Q = \sum_{\forall C_x} \left(\frac{1}{2m} \sum_{(i,j)\in C_x} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \right)$$

Badly connected communities

In 2019 it was observed that the Louvain algorithm may identify communities that are internally disconnected!

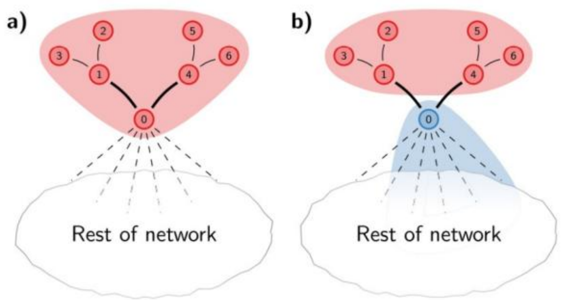


Figure 2. Disconnected community. Consider the partition shown in (**a**). When node 0 is moved to a different community, the red community becomes internally disconnected, as shown in (**b**). However, nodes 1–6 are still locally optimally assigned, and therefore these nodes will stay in the red community.

Leiden algorithm
Agglomerative algorithm: starts from a partition with singletons and aggregates nodes to optimize modularity (but it works also with other quality functions as well).
Guarantees that the communities are well connected and it is faster than Louvain algorithm.
Works in three phases, repeated until no further improvement is possible.

Algorithm

0. Assign every node to its own community
1. Phase 1: Local moving
    a. For each node, move it to the neighboring community that gives the highest increase in quality (modularity in our case)
    b. This step is similar to Louvain
2. Phase 2: Refinement (community splitting)
    a. For each community found in the local moving phase, ensure that the community is internally well-connected (i.e., there are no loosely attached parts)
    b. This is the major improvement over Louvain, it prevents badly connected communities from being accepted thanks to the computation of refined partitions
3. Phase 3: Aggregation
    a. Communities become "super nodes"
    b. This step is similar to Louvain, it considers the partition obtained in Phase 1, with the refinement of Phase 2
4. Repeat Phase 1, Phase 2, and Phase 3 until no more improvement in the quality function is possible (max modularity)

As said, different quality metrics can be used, for example CPM (Constant Potts Model) is a quality function used to detect communities derived from statistical physics, which allows more control over the granularity of detected communities. If interested check online ;)

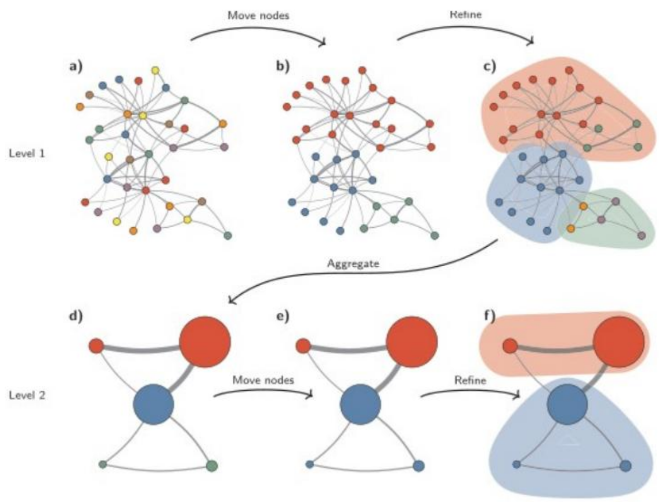


Figure 3. Leiden algorithm. The Leiden algorithm starts from a singleton partition (**a**). The algorithm moves individual nodes from one community to another to find a partition (**b**), which is then refined (**c**). An aggregate network (**d**) is created based on the refined partition, using the non-refined partition to create an initial partition for the aggregate network. For example, the red community in (**b**) is refined into two subcommunities in (**c**), which after aggregation become two separate nodes in (**d**), both belonging to the same community. The algorithm then moves individual nodes in the aggregate network (**e**). In this case, refinement does not change the partition (**f**). These steps are repeated until no further improvements can be made.

Quality of a partition

The quality of a community partition in a graph is crucial to ensure the algorithm actually captures a meaningful structure, and several metrics exist beyond modularity

- Conductance measures how well-separated a community is from the rest of the graph (no built-in NetworkX function)
- Coverage of a partition is the ratio of the number of intra-community edges to the total number of edges in the graph

$$C = \frac{\sum_{i=1}^{k} e(C_i)}{L}$$

$e(C_i)$ is the number of edges that have both endpoints within community C_i
L is the total number of edges in the graph
k is the number of communities

The coverage values range in [0,1]

- C = 0: none of the graph's edges are contained within any community
- C = 1: all edges are contained within communities, e.g., there are no edges connecting different communities

Performance evaluates the fraction of correctly classified node pairs (i.e., pairs that are either correctly connected within communities or correctly disconnected between communities) relative to the total number of possible node pairs.

The performance values range in [0,1]

- P = 0: worst case where none of the node pairs are correctly classified with respect to the partition
- P = 1: ideal case where every pair is classified correctly; every intracommunity pair is connected by an edge, and every inter-community pair is not connected

NetworkX has a built function to compute both coverage and performance, see partition_quality(G, partition).

If a ground truth exist, e.g., the real way the nodes of a graph are grouped together into communities is known, we can compute the Normalized Mutual Information (NMI).
NMI measures the similarity between the detected partition and the ground truth.
The values of NMI range in [0, 1] and higher is better.
adjusted_rand_score(...) is a Python function from scikitlearn used to compute NMI; it compares the output of a community detection (or clustering) algorithm against a ground truth (if available), ignoring permutations within the communities.

Another metric is the Adjusted Rand Index (ARI), that measures the similarity between the partition and the ground truth, also considering that random partitions might share some similarity by chance.
The values of ARI range in [-1,1].

- ARI = 1.0 is a perfect match
- ARI ≈ 0.0 is a random assignment (expected value)
- ARI < 0.0 is worse than random (rare, but possible)

normalized_mutual_info_score(...) is another Python function from scikit-learn particularly useful to compare two community assignments, like computed vs. ground truth.

Take away points

Detecting communities is a challenging task.
The Leiden algorithm was introduced to address some limitations of the Louvain algorithm.
Multiple algorithms exist, and they do not necessarily produce the same results.
When a ground truth is available, it is possible to evaluate the quality of an algorithm's results, but this is not always the case.
Community detection is the topic of Assignment 1, see AulaWeb for the details.

Tips for Assignment 1

What if your dataset is too large?

Various techniques can be used to reduce the size of a large graph.
The resulting reduced graph will differ in structure and properties from the original, so careful consideration is needed when interpreting the results.

Reduction techniques

1. K-core can be used to delete nodes with degree less than k from the graph.
This choice significantly changes the structure and properties:

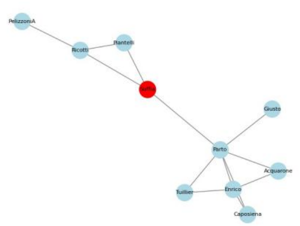
- the graph may lose peripheral structure since only highdegree nodes are kept
- it has a different degree distribution
- the diameter decreases
- dense regions are maintained

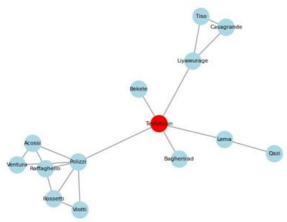
2. Sampling techniques

- Node sampling: randomly select a subset of nodes and include all edges among them
- Edge sampling: randomly select a subset of edges and include the nodes connected to them

Question: Which technique has more chances to maintain hubs?

- Snowball sampling: start from a seed node and iteratively include neighbors up to a certain depth
- This is particularly useful to explore local structures, like ego networks, e.g., representations of the social relationships of an individual (ego) with its social peers (alters)





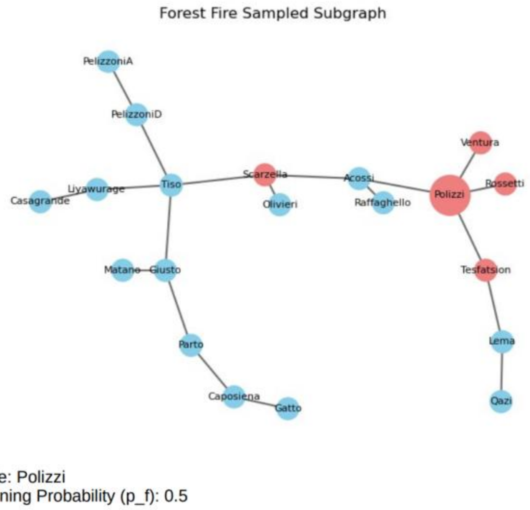
- Forest Fire sampling: probabilistic version of snowball sampling that simulates how a "fire" spreads through the graph

Algorithm

- Start at one random seed node (like lighting a match)
- From the seed node, "burn" a random subset of its neighbors, with some probability
- Each burned neighbor then burns their own neighbors, with some probability, and so on, recursively

Parameters

- Forward Burning Probability (p_f): probability that a neighbor gets selected and burned
- Backward Burning Probability (p_b): if the graph is directed, this allows for traversal against the edge direction



3. Community detection and subgraph extraction

Use an algorithm to detect communities.

Select a community (or a few) and work on the induced sub-graph.

Especially effective if your graph is modular or clustered in nature.

Property	Original Graph	Reduced Graph
Size (Nodes & Edges)	Full node and edge set	Subset — often significantly smaller
Connectivity	May be fully connected	May have many disconnected components
Community Structure	Clear, well-separated communities	Weaker, merged, or fragmented communities
Node Centrality	High-centrality nodes accurately identified	Some central nodes may be missing or misranked
Degree Distribution	Preserved	May be flattened or biased
Clustering Coefficient	Natural level of local clustering	Often lower, especially with edge sampling
Shortest Paths	True shortest paths reflect real distances	Some paths broken, longer or missing entirely
Assortativity	Realistic degree correlations	May change significantly
Diffusion Potential	High, especially in dense or connected graphs	Lower, limited by missing links or components

Take away points

For large dataset there might be issues due to the complexity of many metrics but it is possible to reduce the graph using different techniques; the price to be paid is that many characteristics of the original graph will be lost.

For the first assignment, where the community structure is the main interest, you can use community-based sub-graphs.

If the interest is on local influence or ego-centric analysis, snowball sampling works best.

For studying global dynamics (e.g., diffusion over a network), forest fire reduction is recommended.

K-core is the easiest technique, with its pros and cons.

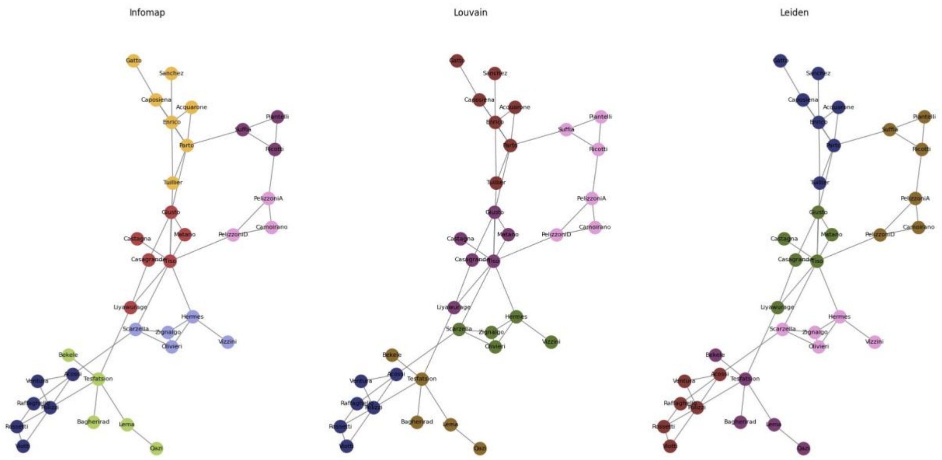
Other algorithms

The literature on community detection is broad, several algorithms exists.

Infomap

Infomap finds communities (or clusters) in a network thinking about information flow, that is how things like messages, people, or ideas move around the network.
It uses the idea of a random walker (similar to the random surfer already discussed for PageRank) that jumps from node to node.
If the network has "good" communities, the random walker will stay inside those communities for a while before jumping to other communities.
Infomap tries to compress the description of this walk as much as possible.
The description of the random walk is built internally by the algorithm that:

- simulates a random walk
- tries different ways of grouping nodes into communities
- for each grouping, calculates the codelength, that is how many bits are needed to say
    - which community the walker is in (inter-community moves, jumps)
    - which nodes inside that community the walker is moving (intracommunity moves)
- returns the grouping that gives the shortest total codelength



Overlapping communities

In real-world networks a node can belong to more than one community.
Finding overlapping community in a graph is more difficult, but possible.
Several algorithms exist, one example is SLPA (SpeakerListener Label Propagation Algorithm), an extension of traditional Label Propagation, designed to model how information spreads in social networks.

Speaker-Listener Label Propagation Algorithm

Works in two rounds, ruled by the parameters:

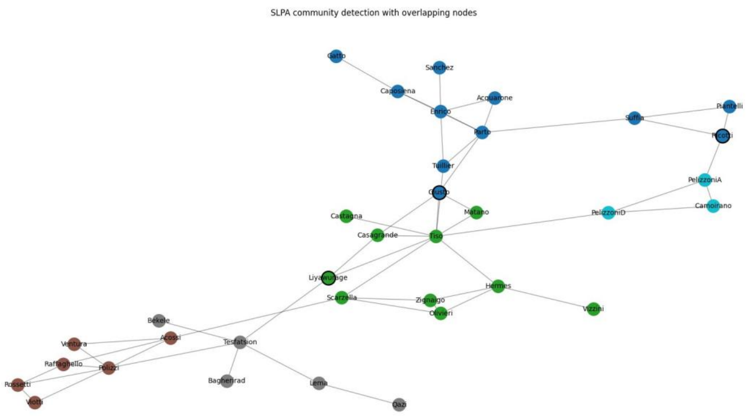
- t, the number of iterations (usually 20–100)
- r, threshold for the frequency of the labels known by the nodes (0.1–0.5)

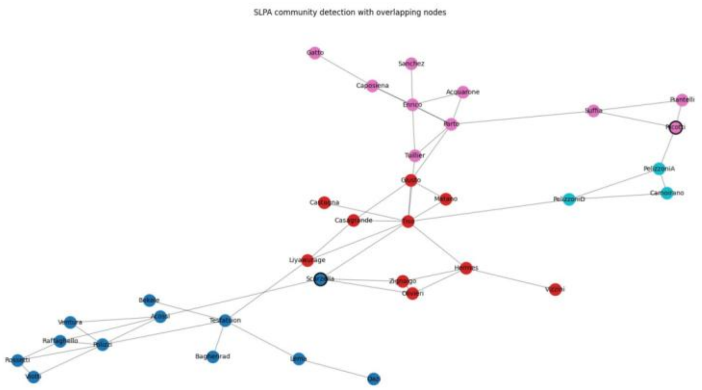
Algorithm
Phase 0: each node gets a unique label (like its own ID)
Phase 1: For t times (iterations)
For each node u:

- u becomes a listener
- for each node v neighbor of u, v becomes a speaker, and sends to u one of its own labels, chosen randomly from its memory (based on how often it has seen them)
- u picks one label (choosing with majority vote or at random) and stores the label in its memory

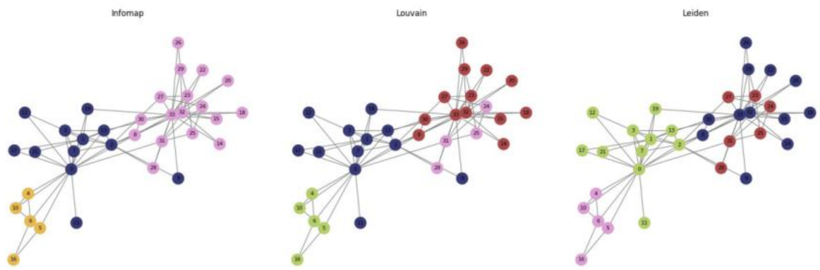
Phase 2:
Post-processing For each node u:
- if a label appears with a frequency > r, the node is considered a member of that label's community, otherwise ignore the label

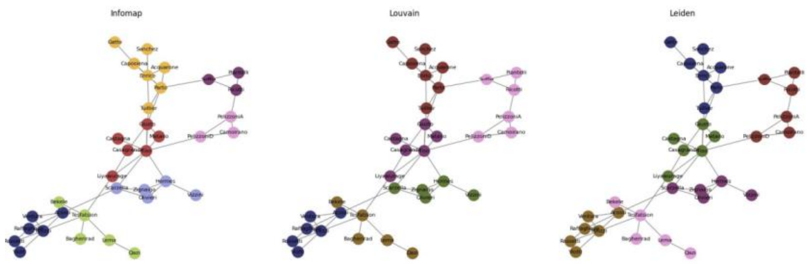
Lower r → more overlapping
Higher r → tighter communities





Community detection is not deterministic
While Zachary's Karate Club graph is often used as a benchmark, also this graph does not have a single "true" community structure.
The ground truth is based on the split in two communities ("Mr. Hi" against "Officer") but different algorithms might find different results :(





Infomap vs Louvain — ARI: 0.936, NMI: 0.968
Infomap vs Leiden — ARI: 0.936, NMI: 0.968
Louvain vs Leiden — ARI: 1.000, NMI: 1.000

Take away points

Community detection is challenging: many algorithms (like Louvain, Leiden, SLPA) are parameter-sensitive reflecting the complexity and ambiguity of real-world networks.
No ground truth: most real-world networks do not come with “correct” community labels and therefore their evaluation is tricky.
Algorithms make different assumptions:

- Louvain: modularity (but suffers from resolution limit)
- Leiden: better partition quality
- Infomap: information-theoretic flow
- SLPA: communication-based influence

To compare different partitions there are metrics like Normalized Mutual Information (NMI) or Adjusted Rand Index (ARI) to assess similarity to a known ground truth or between the different results of the algorithms.

NetworkX vs igraph

Fast and open source library to manipulate and analyze graphs, it can be used to:

- Create, manipulate, and analyze networks
- Convert graphs from/to networkx, graph-tool and many file formats
- Plot networks using Cairo, matplotlib, and plotly

For community detection several options exist:

- nx.community.girvan_newman(G), the package python-louvain, the package infomap, all available with NetworkX
- igraph that implements many algorithms (Louvain, Leiden, Infomap) and has functions to convert from/to NetworkX
- to convert from/to the different libraries, utilities are available online (ask to GPT or other LLMs)

Network diffusion: Epidemic spreading

Network diffusion
Having passed through a global pandemic, or having seen meme “go viral”, we are all familiar with the spread of “things” through network nodes and links.
What happens after a network forms?
Links can be considered like pipes that carry “things” which depend on the domain under study.
Sometimes, network flows should be avoided or contained, for example in the case of biological contagions, social contagion with misinformation, computer viruses.

Network properties are important in the diffusion process.

Epidemic spreading

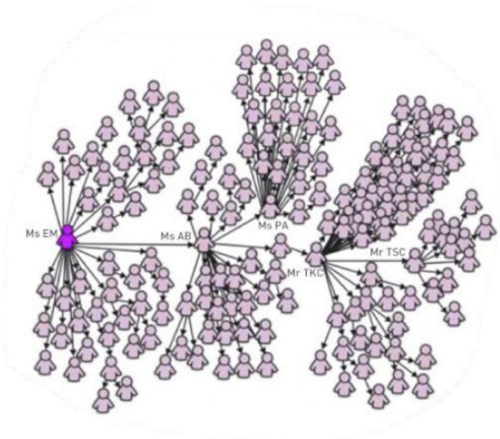
High population density

High mobility

Nowadays, it takes just a few hours to fly across continents.
Perfect conditions for epidemic spreading.

Biological viruses
Pathogens spread on their respective contact network, examples include:

- airborne diseases like influenza, SARS, tuberculosis, or COVID transmitted when two individuals breathe the air in the same room
- contagious diseases and parasites transmitted when people touch each other; for example the Ebola virus is transmitted via contact with a patient's body fluid (fatality rates from 25% to 90%)
- and many others...



Digital viruses
A computer virus is a self-reproducing program that can transmit a copy of itself from computer to computer.
Its spreading pattern has many similarities to the spread of pathogens.
But digital viruses also have many unique features, determined by the technology behind the specific malicious program.

Spreading processes

Various phenomena can be explained as spreading processes on networks.

Phenomena	Agent	Network
Venereal Disease	Pathogens	Sexual Network
Rumor Spreading	Information, Memes	Communication Network
Diffusion of Innovations	Ideas, Knowledge	Communication Network
Computer Viruses	Malwares, Digital viruses	Internet
Mobile Phone Virus	Mobile Viruses	Social Network/Proximity Network
Bedbugs	Parasitic Insects	Hotel - Traveler Network
Malaria	Plasmodium	Mosquito - Human network

Epidemic models

Classical epidemic models divide the population into different compartments, corresponding to different stages of the disease.

The two key compartments are:

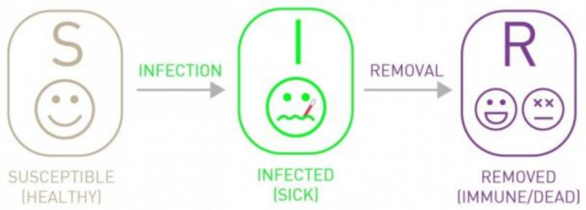
- **Susceptible (S)** Healthy individuals who have not yet contracted the pathogen
- **Infected (I)** Contagious individuals who have contracted the pathogen and hence can infect others

SIS epidemic model

By adding a third compartment, e.g.,

- **Recovered (R)** or Removed Individuals who have been infected before, but have recovered from the disease (or died) and are not infectious anymore

we have the SIR model



SIR model assumptions:

- Fixed population: no births/deaths or migration
- Once recovered, individuals gain immunity or die (they move to R)
- Homogeneous mixing: each individual is equally likely to contact any other

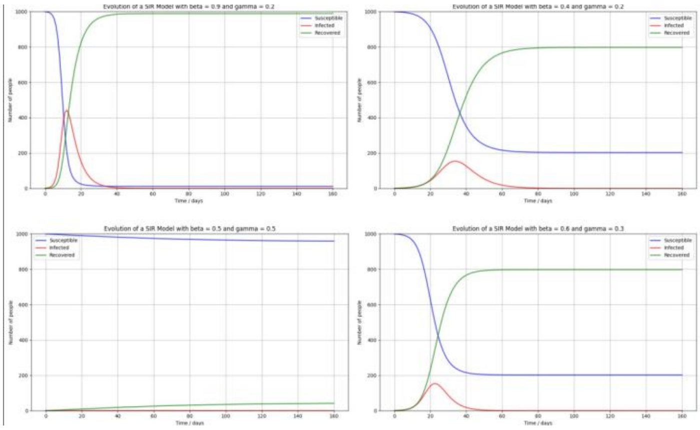
Given

- S(t): number of Susceptible individuals (can catch the disease) at time t
- I(t): number of Infected individuals (infected and contagious) at time t
- R(t): number of Recovered individuals (immune or removed) at time t
- N= S(t) + I(t) + R(t): total population (constant)
- β: transmission rate
- γ: recovery rate

The SIR model is ruled by the following equations

$$\frac{dS}{dt} = -\beta\frac{SI}{N} \qquad \frac{dS}{dt} = -\beta\frac{SI}{N}$$
$$\frac{dI}{dt} = \beta\frac{SI}{N} - \gamma I \qquad \boxed{\frac{dI}{dt} = \beta\frac{SI}{N} - \gamma I}$$
$$\frac{dR}{dt} = \gamma I \qquad \frac{dR}{dt} = \gamma I$$

The second equation determines whether an epidemic grows or shrinks (we will see this when considering also the underlying network).



Epidemic models and networks

Homogeneous mixing approximation might be reasonable for a small population, for example for small villages where everybody knows everyone else (or for our class).

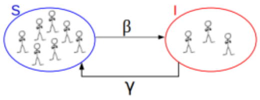
In real situations, the individuals live within a contact network which mediates the spreading of the disease.

Important note: state transitions in epidemic models can be labeled by

- **Rates:** representing the number of individuals that can become infected or recover over a specified period of time
- **Probabilities:** representing the likelihood of a specific individual to become infected or to recover in a specific event

In the next slides I will use probabilities.

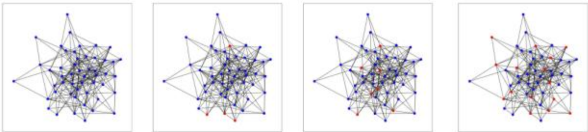
Algorithm for the SIS epidemic model



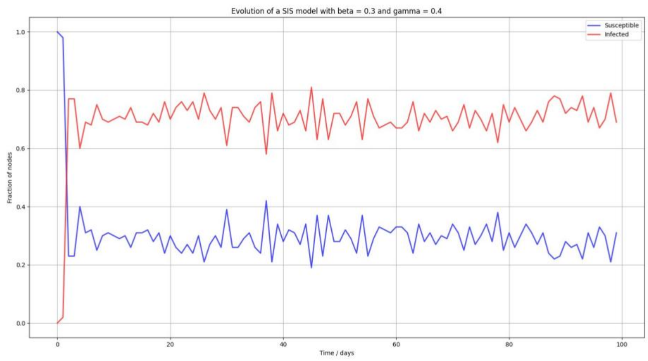
At each iteration visit all nodes

For each node i

- If i is in S, loop over its neighbors
    - For each infected neighbor, i can be infected with probability β
- If i is in I, i becomes susceptible with probability γ



The dynamics produces a number of transitions from S to I and from I to S that, under certain conditions, can be sustained indefinitely.

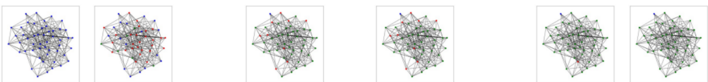


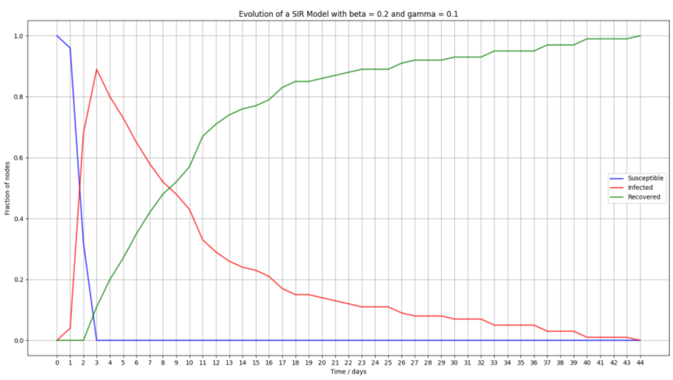
At each iteration visit all nodes

For each node i

- If i is in S, loop over its neighbors
    - For each infected neighbor, i can be infected with probability β
- If i is in I, i moves to the recovered state R with probability γ

Repeat until the number of infected nodes reaches 0





SIR epidemic model

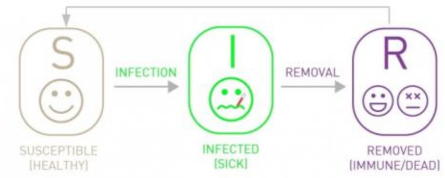
In the SIR model

- the number of people infected (red line) rises exponentially when the fraction of susceptible people is very large
- after the initial growth, the infection will reach a peak, when the number of individuals will start recovering (green line)
- the fraction of people infected always goes down to zero when less individuals can become infected
- the sum of the points on a vertical line on the three curves is always equal to 1 (100%) or N (number of individuals in the system)

Synchronization

Researchers also observed the tendency of epidemics for certain diseases to synchronize across a population, sometimes producing strong oscillations in the number of affected individuals over time. The first step in producing a model with oscillations is to allow the disease to confer temporary but not permanent immunity on infected individuals.
This is obtained by combining SIR and SIS models.

SIRS epidemic model



New feature of the model: after t_I steps, node i is no longer infectious. It then enters the R state for a fixed number of steps t_R. During this time, it cannot be infected with the disease, nor does it transmit the disease to other nodes. After t_R steps, i returns to state S.

SIRS epidemic model + Small world

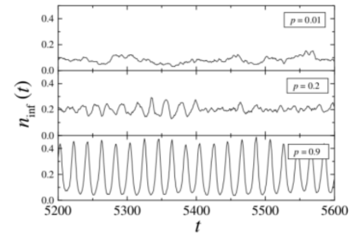


FIG. 1. Fraction of infected elements as a function of time. Three time series are shown, corresponding to different values of the disorder parameter p, as shown in the legends. Other parameters are: $N = 10^4$, $K = 3$, $\tau_I = 4$, $\tau_R = 9$, $N_{\text{inf}}(0) = 0.1$.

Basic reproduction number R_0

R_0 is the basic reproduction number, which represents the average number of people that one infected person will pass the virus to, in a fully susceptible population.

For the original Wuhan strain, estimates converged around 2.5–3.0. For Omicron, with much faster spread, estimates shot up to 8–12 or even more, depending on the setting. Source: ChatGPT :)

Disease	Transmission	R_0
Measles	Airborne	12-18
Pertussis	Airborne droplet	12-17
Diptheria	Saliva	6-7
Smallpox	Social contact	5-7
Polio	Fecal-oral route	5-7
Rubella	Airborne droplet	5-7
Mumps	Airborne droplet	4-7
HIV/AIDS	Sexual contact	2-5
SARS	Airborne droplet	2-5
Influenza (1918 strain)	Airborne droplet	2-3

After one iteration

- the average number of infections caused by a single individual is β‹k›

During the spreading process

- with I infected individuals we have I_{sec} = β‹k›I
- every sick individual will recover with probability γ
- with I infected individuals we have I_{rec} = γI

For the epidemic to spread we must have $I_{sec} > I_{rec}$

– $\beta\langle k\rangle I > \gamma I$

– $\frac{\beta\langle k\rangle}{\gamma} > 1$

– $\beta\langle k\rangle I > \gamma I$

– $\boxed{\frac{\beta\langle k\rangle}{\gamma}} > 1 \longrightarrow R_0$

If $R_0 < 1$, then each infected individual is passing the infection to less than one other individual and, with probability 1, the disease dies out after a finite number of iterations.
If $R_0 > 1$, then with probability greater than 0 the disease persists by infecting more than one person in each iteration.

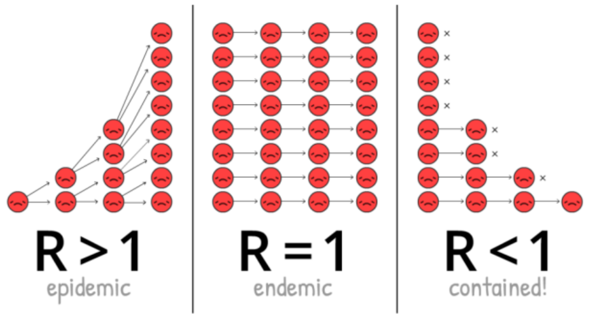
Note: R_0 specifically applies to a population of people who were previously free of infection and have not been vaccinated yet; it is usually estimated retrospectively from serial epidemiological data.

Effective reproduction number R

Often you will also find the letter R (or R_t) for the effective reproduction number during the outbreak. R changes over time as the epidemic progresses due to:

- fewer susceptible people (some get infected and recover)
- interventions like masks, lockdowns, or vaccination
- behavior changes (e.g., people avoid crowds)
- seasonality, new variants, etc.

These all reduce (or sometimes increase) the effective transmission rate.



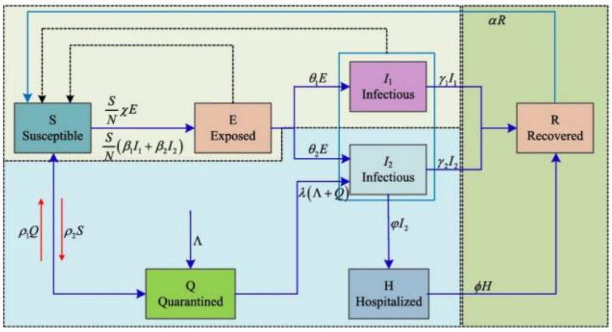
Things are indeed more complex... the population is not homogeneous, the definition must account for the fact that a typical infected individual may not be an average individual.
Around the critical value R = 1, it can be worth investing large amounts of effort even to produce small shifts in the effective reproduction number.

How?

It is customary to suggest two basic kinds of public-health measures
- quarantining people, which reduces ‹k›
- encouraging behavioral measures such as better sanitary practices to reduce the spread of germs, which reduces β

Threshold effect?
Real contact networks are not homogeneous.
R < 1 is a necessary but not sufficient condition for stopping a disease.
Which is the role of the hubs?

SEIR modeling of the COVID-19



State-based targeted vaccination
“Vaccination has become one of the most prominent measures for preventing the spread of infectious diseases in modern times. However, mass vaccination of the population may not always be possible due to high costs, severe side effects, or shortage. Therefore, identifying individuals with a high potential of spreading the disease and targeted vaccination of these individuals is of high importance.”

Various strategies for identifying the superspreaders have been proposed:
- vaccinating the highest degree nodes is perhaps the most studied strategy
- vaccinating nodes with the highest betweenness centrality score is acknowledged as the most effective strategy

Some studies suggest a vaccination approach that requires only local information about the node’s neighborhood by relying on the friendship paradox: picking a random neighbor of a random node is more likely to result in a central node than just picking a random node...
The paper of Lev and Shmueli proposes a novel targeted vaccination strategy based on the Infectious Betweenness (IB) Centrality, that considers both the static network topology and the dynamic states of the network nodes over time using the SIR model.
IB aims at identifying nodes that serve as bridges between network components. However, in contrast to betweenness centrality, this strategy focuses on finding bridges between infected nodes and susceptible nodes.

Network topology and diffusion

Network topology is important for diffusion:

- the path length between any pair of nodes and the redundancy in short paths govern the probability of a long diffusion
- connected components forms the maximum diffusion potential in a network because for each pair of nodes at least one path exists
- also bridges among different communities facilitate the spread among otherwise distant populations (see Watts and Strogatz)
- one efficient diffusion structure is a network tree; efficiency is reduced when potential paths loop back on themselves in triangles: high local clustering can slow down diffusion
- disassortative mixing with many star-like local networks can enable efficient diffusion

In static networks nodes and edges are are present for transmission during all the epidemic spreading. This is not always the case since infection can transmit only forward in time and we can have more precise analysis if we know when edges were added to the network.

Take away points

SIS, SIR and SIRS models assume homogeneous mixing, e.g., an infected individual can infect other individuals, ignoring their role in a network.
At the beginning, when the number of infected individuals is small, the disease spreads freely and the number of infected individuals increases exponentially.
The outcomes are different for large times (final regime):

- SIS and SIRS model: either reach an endemic state, in which a finite fraction of individuals are always infected, or the infection dies out
- SIR model: everyone recovers (or dies) at the end

The reproduction number predicts the long-term fate of an epidemic: for $R > 1$ the pathogen persists in the population, while for $R < 1$ it dies out naturally.
Things become more complex when we consider network topology (and mobility).

Dynamics: Social contagion

Following the crowd?

The aggregate behavior of many people with limited information can sometime produce very accurate results.
If many people are guessing independently, then the average of their guesses is often a surprisingly good estimate of whatever they are guessing about (the number of jelly beans in a jar, or the weight of a bull at a fair).
The key to this argument of course is that the individuals each have private information (their signals), and they guess independently, without knowing what the others have guessed.
If instead they guess sequentially, and can observe the earlier guesses of others, then we can observe a cascade setting and there would be no reason to expect the average guess to be good at all.
There are many settings in which it may be rational for an individual to imitate the choices of others even if the individual's own information suggests an alternative choice.
Suppose that you are choosing a restaurant in an unfamiliar town, and based on your own search about restaurants you intend to go to restaurant A. However, when you arrive you see that no one is eating in restaurant A while restaurant B next door is nearly full.

Information cascade has the potential to occur when people make decisions sequentially, with later people watching the actions of earlier people, and from these actions inferring something about what the earlier people know.
In the restaurant example, when the first diners to arrive chose restaurant B, they conveyed information to later diners about what they knew.
A cascade then develops when people abandon their own information in favor of inferences based on earlier people's actions.

Social contagion
The spreading processes of tweets, URL, video, meme, ideas, innovations,… are also called social contagion because they resemble a disease that is transmitted via contact between individuals.
However, in this context individuals are first exposed to the information (the innovation) and then they can decide to take action (or not).
An example of diffusion of innovation was the COVID-19 vaccination: observing others taking the vaccine encouraged other people to do the same

Propagation depends on many factors

- Resistance to changes
- Number of exposures
- Type of information, since different "things" spread differently
- Information can change for technical reasons and also intentionally (misinformation)

Of course, networks play a central role
In any model of social contagion, a certain number of individuals (adopters) are initially activated, e.g., they adopt the new idea, behavior, etc
Then each inactive node is activated (or not) according to some rule that depends on the number of active neighbors and on other parameters.
This is often called complex contagion, as opposed to the simple contagion in epidemic diffusion, when a single exposure to an infected person can transmit the illness.
The biggest difference between social and biological contagion lies in the process by which one person "infects" another.

With social contagion, people are making decisions to adopt a new idea or innovation

- Most often, multiple exposures are required before an individual adopts a change in state
- In network jargon, how many nodes need to adopt a new idea before other neighbors do the same?

With biological contagion, as we have seen, there is a lack of decision-making in the transmission from one person to another, and the process is modeled as random.
A single exposure is enough and the spreading is influenced by the contagiousness of the disease and the connectivity of the network (highly connected networks will see faster spread).

It is very hard to measure

- Word-of-mouth influence?
- Who influences whom?
- Why some individuals are super influencers?

Independent cascade model (Simple contagion)
Threshold model (Complex contagion)

Simple contagion

Simple contagion describes the spread of “things” that require only one contact or exposure for transmission.

Example: Learning a viral meme or news story from a single social media post.

Spreads quickly through weak ties.

Highly efficient in networks with many random links (like small-world or power-law networks).

Simple contagion is one-to-one.

As soon as a node becomes active it has a chance (only one) to “convince” each of its inactive neighbors with some influence probability p, which can be different for each pair of nodes (similar to epidemic spreading).

Cascades can occur in the network.

Complex contagion

Complex contagion refers to behaviors or ideas that require multiple sources of exposure before a person adopts them.

Requires social reinforcement.

Examples:

- Joining a protest or political movement
- Adopting a new technology or lifestyle change
- Engaging in risky or controversial behaviors

Understanding social contagion helps:

- Marketers design better campaigns
- Health organizations promote effective public health messages
- Activists build stronger movements
- Platforms encourage or discourage viral behavior

Threshold model

An individual will take action only if a certain number or fraction of their neighbors have done so (e.g., a threshold is needed).

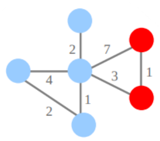
Individuals can be in two states:

- Susceptible
- Active (e.g., has adopted the innovation)

Each individual has a personal threshold.

The influence on a node i is expressed by

$$\boxed{I(i) = \sum_{j:active} w_{ji}}$$



The condition for the activation of node i considers its specific threshold θ_i, assigned before the process starts.

$$\boxed{I(i) \geq \theta_i}$$

The threshold indicates the tendency of a node to be influenced by its neighbors.
The threshold θ_i can be an integer number (linear threshold) or a fraction (fractional threshold).

Algorithm
- Select a network
- Assign a threshold to each node (it can be the same value)
- Activate n nodes (select specific nodes or at random)
- At each step
  - All active nodes remain active
  - Each inactive node is activated if the number (fraction) of active neighbors is at or above its threshold
  - Repeat until no further node can be activated

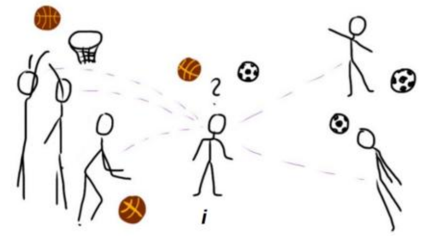
The order in which nodes are considered should not affect the result
- **Asynchronous** implementation nodes are evaluated in a different random sequence at each iteration
- **Synchronous** implementation the new activation state of each node is determined using the activation states of its neighbors from the previous iteration; all nodes are updated at the end of the iteration

We have certain neighbors and the benefit of adopting a new behavior increases as more and more of our neighbors adopt it:
- Each node in the network has a choice between two possible behaviors, labeled A and B
- If nodes i and j are connected by an edge, then there is an incentive for them to have their behaviors match

We can capture this with a game in which i and j are the players and A and B are the possible strategies.
The payoffs are defined as follows:
- if both i and j choose A, they get a payoff a > 0
- if both i and j choose B, they get a payoff b > 0
- if one chooses A while the other chooses B, their payoff is 0

	A	B
A	a, a	$0, 0$
B	$0, 0$	b, b

Node i has d neighbors (5), and a fraction p (3/5) adopt A (basket) while the remaining (1-p) (2/5) adopt B (soccer).



What should i do in order to maximize the payoff?

Node i has d neighbors:

- fraction p plays basket (adopt A)
- fraction (1-p) plays soccer (adopt B)
- if i chooses A, it gets a payoff p*d*a
- if i chooses B, it gets a payoff (1-p)*d*b
- node i should choose A if
    - $p \cdot d \cdot a \geq (1-p) \cdot d \cdot b$ or
    - $p \geq b / (a + b)$

Let $\theta = b / (a + b)$

If at least $\theta = b / (a+b)$ neighbors follow A, then you should too! In the example, the number of players is not enough to decide but we should also know the values of a and b

- If a = b = 1, then $\theta = ½$
- P_{basket} = 3/5 > ½ hence choose basketball

Two equilibria

- Everyone adopts A
- Everyone adopts B

Suppose all nodes adopt B as a default behavior.

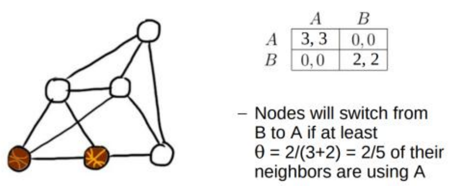
What if some nodes switch at random to A: will a cascade occur?

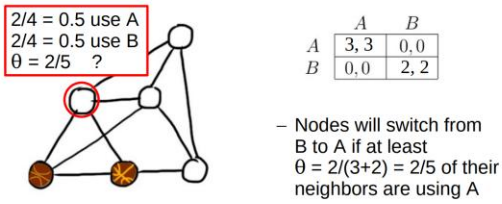
The answer depends on

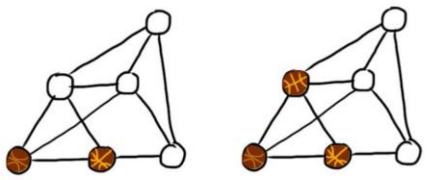
- The network structure
- The choice of the initial adopters
- The value of the threshold θ

<u>Cascade example</u>

Suppose 2 nodes start playing basket (A) due to external factors (e.g., they get a free pair of shoes).

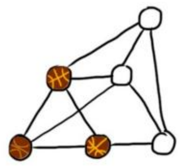






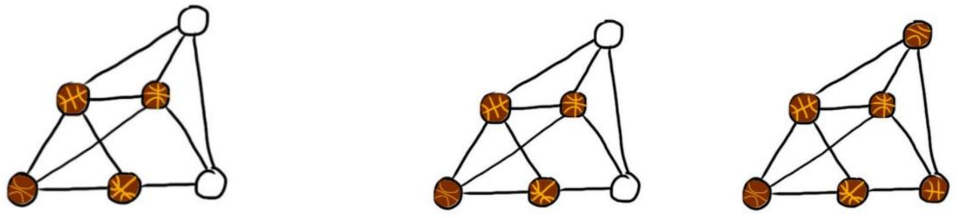
θ = 2/(3+2) = 2/5

Which node(s) will switch to playing basket next?



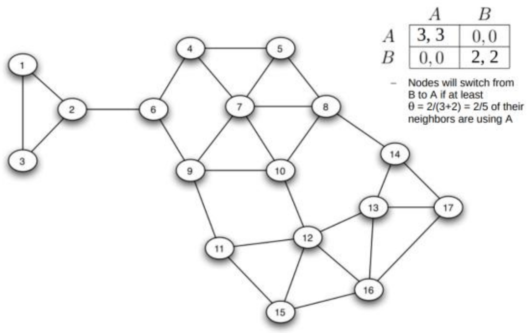
θ = 2/(3+2) = 2/5

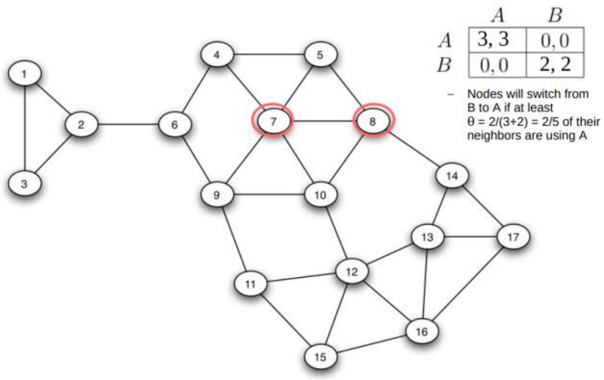
The complete cascade

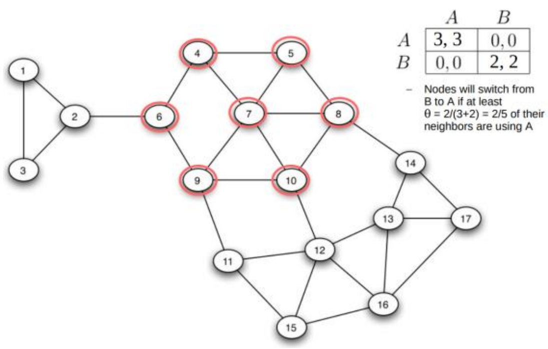


θ = 2/(3+2) = 2/5

<u>Larger example</u>







A was only able to spread to a set of nodes where there was sufficiently dense internal connectivity. As a result, we get coexistence between A and B, with boundaries in the network where the two meet. What is the role of communities in social contagion?

Role of communities
Create isolated groups impervious to outside ideas.
Allow different opinions in different parts of the network.
For example, certain industries heavily use Apple computers despite the general prevalence of Windows: if most of the people around you use Apple software, it is in your interest to do so as well.
Some strategies can be adopted when A and B are competing.
Perhaps the most direct way, when possible, would be for the maker of A to increase the quality of its product slightly.
What if we change the payoff a in the coordination game from a = 3 to a = 4?

A would be able to break into the other parts of the network that are currently resisting it. This captures an interesting sense in which making an existing innovation slightly more attractive can greatly increase its reach.

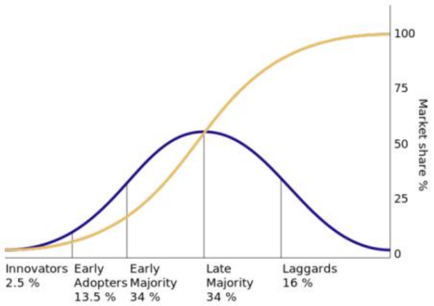
A different strategy for increasing the spread of A would be to convince a small number of key people in the part of the network using B to switch to A, choosing these people carefully so as to get the cascade can occur.

Diffusion of innovation

There is a crucial difference between:

- Learning about a new idea
- Deciding to adopt it

The diffusion of innovation has been deeply studied by sociologists and economists in the 20th century



Take away points

Common belief: information and innovation spread between individuals like a pathogen

- each exposure by an informed friend potentially results in an "infected" individual

Indeed, in social contagion, users can take decision and therefore the propagation often requires more exposures.

The threshold model assumes a binary choice (adopt or not) and does not capture the nuances of individual decision-making.

Real-world social influence is more complex and can involve multiple degrees of influence from different connections.

Despite these limitations, the threshold model provides a valuable starting point for understanding social contagion.

	Simple contagion	**Complex contagion**
Exposure	One	Multiple
Examples	Common cold, rumors	Learning a skill, taking a vaccine, using a new technology
Spreading	Contagiousnes, network structure	Social influence, network structure

Network robustness

A network is robust if the failure of some of its components does not affect its function.
Is Internet robust?
Is the Web robust?
Is the airport network robust?
Is the food web robust?

Errors and failures can corrupt all human designs:

- The failure of a component in your car's engine or a wiring error in your computer chip cause fatal errors

Many systems have, however, a remarkable ability to sustain their basic function even when some of their components fail (today we use the word resilience)

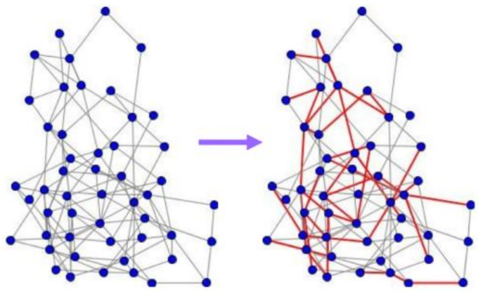
- Large organizations can function despite numerous absent employees
- Around 3% of the routers on the Internet are non-functional but the network keeps delivering packets

Nodes in a network can describe a variety of entities: people, routers, proteins, neurons, websites, airports.
In this high-level representation, we can assume that when nodes or links stop working we can delete them from the network, to see how the structure, and consequently the behavior of the network, change.
The standard robustness test for networks consists of checking how the connectedness is affected as more and more nodes or links are removed.

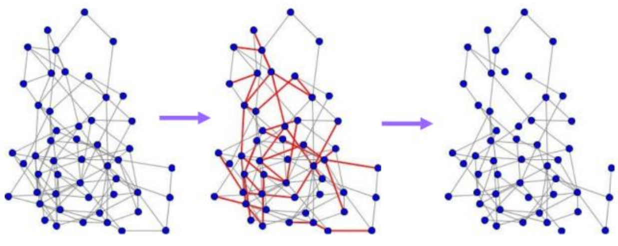
We can consider

- changes in the size of the giant component
- changes in the length of the diameter

Example: remove edges at random



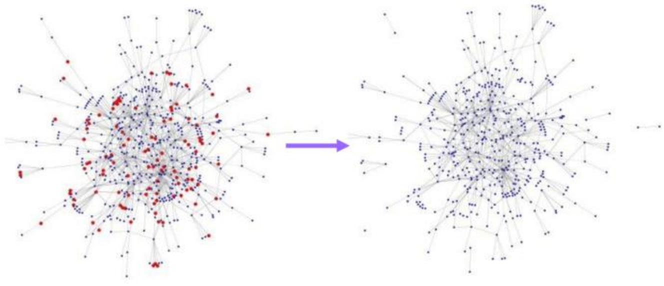
50 nodes (N), 116 edges (L), average degree 4.64*
By removing around 25% of the edges at random, how much would reduce the giant component?



The giant component is still there!
*Recall: average degree in undirected graph: 2L/N

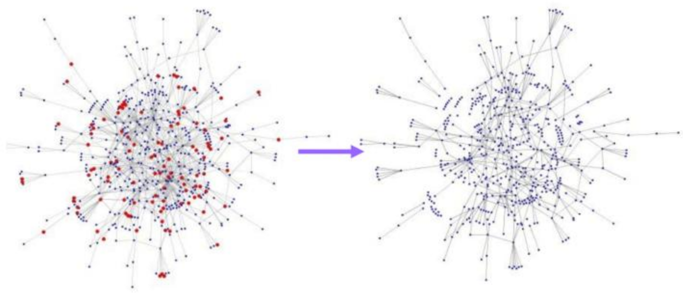
Which edge removal strategy causes most damage?
Select edges with highest betweenness!

20% of nodes removed at random



574 nodes in giant component **427** nodes in giant component

22 most connected nodes removed (2.8% of the nodes)

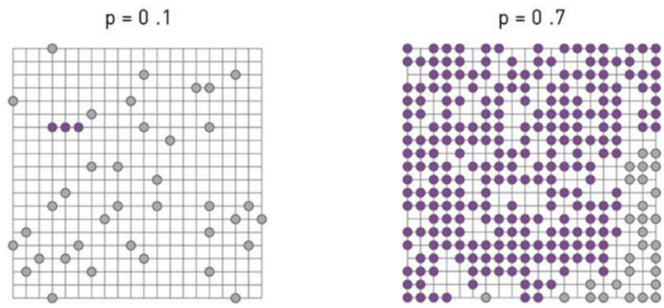


574 nodes in giant component **301** nodes in giant component

Which node removal strategy causes most damage?
What happens in the case of scale-free networks?

Percolation Theory*

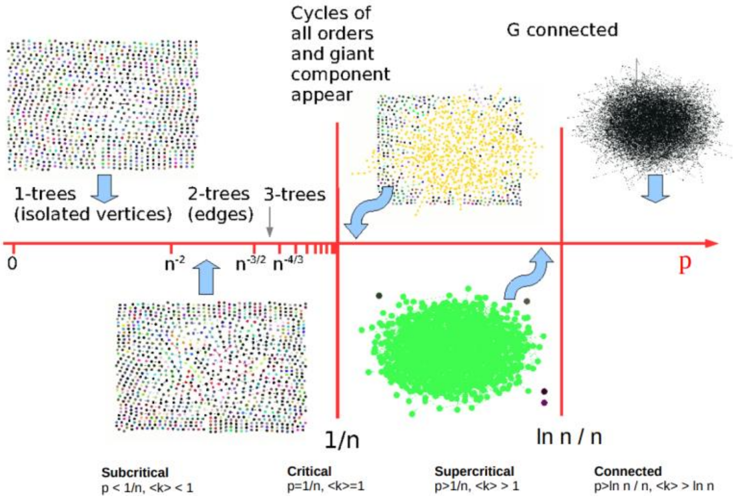
Suppose we have a square lattice where we place pebbles with probability p at each intersection



*Percolation theory is a branch of mathematics and statistical physics that studies what happens to a system when adding elements or connections probabilistically. Introduced for regular graphs, it can be extended to other network structures.

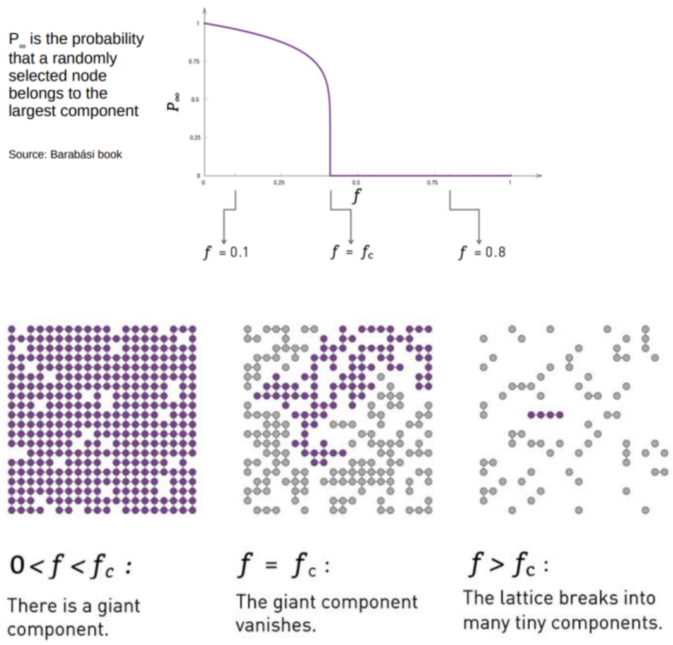
The higher is p, the larger are the clusters.
A key prediction of percolation theory is that the cluster size does not change gradually with p.
When p approaches a critical value p_c, the small clusters join, leading to the emergence of a large cluster at p_c.
At p_c we observe a phase transition, from tiny clusters to a “giant” cluster.

Recall: Evolution of a random network!



Inverse Percolation Theory*

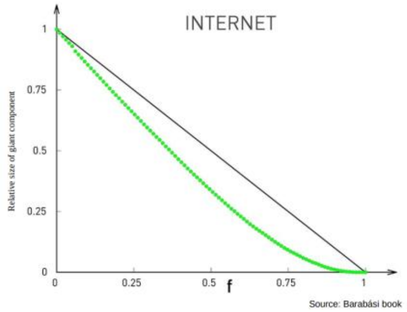
Inverse percolation theory can be used to describe the impact of node failures on the integrity of a network: randomly remove a fraction f of nodes, asking how their absence impacts the integrity of the lattice.



Also the fragmentation process is not gradual, but it is characterized by a critical threshold f_c.
The two processes can be mapped into each other by choosing f = 1 – p.

What happens if the underlying network is not as regular as a square lattice?
What happens if the network is scale-free?

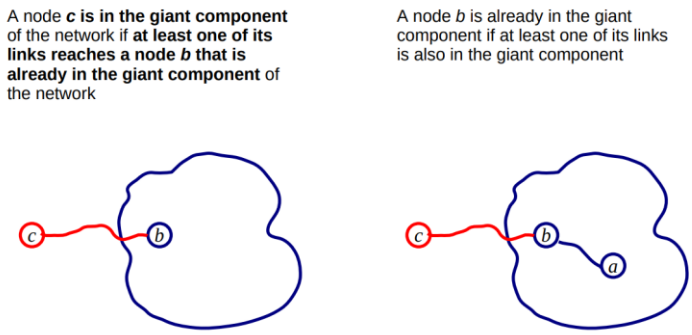
Robustness of scale-free networks



Simulation experiment: Random selection of nodes in the router-level map. The plots indicate that the Internet, and in general scale-free networks, do not fall apart after the removal of a finite fraction of nodes. We need to remove almost all nodes to fragment these networks (e.g., f_c = 1).

Why scale-free networks are so robust to random failures?
To understand the origin of the anomalously high f_c characterizing the Internet and scale-free networks, we can calculate f_c for a network with an arbitrary degree distribution.

Molloy-Reed Criterion



For a network to have a giant component, most nodes that belong to it must be connected to at least two other nodes.

Using a complex mathematical proof, it is possible to know the critical threshold f_c representing the fraction of nodes whose removal breaks the giant component of a network.

Molloy-Reed criterion: a random network has a giant component if and only if.

$$\boxed{\frac{\langle k^2 \rangle}{\langle k \rangle} > 2}$$

An important property of this finding is that the critical threshold depends only on the first and second moments of the degree distribution and is valid for any arbitrary degree distribution.

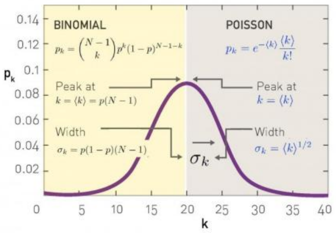
Molloy-Reed Criterion

For Erdős–Rényi (ER) graphs

- The degree distribution is binomial, approximated by Poisson in the large N
- In a Poisson distribution with mean λ we have ‹k› = λ and ‹k^2› = λ^2+λ

$$\frac{\langle k^2\rangle}{\langle k\rangle} = \frac{\lambda^2+\lambda}{\lambda} = \lambda+1$$

$$\frac{\langle k^2\rangle}{\langle k\rangle} > 2 \quad \Rightarrow \quad \lambda+1>2 \quad \Rightarrow \quad \lambda>1$$



- This is exactly the same as the Erdős–Rényi result, e.g., a giant component appears when k >1!

For networks with long-tail distributions ‹k^2› can be very large or even diverge meaning the condition $\frac{\langle k^2\rangle}{\langle k\rangle} > 2$ is much easier to satisfy .

Therefore, even sparse networks with low ‹k› can still have a giant component, a key reason why power law (specially scale-free) networks are so connected.

Critical threshold f_c

By applying the Molloy-Reed criteria the critical threshold f_c follows the formula.

$$\boxed{f_c = 1 - \frac{1}{\frac{\langle k^2\rangle}{\langle k\rangle} - 1}}$$

For ER graphs we have $\frac{\langle k^2\rangle}{\langle k\rangle} = 1 + \langle k\rangle$

$$\boxed{f_c^{ER} = 1 - \frac{1}{\langle k\rangle}}$$

Random graphs (ER)

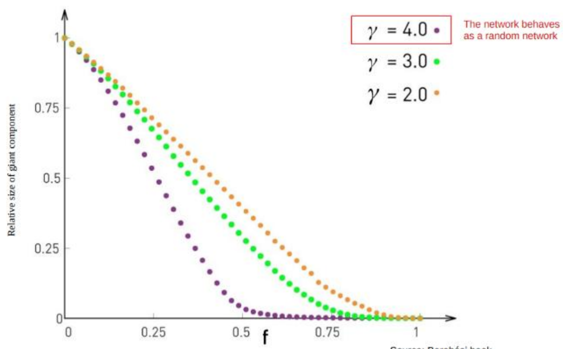
- The denser is a random network (e.g., the larger is ‹k›), the higher is its f_c, and hence the number of nodes to remove to break it apart
- Moreover, f_c is always smaller than 1, and hence the random network will break apart after the removal of a finite fraction of nodes

For power law distributions, we know that ‹k^2› diverges for γ < 3 in the N→∞ limit, hence

$$f_c = 1 - \frac{1}{\frac{\langle k^2\rangle}{\langle k\rangle} - 1}$$

converges to 1 for large scale-free networks.

In scale-free networks, the random removal of a finite fraction of the nodes does not break the network; to fragment a scale-free network we must remove all of its nodes.



Source: Barabási book

Is this robustness relevant also for finite networks?

A network displays enhanced robustness if its breakdown threshold f_c deviates from the random network (ER) prediction.

$$f_c > f_c^{ER}$$

The degree distribution does not need to follow a strict power law to display enhanced robustness, as long as ‹k^2› is larger than expected for a random network of similar size.

Estimated critical threshold f_c

Network	Random Failures (Real Network)	Random Failures (Randomized Network)
Internet	0.92	0.84
WWW	0.88	0.85
Power Grid	0.61	0.63
Mobile Phone Calls	0.78	0.68
Email	0.92	0.69
Science Collaboration	0.92	0.88
Actor Network	0.98	0.99
Citation Network	0.96	0.95
E. Coli Metabolism	0.96	0.90
Protein Interactions	0.88	0.66

Source: Barabási book

For most networks, for random failures f_c > f_c^{ER} indicating that these networks display enhanced robustness. The power grid lacks this property, a consequence of the fact that its degree distribution is exponential, and also the actor network, which has a very high ‹k›, diminishing the role of the high ‹k^2›.

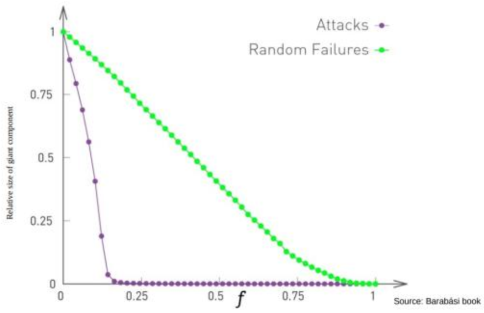
What if we do not remove the nodes randomly, but go after hub?

Scale-free network under attack

We first remove the highest degree node, followed by the node with the next highest degree, and so on…

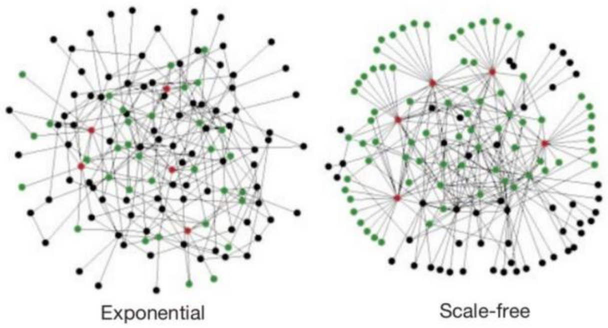
This process mimics an attack on the network, as it assumes a detailed knowledge of the network topology, an ability to target the hubs, and a desire to deliberately shut down the network.

The critical point (absent in random failures), re-emerges under attack. And it has a remarkably low value.



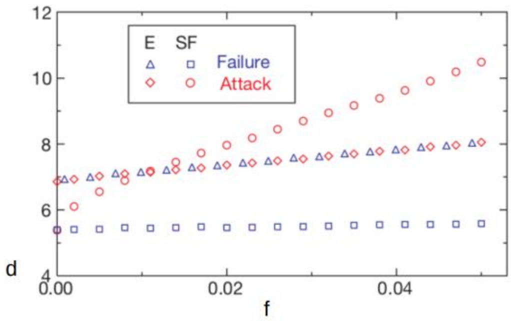
For an attack we remove the nodes in a decreasing order of their degrees: we start with the largest hub, followed by the next largest and so on. The removal of only a few hubs can disintegrate the network.

Barabási et al. experiment



The interconnectedness of a network is described by its diameter d, defined in this paper as the average length of the shortest paths between any two nodes in the network.
The diameter characterizes the ability of two nodes to communicate with each other: the smaller d is, the shorter is the expected path between them.

Changes in the diameter d of the network



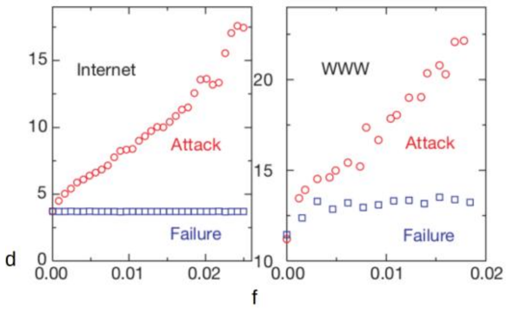
For the exponential network (e.g., ER) the diameter increases monotonically with f:

- This behavior is rooted in the homogeneity of the network
- Since all nodes have approximately the same number of links, they all contribute equally to the diameter of the network; the removal of any node causes the same amount of damage

For the scale-free network (SF) the diameter remains unchanged under an increasing level of errors:

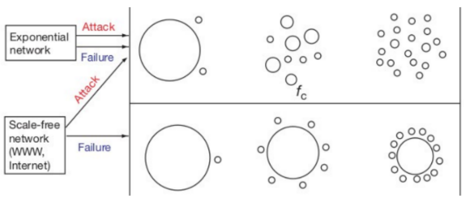
- Robustness is rooted in the inhomogeneous connectivity distribution

BUT
the behavior is different with an informed agent that attempts to deliberately damage a network targeting the most connected nodes.



Changes in the giant component (clusters)

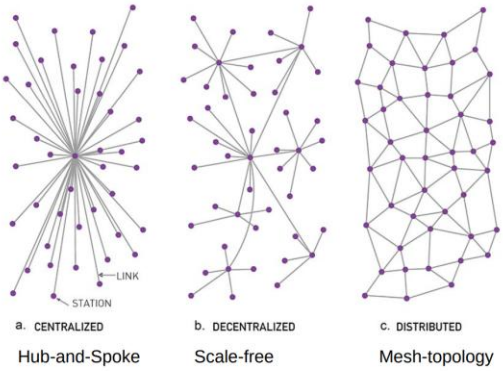
$$f_c^{e} \simeq 0.28$$
$$f_c^{sf} \simeq 0.18$$



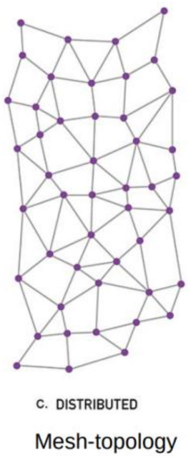
Achille’s heel of the Internet

While random node failures do not fragment a scale-free network, an attack that targets the hubs can easily destroy such a network.
This fragility is bad news for the Internet, as it indicates that it is inherently vulnerable to deliberate attacks.
It can be good news in medicine, as the vulnerability of bacteria to the removal of their hub proteins offers avenues to design drugs that kill unwanted bacteria.

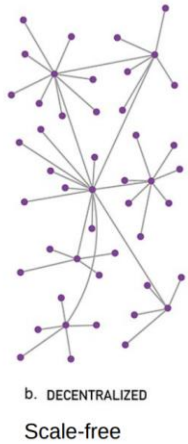
Paul Baran and the Internet (1959)



Ideal survivable architecture for Baran



Internet today



Take away points

Real-world networks are constantly under stress, facing potential disruptions like node failures or attacks.
To assess network robustness it is possible to see how the giant component changes: a network is more robust if:

- it can maintain a giant component even after losing some nodes or edges
- it shows a higher critical threshold f_c

Another possibility is analyzing the length of the diameter.
Random networks behave in the same way under random and target attacks:

- They have a critical threshold f_c: if a fraction of nodes larger than the threshold is removed, the networks break into tiny disconnected components

Scale-free networks behave differently under random and target attacks:

- For N → ∞, f_c is equal to 1 under random attacks, and the networks are extremely robust
- These same networks are very fragile under target attacks

Cascading failures

Modeling cascading failures

In real networks (systems) the activity of each node (component) depends on the activity of its neighborhood and the failure of a node can induce the failure of the connected nodes.
A cascading failure is the result of a positive feedback loop that occurs when change in one direction causes further changes in the same direction.
When a system is overloaded and it starts to return mostly errors, other components may respond in a manner that makes the problem worse and worse.

<u>Examples</u>
Power grids: A single power line outage can overload other lines, leading to cascading blackouts.
Computer networks: A failing router can overwhelm other routers, disrupting internet traffic for large areas.
Transportation systems: A road closure can divert traffic to other roads, causing traffic jam and gridlocks.
Financial markets: The failure of one company or bank can trigger a domino effect, causing widespread financial losses

Numerous models have been proposed to capture the dynamics of cascading events, all sharing the same key ingredients:

1) Initial failure: A node or link fails (e.g., overload, attack, malfunction)
2) Load redistribution: The network tries to compensate by rerouting loads or connections
3) Overloading of neighbors: The new load may exceed the capacity of neighboring nodes or links
4) Progressive Failures: More nodes/links fail, further stressing the network
5) Systemic Collapse: The entire system (or a large part) becomes nonfunctional (domino effect)

Cascading effects are observed in systems of rather different nature, and the distribution of the size of the cascade s is well approximated by a power law distribution, with avalanche exponent α

$$\boxed{p(s) \approx s^{-\alpha}}$$

Most cascades are too small to be noticed (for example local blackouts).
A few, however, are huge and have a global impact.

Avalanche exponents in real systems

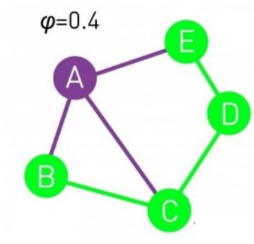
Source	Exponent	Cascade
Power grid (North America)	2.0	Power
Power grid (Sweden)	1.6	Energy
Power grid (Norway)	1.7	Power
Power grid (New Zealand)	1.6	Energy
Power grid (China)	1.8	Energy
Twitter Cascades	1.75	Retweets
Earthquakes	1.67	Seismic Wave

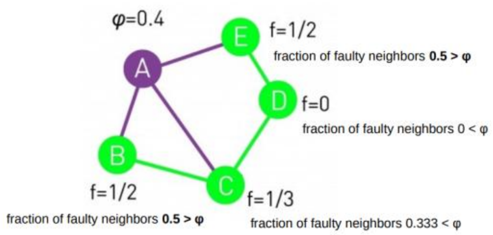
Two models

1) Failure Propagation Model (already seen in the lecture on Social Contagion)
2) Branching Model (close to what seen in the lecture on Epidemic Spreading)

1. Failure propagation model

Consider a network with an arbitrary degree distribution, where each node contains an "agent".
Agent i can be in the state 0 (active or healthy) or 1 (inactive or failed), and is characterized by a breakdown threshold φ_i = ϕ for all i.
All agents are initially in the healthy state 0.
At time t = 0 one agent switches to state 1, corresponding to an initial component failure or to the release of a new piece of information.





Depending on the local network topology, an initial perturbation can:

- die out immediately, failing to induce the failure of any other node
- lead to the failure of multiple nodes

Simulation experiments document three regimes with distinct avalanche characteristics.

Subcritical Regime

If ‹k› is high (dense network), changing the state of a node is unlikely to move other nodes over their threshold, as the healthy nodes have many healthy neighbors. In this regime cascades die out quickly

Supercritical Regime

If ‹k› is small (sparse network), flipping a single node can put several of its neighbors over the threshold, triggering a global cascade. In this regime perturbations induce major breakdowns.

Critical Regime

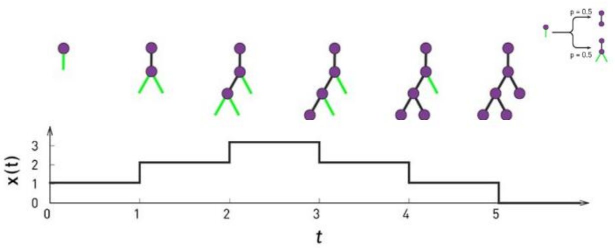
At the boundary of the subcritical and supercritical regime the avalanches have widely different sizes.

2. Branching model

This is the simplest model that still captures the basic features of a cascading event and it has been studied to calculate the expected size of the cascade.

Each cascading failure follows a branching process:

- The first node which fails becomes the root of a tree
- The branches of the tree are the nodes whose failure was triggered by this initial failure; often they are called offsprings
- Each active node produces k offsprings, where k is selected from a distribution p_k If the value selected for k is 0, the branch dies out



The size of the avalanche corresponds to the size of the tree when all active nodes dies out (e.g., x(t)=0).
The branching model predicts the same phases as those observed in the cascading failures model.

Subcritical Regime: ‹k› < 1
For ‹k› < 1 on average each branch has less then one offspring. Consequently each tree will terminate quickly.
Supercritical Regime: ‹k› > 1
For ‹k› > 1 on average each branch has more than one offspring. Consequently the tree will continue to grow indefinitely. Hence in this regime all avalanches are global.
Critical Regime: ‹k› = 1
For ‹k› = 1 on average each branch has exactly one offspring. Consequently some trees are large and others die out shortly. Numerical simulations indicate that in this regime the avalanche size distribution follows the power law.

If p_k is exponentially bounded, e.g., it has an exponential tail like random graphs, calculations predict α = 3/2.

If p_k is scale-free, e.g., it has a long tail, then the avalanche exponent depends on the power law exponent γ.

$$\alpha = \begin{cases} 3/2, & \gamma \geq 3 \\ \gamma/(\gamma - 1), & 2 < \gamma < 3 \end{cases}$$

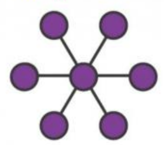
Avalanche exponents empirically found are all between 1.5 and 2, as predicted by the formula.

3. Motter and Lai model
Other models have been proposed in the literature, taking into account different parameters.
The paper by Motter and Lai defines the load of each node, L_i, as the number of shortest paths passing through it, e.g., the load L_i for node i is equal to its betweenness centrality.
Each node has also a capacity C_i, which is proportional to its initial load (a time 0): C_i = (1 + α) L_i^0
α ≥ 0 is the tolerance parameter.

For the cascade we can have:
- A node (or multiple) is removed due to a failure or an attack
- The shortest paths are recomputed, that is the loads Li are redistributed after the node removal
- If the new load on a node exceeds its capacity (L_i > C_i), the node fails
- This triggers new computations, possibly new overloads, and a cascading failure

Building robustness
Can we enhance a network's robustness? Designing networks that are simultaneously robust to attacks and random failures appears to be a conflicting desire.

Hub-and-Spoke networks



⟨k⟩ = 12 / 7

Hub-and-Spoke networks

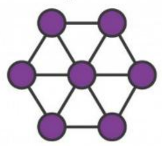
Tolerant to random failures
Fragile to target attacks

Hub-and-Spoke networks

We can reinforce the network by **adding redundancy** and connect peripheral nodes

A **higher <k>** can be considered a **cost** to build and maintain the network

Hub-and-Spoke networks



⟨k⟩ = 24 / 7

A network that is robust to both random failures and attacks has a single hub, and the rest of the nodes have the same degree.
A combination of simulation and analytical results showed that with N nodes the highest robustness is achieved when K_{max} is around $N^{2/3}$.
The removal of the K_{max} hub causes a major one-time loss, the remaining low degree nodes are robust against subsequent targeted removal since they form a giant component on their own.

Building robustness (Motter)
Can we avoid cascading failures? The first instinct is to reinforce the network by adding new links but in a counter-intuitive fashion, the impact of cascading failures can be reduced through selective node and link removal.
Each cascading failure has two parts

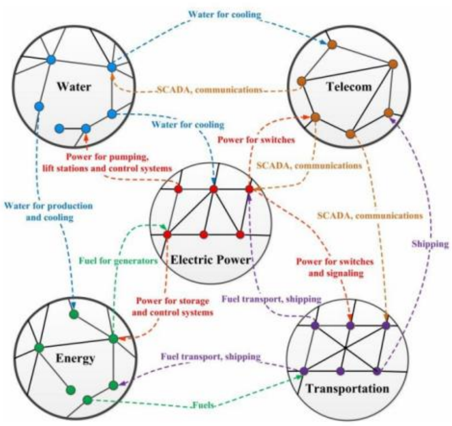
- (i) Initial failure
- (ii) Propagation
  Simulations indicate that the size of a cascade can be reduced if we intentionally remove additional nodes or edges right after the initial failure, but before the failure could propagate

Motter suggests to remove nodes that have low load but generate a lot of load:

- Find nodes with small load L_i (not central themselves) but that generate many paths, their presence causes load elsewhere in the network
- To estimate the load generated by each node i it is possible to count number of shortest paths where node i is a source or target (not just intermediate)
- These nodes are not vital by themselves, but contribute indirectly to overloads elsewhere

In summary
A better understanding of the network topology is essential to improve the robustness of complex systems.
We can enhance robustness by either designing networks that are simultaneously robust to both random failures and attacks, or by interventions that limit the spread of cascading failures.
Does these interventions remind you of something?
These results may suggest that we should redesign the topology of infrastructural networks. But they were built incrementally over decades, following a self-organized growth and it is unlikely that we would ever be given a chance to rebuild them.

Things are much more complex...



Diverse infrastructures such as water supply, transportation, fuel and power stations are coupled together and depend on each other for functioning.

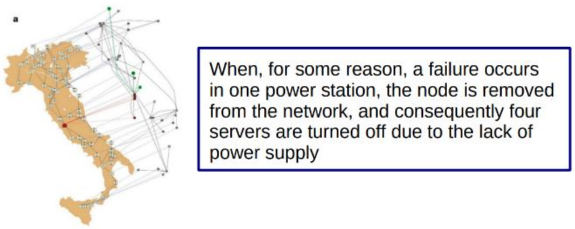
Interdependent networks are extremely sensitive to random failures, and in particular to targeted attacks, such that a failure of a small fraction of nodes in one network can trigger an iterative cascade of failures in several interdependent networks.

Single network robustness
Network robustness (see previous lecture) allows to study how the size of the giant component is changed when a fraction f of nodes (or links) is removed.

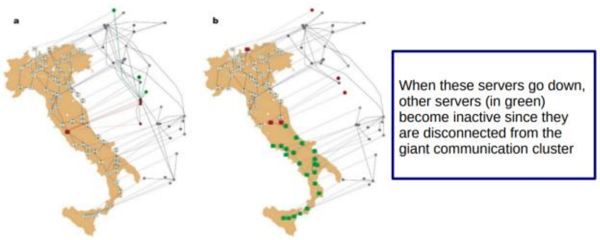
Interdependent networks robustness
What if we have 2 networks, A and B and:
- Every A-node depends on a B-node, and vice versa
- The dependency is such that coupled nodes are only active if both are connected to the giant component of their network
- Network A and network B have degree distributions p_A(k) and p_B(k) respectively
- Dependency links between networks connect random pairs (A_i, B_i), with the constraint of only one inter-network link per node

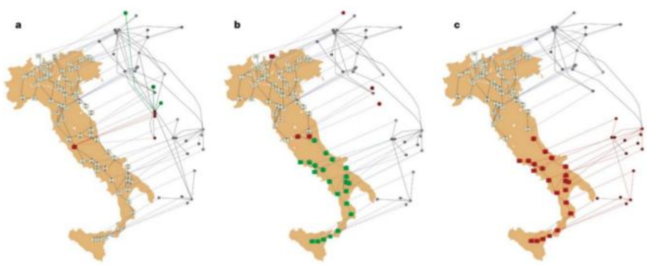
Network robustness can be studied by removing nodes in network A and the coupled nodes in network B:
- When a fraction f is removed, a phase transition occurs at a certain threshold f_c
- This threshold f_c is lower than the one for a single network, revealing a significant increase in vulnerability due to the coupling

Things are much more complex...
A real-world example of a cascade of failures (concurrent malfunction) is the electrical blackout that affected much of Italy on 28 September 2003: the shutdown of power stations directly led to the failure of nodes in the Internet communication network, which in turn caused further breakdown of power stations.



Over the map is the network of the Italian power network and, slightly shifted to the top, is the communication network that controls the power distribution. Every server was considered to be connected to the geographically nearest power station.





A sequence of events takes place, in a cascade fashion, which shows how a fail in a single power station can lead to a cascade of events ending in a blackout spanning over more than half of the system.

Failures in network of networks

Key factors influencing cascading failures:

- **Interdependence:** The more interconnected the elements in a network are, the more susceptible it is to cascading failures. A failure in one system can easily propagate to other dependent systems
- **Lack of redundancy:** If there are no backup systems or alternative pathways for data or resources to flow, a failure in one area can quickly become widespread
- **Threshold effects:** Sometimes, failures only occur when a certain level of stress is exceeded. If a cascading failure pushes a critical element beyond its threshold, it can fail catastrophically

Take away points

Malloy-Reed criteria, a giant component exists if $\frac{\langle k^2 \rangle}{\langle k \rangle} > 2$

Random failures: $f_c = 1 - \frac{1}{\frac{\langle k^2 \rangle}{\langle k \rangle} - 1}$

Random networks: $f_c^{ER} = 1 - \frac{1}{\langle k \rangle}$

Scale-free networks (not discussed in class):

$$f_c = \begin{cases} 1 - \frac{1}{\frac{\gamma-2}{3-\gamma} k_{\min}^{\gamma-2} k_{\max}^{3-\gamma} - 1} & 2 < \gamma < 3 \\ 1 - \frac{1}{\frac{\gamma-2}{\gamma-3} k_{\min} - 1} & \gamma > 3 \end{cases}$$

Enhanced robustness: $f_c > f_c^{ER}$

Cascading failures: $p(s) \approx s^{-\alpha}$

$$\alpha = \begin{cases} 3/2, & \gamma \geq 3 \\ \gamma/(\gamma - 1), & 2 < \gamma < 3 \end{cases}$$

Networks of networks are more vulnerable due to their coupling